

16. Assertions

16.1 General

This clause describes the following:

- Immediate assertions
- Concurrent assertions
- Sequence specifications
- Property specifications

16.2 Overview

An assertion specifies a behavior of the system. Assertions are primarily used to validate the behavior of a design. In addition, assertions can be used to provide functional coverage and to flag that input stimulus, which is used for validation, does not conform to assumed requirements.

An assertion appears as an assertion statement that states the verification function to be performed. The statement shall be of one of the following kinds:

- **assert**, to specify the property as an obligation for the design that is to be checked to verify that the property holds.
- **assume**, to specify the property as an assumption for the environment. Simulators check that the property holds, while formal tools use the information to generate input stimulus.
- **cover**, to monitor the property evaluation for coverage.
- **restrict**, to specify the property as a constraint on formal verification computations. Simulators do not check the property.

There are two kinds of assertions: *concurrent* and *immediate*.

- Immediate assertions follow simulation event semantics for their execution and are executed like a statement in a procedural block. Immediate assertions are primarily intended to be used with simulation. There is no immediate **restrict** assertion statement.
- Concurrent assertions are based on clock semantics and use sampled values of their expressions (see [16.5.1](#)). One of the goals of SystemVerilog assertions is to provide a common semantic meaning for assertions so that they can be used to drive various design and verification tools. Many tools, such as formal verification tools, evaluate circuit descriptions using cycle-based semantics, which typically rely on a clock signal or signals to drive the evaluation of the circuit. Any timing or event behavior between clock edges is abstracted away. Concurrent assertions incorporate this clock semantics. While this approach generally simplifies the evaluation of a circuit description, there are a number of scenarios under which this cycle-based evaluation provides different behavior from the standard event-based evaluation of SystemVerilog.

This clause describes both types of assertions.

16.3 Immediate assertions

The immediate assertion statement is a test of an expression performed when the statement is executed in the procedural code. The expression is nontemporal and is interpreted the same way as an expression in the condition of a procedural **if** statement. In other words, if the expression evaluates to **X**, **Z**, or **0**, then it is interpreted as being false, and the assertion statement is said to fail. Otherwise, the expression is interpreted as being true, and the assertion statement is said to pass or, equivalently, to succeed.

There are two modes of immediate assertions, *simple immediate assertions* and *deferred immediate assertions*. In a simple immediate assertion, pass and fail actions take place immediately upon assertion evaluation. In a deferred immediate assertion, the actions are delayed until later in the time step, providing some level of protection against unintended multiple executions on transient or “glitch” values. Deferred immediate assertions are described in detail in [16.4](#).

The *immediate_assertion_statement* is a *statement_item* and can be specified anywhere a procedural statement is specified. The execution of immediate assertions can be controlled by using assertion control system tasks (see [20.12](#)).

```

procedural_assertion_statement ::=                                     //from A.6.10
    ...
    | immediate_assertion_statement
    ...
immediate_assertion_statement ::=
    simple_immediate_assertion_statement
    | deferred_immediate_assertion_statement
simple_immediate_assertion_statement ::=
    simple_immediate_assert_statement
    | simple_immediate_assume_statement
    | simple_immediate_cover_statement
simple_immediate_assert_statement ::=
    assert ( expression ) action_block
simple_immediate_assume_statement ::=
    assume ( expression ) action_block
simple_immediate_cover_statement ::=
    cover ( expression ) statement_or_null
deferred_immediate_assertion_item ::= [ block_identifier : ] deferred_immediate_assertion_statement
deferred_immediate_assertion_statement ::=
    deferred_immediate_assert_statement
    | deferred_immediate_assume_statement
    | deferred_immediate_cover_statement
deferred_immediate_assert_statement ::=
    assert #0 ( expression ) action_block
    | assert final ( expression ) action_block
deferred_immediate_assume_statement ::=
    assume #0 ( expression ) action_block
    | assume final ( expression ) action_block
deferred_immediate_cover_statement ::=
    cover #0 ( expression ) statement_or_null
    | cover final ( expression ) statement_or_null
action_block ::=                                                     //from A.6.3
    statement_or_null
    | [ statement ] else statement_or_null

```

Syntax 16-1—Immediate assertion syntax (excerpt from [Annex A](#))

An immediate assertion statement may be an immediate **assert**, an immediate **assume**, or an immediate **cover**.

The immediate **assert** statement specifies that its expression is required to hold. Failure of an immediate **assert** statement indicates a violation of the requirement and thus a potential error in the design. If an **assert** statement fails and no **else** clause is specified, the tool shall, by default, call `$error`, unless `$assertcontrol` with `control_type` 9 (FailOff) is used to suppress the failure.

The immediate **assume** statement specifies that its expression is assumed to hold. For example, immediate **assume** statements can be used with formal verification tools to specify assumptions on design inputs that constrain the verification computation. When used in this way, they specify the expected behavior of the environment of the design as opposed to that of the design itself. In simulation, an immediate **assume** may behave as an immediate **assert** to verify that the environment behaves as assumed. A simulation tool shall provide the capability to check the immediate **assume** statement in this way.

The *action_block* of an immediate **assert** or **assume** statement specifies what actions are taken upon success or failure of the assertion. The statement associated with success is the first statement. It is called the *pass statement* and shall be executed if the *expression* evaluates to true. The pass statement can, for example, record the number of successes for a coverage log, but can be omitted altogether. If the pass statement is omitted, then no user-specified action is taken when the assert *expression* of the immediate **assert** or **assume** statement is true. The statement associated with **else** is called the *fail statement* and shall be executed if the *expression* evaluates to false. The **else** statement can also be omitted. The action block shall be enabled to execute immediately after the evaluation of the assert *expression* of the immediate **assert** or **assume** statement. The execution of pass and fail statements can be controlled by using assertion action control tasks. The assertion action control tasks are described in [20.12](#).

The immediate **cover** statement specifies that successful evaluation of its *expression* is a coverage goal. Tools shall collect coverage information and report the results at the end of simulation or on demand via an assertion API (see [Clause 39](#)). The results of coverage for an immediate **cover** statement shall contain the following:

- Number of times evaluated
- Number of times succeeded

A pass statement for an immediate **cover** may be specified in *statement_or_null*. The pass statement shall be executed if the expression evaluates to true. The pass statement shall be enabled to execute immediately after the evaluation of the expression of the immediate **cover**.

The optional statement label (identifier and colon) creates a named block around the assertion statement (or any other statement), and the hierarchical name of the scope can be displayed using the `%m` format specification.

The information about assertion failure can be printed using one of the following severity system tasks in the action block:

- `$fatal` is a run-time fatal.
- `$error` is a run-time error.
- `$warning` is a run-time warning.
- `$info` indicates that the assertion failure carries no specific severity.

The syntax for these severity system tasks is shown in [20.10](#).

The severity system tasks can be used in assertion pass or fail statements. These tasks shall print the same tool-specific message when used either in a pass or a fail statement. For example:

```
assert_f: assert (f) $info("passed"); else $error("failed");  
  
assume_inputs: assume (in_a || in_b) $info("assumption holds");
```

```
else $error("assumption does not hold");

cover_a_and_b: cover (in_a && in_b) $info("in_a && in_b == 1 covered");
```

For example, a formal verification tool might prove `assert_f` under the assumption `assume_inputs` expressing the condition that `in_a` and `in_b` are not both 0 at the same time. The `cover` statement detects whether `in_a` and `in_b` are both simultaneously 1.

If more than one of these system tasks is included in the action block, then each shall be executed as specified.

If the severity system task is executed at a time other than when the immediate `assert` or `assume` fails, the actual failure time of the immediate `assert` or `assume` can be recorded and displayed programmatically. For example:

```
time t;

always @(posedge clk)
  if (state == REQ)
    assert (req1 || req2)
    else begin
      t = $time;
      #5 $error("assert failed at time %0t",t);
    end
```

If the immediate `assert` fails at time 10, the error message shall be printed at time 15, but the user-defined string printed will be “assert failed at time 10.”

Because the fail statement, like the pass statement, is any legal SystemVerilog procedural statement, it can also be used to signal a failure to another part of the testbench.

```
assert (myfunc(a,b)) count1 = count + 1; else ->event1;
assert (y == 0) else flag = 1;
```

16.4 Deferred assertions

```
immediate_assertion_statement ::= // from A.6.10
...
| deferred_immediate_assertion_statement
deferred_immediate_assertion_item ::= [ block_identifier : ] deferred_immediate_assertion_statement
deferred_immediate_assertion_statement ::=
  deferred_immediate_assert_statement
  | deferred_immediate_assume_statement
  | deferred_immediate_cover_statement
deferred_immediate_assert_statement ::=
  assert #0 ( expression ) action_block
  | assert final ( expression ) action_block
deferred_immediate_assume_statement ::=
  assume #0 ( expression ) action_block
  | assume final ( expression ) action_block
```

```
deferred_immediate_cover_statement ::=
    cover #0 ( expression ) statement_or_null
    | cover final ( expression ) statement_or_null
```

Syntax 16-2—Deferred immediate assertion syntax (excerpt from [Annex A](#))

Deferred assertions are a kind of immediate assertion. They can be used to suppress false reports that occur due to glitching activity on combinational inputs to immediate assertions. Since deferred assertions are a subset of immediate assertions, the term *deferred assertion* (often used for brevity) is equivalent to the term *deferred immediate assertion*. The term *simple immediate assertion* refers to an immediate assertion that is not deferred. In addition, there are two different kinds of deferred assertions: *observed deferred immediate assertions* and *final deferred immediate assertions*.

A deferred assertion is similar to a simple immediate assertion, but with the following key differences:

- Syntax: Deferred assertions use #0 (for an observed deferred assertion) or **final** (for a final deferred assertion) after the verification directive.
- Deferral: Reporting is delayed rather than being reported immediately.
- Action block limitations: Action blocks may only contain a single subroutine call.
- Use outside procedures: A deferred assertion may be used as a *module_common_item*.

Deferred assertion syntax is similar to simple immediate assertion syntax, with the difference being the specification of a #0 or **final** after the **assert**, **assume**, or **cover**:

```
assert #0 (expression) action_block
assert final (expression) action_block
```

As with all immediate assertions, a deferred assertion's *expression* is evaluated at the time the deferred assertion statement is processed. However, in order to facilitate glitch avoidance, the reporting or action blocks are scheduled at a later point in the current time step.

The pass and fail statements in a deferred assertion's *action_block*, if present, shall each consist of a single subroutine call. The subroutine can be a task, task method, void function, void function method, or system task. The requirement of a single subroutine call implies that no begin-end block shall surround the pass or fail statements, as **begin** is itself a statement that is not a subroutine call. In the case of a final deferred assertion, the subroutine shall be one that may be legally called in the Postponed region (see [4.4.2.9](#)). A subroutine argument may be passed by value as an input or passed by reference as a **ref** or **const ref**. Actual argument expressions that are passed by value, including function calls, shall be fully evaluated at the instant the deferred assertion expression is evaluated. It shall be an error to pass automatic or dynamic variables as actuals to a **ref** or **const ref** formal. The processing of the action block differs between observed and final deferred assertions as follows:

- For an observed deferred assertion, the subroutine shall be scheduled in the Reactive region. Actual argument expressions that are passed by reference use or assign the current values of the underlying variables in the Reactive region.
- For a final deferred assertion, the subroutine shall be scheduled in the Postponed region. Actual argument expressions that are passed by reference use the current values of the underlying variables in the Postponed region.

Deferred assertions may also be used outside procedural code, as a *module_common_item*. This is explained in more detail in [16.4.3](#).

In addition to deferred **assert** statements, deferred **assume** and **cover** statements are also defined. Other than the deferred evaluation as described in this subclause, these **assume** and **cover** statements behave the

same way as the simple immediate **assume** and **cover** statements described in [16.3](#). A deferred **assume** will often be useful in cases where a combinational condition is checked in a function, but needs to be used as an assumption rather than a proof target by formal tools. A deferred **cover** is useful to avoid crediting tests for covering a condition that is only met in passing by glitched values.

16.4.1 Deferred assertion reporting

When a deferred assertion passes or fails, the action block is not executed immediately. Instead, the action block subroutine call (or `$error`, if an **assert** or **assume** fails and no *action_block* is present) and the current values of its input arguments are placed in a *deferred assertion report queue* associated with the currently executing process. Such a call is said to be a *pending assertion report*.

If a *deferred assertion flush point* (see [16.4.2](#)) is reached in a process, its deferred assertion report queue is cleared. Any pending assertion reports will not be executed.

In the Observed region of each simulation time step, each pending observed deferred assertion report that has not been flushed from its queue shall *mature*, or be confirmed for reporting. Once a report matures, it may no longer be flushed. Then the associated subroutine call (or `$error`, if the assertion fails and no action block is present) is executed in the Reactive region, and the pending assertion report is cleared from the appropriate process's deferred assertion report queue.

Note that if code in the Reactive region modifies signals and causes another pass to the Active region to occur, this still may create glitching behavior in observed deferred assertions, as the new passage in the Active region may re-execute some of the deferred assertions with different reported results. In general, observed deferred assertions prevent glitches due to order of procedural execution, but do not prevent glitches caused by execution loops between regions that the assignments from the Reactive region may cause.

In the Postponed region of each simulation time step, each pending final deferred assertion report that has not been flushed from its queue shall mature. Then the associated subroutine call (or `$error`, if the assertion fails and no action block is present) is scheduled in the same Postponed region, and the pending assertion report is cleared from the appropriate process's deferred assertion report queue. Due to their execution in the non-iterative Postponed region, final deferred assertions are not vulnerable to the potential glitch behavior previously described for observed deferred assertions.

16.4.2 Deferred assertion flush points

A process is defined to have reached a deferred assertion flush point if any of the following occur:

- The process, having been suspended earlier due to reaching an event control or wait statement, resumes execution.
- The process was declared by an **always_comb** or **always_latch**, and its execution is resumed due to a transition on one of its dependent signals.
- The outermost scope of the process is disabled by a **disable** statement (see [16.4.4](#))

The following example shows how deferred assertions might be used to avoid undesired reports of a failure due to transitional combinational values in a single simulation time step:

```
assign not_a = !a;
always_comb begin : b1
    a1: assert (not_a != a);
    a2: assert #0 (not_a != a); // Should pass once values have settled
end
```

When *a* changes, a simulator could evaluate assertions *a1* and *a2* twice—once for the change in *a* and once for the change in *not_a* after the evaluation of the continuous assignment. A failure could thus be reported during the first execution of *a1*. The failure during the first execution of *a2* will be scheduled on the process's deferred assertion report queue. When *not_a* changes, the deferred assertion queue is flushed due to the activation of *b1*, so no failure of *a2* will be reported.

This example illustrates the behavior of deferred assertions in the presence of time delays:

```
always @(a or b) begin : b1
  a3: assert #0 (a == b) rptobj.success(0); else rptobj.error(0, a, b);
  #1;
  a4: assert #0 (a == b) rptobj.success(1); else rptobj.error(1, a, b);
end
```

In this case, due to the time delay in the middle of the procedure, an Observed region will always be reached after the execution of *a3* and before a flush point. Thus any passes or failures of *a3* will always be reported. For *a4*, during cycles where either *a* or *b* changes after it has been executed, failures will be flushed and never reported. In general, deferred assertions must be used carefully when mixed with time delays.

The following example illustrates a typical use of a deferred **cover** statement:

```
assign a = ...;
assign b = ...;
always_comb begin : b1
  c1: cover (b != a);
  c2: cover #0 (b != a);
end
```

In this example, it is important to make sure some test is covering the case where *a* and *b* have different values. Due to the arbitrary order of the assignments in the simulator, it might be the case that in a cycle where both variables are being assigned the same value, *b1* executes while *a* has been assigned but *b* still holds its previous value. Thus *c1* will be triggered, but this is actually a glitch, and probably not a useful piece of coverage information. In the case of *c2*, this coverage will get added to the deferred report queue, but when *b1* is executed the next time (after *b* has also been assigned its new value), that coverage point will be flushed, and *c2* will correctly not get reported as having been covered during that time step.

The next example illustrates a case where, due to short-circuiting (see [11.3.5](#)), the result of a deferred assertion may not appear at first glance to be consistent with the signal values at the end of a time step.

```
function f(bit v);
  p: assert #0 (v);
  ...
endfunction
always_comb begin: myblk
  a = b || f(c);
end
```

Suppose, during some time step, the following sequence of events occurs:

- *b* is set to 0 while *c*==1, and *myblk* is entered. When *f* is called, assertion *p* has a passing value.
- Later in the time step *b* settles at a value of 1, while *c* becomes 0. When the procedure resumes, the previous execution is flushed. This time, due to short-circuiting, *f* is never evaluated—so the new failing value of assertion *p* is never seen.
- In the Reactive region, no passing or failing execution is reported by the simulator on *p*.

NOTE—If the bitwise **|** operator, which does not allow short-circuiting, were used instead of **||** in the assignment to *a*, then *f* would be evaluated each time the assignment was reached.

The following example illustrates the evaluation of subroutine arguments to deferred assertion action blocks.

```
function int error_type (int opcode);
  func_assert: assert (opcode < 64) else $display("Opcode error.");
  if (opcode < 32)
    return (0);
  else
    return (1);
endfunction

always_comb begin : b1
  a1: assert #0 (my_cond) else
    $error("Error on operation of type %d\n", error_type(opcode));
  a2: assert #0 (my_cond) else
    error_type(opcode);
  ...
end
```

Suppose block b1 is executed twice in the Active region of a single time step, with `my_cond == 0`, so it fails assertions a1 and a2 both times. Also suppose `opcode` is 64 the first time it is executed, and 0 the second time. The following will occur during simulation:

- Upon each deferred assertion failure, the subroutine arguments of the action block are evaluated, even though the action block itself is not executed.
 - Upon the first failure of a1, the arguments of `$error` are examined. Since the second argument contains a function call, that function (`error_type(opcode)`, with `opcode=64`) is evaluated. During this function call, `func_assert` fails and displays the message “Opcode error.”
 - Upon the first failure of a2, the arguments of `error_type` are examined. Since its only argument is the expression `opcode`, its value 64 is used and no further evaluation is needed at this time.
 - The pending reports with `opcode=64` are placed on the deferred assertion report queue.
- When block b1 is executed again, the pending reports are flushed from the deferred assertion report queue.
 - Upon the second failure of a1, function `error_type` is called with `opcode==0`, so assertion `func_assert` passes.
 - Upon the second failure of a2, the value of 0 is used for the expression `opcode`, and no further evaluation is needed at this time.
- When the assertions later mature, the `$error` task will be called for a1, and the function `error_type` will be called for a2.

The deferral and flushing prevented a report from the first failure of a1 as expected. But the evaluation of action block subroutine arguments, which happens every time a pending assertion report is queued, caused a function to be called upon each failure. In general, users must be cautious about the contents of action blocks for deferred assertions, since the evaluation of their subroutine arguments on every failure may seem inconsistent with the deferral in some usages.

The following example illustrates the differences between observed deferred assertions and final deferred assertions.

```
module dut(input logic clk, input logic a, input logic b);
  logic c;
  always_ff @(posedge clk)
    c <= b;
  a1: assert #0 (!(a & c)) $display("Pass"); else $display("Fail");
endmodule
```

```

    a2: assert final (!(a & c)) $display("Pass"); else $display("Fail");
endmodule

program tb(input logic clk, output logic a, output logic b);
    default clocking m @(posedge clk);
        default input #0;
        default output #0;
        output a;
        output b;
    endclocking

    initial begin
        a = 1;
        b = 0;
        ##10;
        b = 1;
        ##1;
        a = 0;
    end
endprogram

module sva_svtb;
    bit clk;
    logic a, b;
    ...
    dut dut (.*);
    tb tb (.*);
endmodule

```

In the 11th clock cycle, observed deferred assertion `a1` will first execute in the Active region, and it will fail since at this point `a` and `c` are both 1. This pending assertion report will mature in the Observed region, and the failure report will be scheduled in the Reactive region. However, in the Reactive region of the same time step, the testbench will set `a` to 0, triggering another execution of the implied **always_comb** block containing assertion `a1`. This time `a1` will pass. So both a pass and a fail message will be displayed for `a1` during this time step.

For final deferred assertion `a2`, the behavior will be different. As with `a1`, a pending assertion report will be generated when the assertion fails in the Active region. However, when the value of `a` changes in the Reactive region and the assertion's implicit **always_comb** is resumed, this creates a flush point, so this pending report will be flushed. `a2` will be executed again with the new value, and the new result will be put on the deferred assertion report queue. In the Postponed region, this will mature, and the final passing result of this assertion will be the only one reported.

16.4.3 Deferred assertions outside procedural code

A deferred assertion statement may also appear outside procedural code, in which case it is referred to as a *static deferred assertion*. In such cases, it is treated as if it were contained in an **always_comb** procedure. For example:

```

module m (input a, b);
    a1: assert #0 (a == b);
endmodule

```

This is equivalent to the following:

```

module m (input a, b);
    always_comb begin

```

```

    a1: assert #0 (a == b);
end
endmodule

```

Static deferred assertions in checkers are described in [17.3](#).

16.4.4 Disabling deferred assertions

The **disable** statement shall interact with deferred assertions as follows:

- A specific deferred assertion may be disabled. Any pending assertion reports for that assertion are cancelled.
- When a **disable** is applied to the outermost scope of a procedure that has an active deferred assertion queue, in addition to normal disable activities (see [9.6.2](#)), the deferred assertion report queue is flushed and all pending assertion reports on the queue are cleared.

Disabling a task or a non-outermost scope of a procedure does not cause flushing of any pending reports.

The following example illustrates how user code can explicitly flush a pending assertion report. In this case, failures of `a1` are only reported in time steps where `bad_val_ok` does not settle at a value of 1.

```

always @(bad_val or bad_val_ok) begin : b1
    a1: assert #0 (bad_val) else $fatal(1, "Sorry");
    if (bad_val_ok) begin
        disable a1;
    end
end

```

The following example illustrates how user code can explicitly flush all pending assertion reports on the deferred assertion queue of process `b2`:

```

always @(a or b or c) begin : b2
    if (c == 8'hff) begin
        a2: assert #0 (a && b);
    end else begin
        a3: assert #0 (a || b);
    end
end

always @(clear_b2) begin : b3
    disable b2;
end

```

16.4.5 Deferred assertions and multiple processes

As described in the previous subclauses, deferred assertions are inherently associated with the process in which they are executed. This means that a deferred assertion within a function may be executed several times due to the function being called by several different processes, and each of these different process executions is independent. The following example illustrates this situation:

```

module fsm(...);
    function bit f (int a, int b)
        ...
        a1: assert #0 (a == b);
        ...
    endfunction
    ...

```

```

always_comb begin : b1
    some_stuff = f(x,y) ? ...
    ...
end
always_comb begin : b2
    other_stuff = f(z,w) ? ...
    ...
end
endmodule

```

In this case, there are two different processes that may call assertion a1: b1 and b2. Suppose simulation executes the following scenario in the first passage through the Active region of each time step:

- In time step 1, b1 executes with $x \neq y$, and b2 executes with $z \neq w$.
- In time step 2, b1 executes with $x \neq y$, then again with $x = y$.
- In time step 3, b1 executes with $x \neq y$, then b2 executes with $z = w$.

In the first time step, since a1 fails independently for processes b1 and b2, its failure is reported twice.

In the second time step, the failure of a1 in process b1 is flushed when the process is re-triggered, and since the final execution passes, no failure is reported.

In the third time step, the failure in process b1 does not see a flush point, so that failure is reported. In process b2, the assertion passes, so no failure is reported from that process.

16.5 Concurrent assertions overview

Concurrent assertions describe behavior that spans over time. Unlike immediate assertions, the evaluation model is based on a clock so that a concurrent assertion is evaluated only at the occurrence of a clock tick. The term *clock tick* refers to a time step when a clocking event of a sequence, property, sampled value function, or assertion statement occurs. Due to the need to verify proper behavior of the system and conform as closely as possible to cycle-based semantics, the clocking event should be glitch-free and only transition once during any time step. If the clocking event transitions more than once during a time step, the resulting behavior is undefined.

Concurrent assertions use the sampled values of their expressions except for disable conditions (see [16.15](#)) and clocking events. Expression sampling is explained in [16.5.1](#). Concurrent assertions are evaluated in the Observed region.

16.5.1 Sampling

Concurrent assertions and several other constructs (such as variables referenced in an **always_ff** procedure in a checker, see [17.5](#)) have special rules for sampling values of their expressions. The value of an expression sampled in one of these constructs is called a *sampled value*. In most cases the sampled value of an expression is its value in the Preponed region. This rule has, however, several important exceptions. The rest of this subclause provides the formal definition of sampling.

The default sampled value of an expression is defined as follows:

- The default sampled value of a static variable is the value assigned in its declaration, or, in the absence of such an assignment, it is the default (or uninitialized) value of the corresponding type (see [6.8](#), [Table 6-7](#)).
- The default sampled value of any other variable or net is the default value of the corresponding type (see [6.8](#), [Table 6-7](#)). For example, the default sampled value of variable *y* of type **logic** is 1'b $\times$.

- The default sampled value of the `triggered` event method (see [15.5.3](#)) and the sequence methods `triggered` and `matched` is false (`1'b0`).
- The default sampled value of an expression is defined recursively by evaluating the expression using the default sampled values of its component subexpressions and variables.

A default sampled value is used in the definition of a sampled value of an expression as explained below, and in the definition of sampled value functions when there is a need to reference a sampled value of an expression before time zero (see [16.9.3](#)).

The definition of a sampled value of an expression is based on the definition of a sampled value of a variable. The general rule for variable sampling is as follows:

- The sampled value of a variable in a time slot corresponding to time greater than 0 is the value of this variable in the Preponed region of this time slot.
- The sampled value of a variable in a time slot corresponding to time 0 is its default sampled value.

This rule has the following exceptions:

- Sampled values of automatic variables (see [16.14.6](#)), local variables (see [16.10](#)), and active free checker variables (see [17.7.2](#)) are their current values. However,
 - When a past or a future value of an active free checker variable is referenced by a sampled value function (see [16.9.3](#) and [16.9.4](#)), this value is sampled in the Postponed region of the corresponding past or future clock tick;
 - When a past or a future value of an automatic variable is referenced by a sampled value function, the current value of the automatic variable is taken instead.
- If a variable is an input variable of a clocking block, the variable shall be sampled by the clocking block with `#1step` sampling. Any other type of sampling for the clocking block variable shall result in an error. The sampled value of a such variable is the sampled value produced by the clocking block. This is explained in [Clause 14](#).

The sampled value of an expression is defined as follows:

- The sampled value of an expression consisting of a single variable is the sampled value of this variable.
- The sampled value of a `const` cast expression (see [6.24.1](#) and [16.14.6](#)) is defined as the current value of its argument. For example, if `a` is a variable, then the sampled value of `const'(a)` is the current value of `a`. When a past or a future value of a `const` cast expression is referenced by a sampled value function, the current value of this expression is taken instead.
- The sampled value of the `triggered` event method and the sequence methods `triggered` and `matched` (see [16.13.6](#)) is defined as the current value returned by the event property or sequence method. When a past or a future value of an event property or sequence method is referenced by a sampled value function (see [16.9.3](#) and [16.9.4](#)), this value is sampled in the Postponed region of the corresponding past or future clock tick.
- The sampled value of any other expression is defined recursively using the values of its arguments. For example, the sampled value of an expression `e1 & e2`, where `e1` and `e2` are expressions, is the bitwise AND of the sampled values of `e1` and `e2`. In particular, if an expression contains a function call, to evaluate the sampled value of this expression, the function is called on the sampled values of its arguments at the time of the expression evaluation. For example, if `a` is a static module variable, `s` is a sequence, and `f` is a function, the sampled value of `f(a, s.triggered)` is the result of the application of `f` to the sampled values of `a` and `s.triggered`, i.e., to the value of `a` taken from the Preponed region and to the current value of `s.triggered`.

16.5.2 Assertion clock

The timing model employed in a concurrent assertion specification is based on clock ticks and uses a generalized notion of clock cycles. The definition of a clock is explicitly specified by the user and can vary from one expression to another.

In an assertion, the sampled value is the only valid value of a variable during a clock tick. [Figure 16-1](#) shows the values of a variable as the clock progresses. The value of signal `req` is low at clock ticks 1 and 2. At clock tick 3, the value is sampled as high and remains high until clock tick 6. The sampled value of variable `req` at clock tick 6 is low and remains low up to and including clock tick 9. Notice that the simulation value transitions to high at clock tick 9. However, the sampled value at clock tick 9 is low.

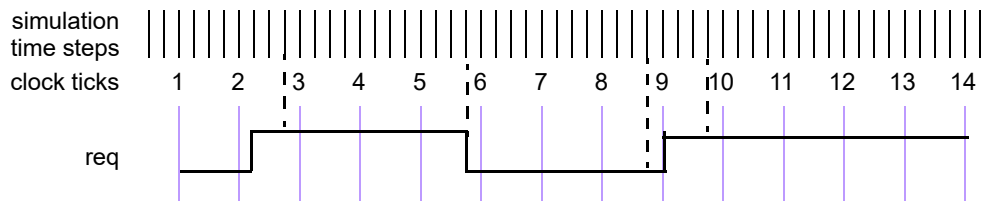


Figure 16-1—Sampling a variable in a simulation time step

An expression used in an assertion is always tied to a clock definition, except for the use of constant or automatic values from procedural code (see [16.14.6](#)). The sampled values are used to evaluate value change expressions or Boolean subexpressions that are required to determine a match of a sequence.

For concurrent assertions, the following statements apply:

- It is important that the defined clock behavior be glitch free. Otherwise, wrong values can be sampled.
- If a variable that appears in the expression for clock also appears in an expression with an assertion, the values of the two usages of the variable can be different. The current value of the variable is used in the clock expression, while the sampled value of the variable is used within the assertion.

The clock expression that controls evaluation of a sequence can be more complex than just a single signal name. Expressions such as `clk iff gating_signal` can be used to represent a gated clock. Other more complex expressions are possible. However, in order to verify proper behavior of the system and conform as closely as possible to truly cycle-based semantics, the user should ensure that the clock expression is glitch-free and only transitions once at any simulation time. (See [9.4.2.3](#) for the reasons that the expression `clk iff gating_signal` avoids the potential glitch in the expression `clk && gating_signal`.)

A reference to `$global_clock` (see [14.14](#)) is understood to be a reference to a *clocking_event* defined in a **global clocking** declaration. A global clock behaves just as any other clocking event. In formal verification, however, `$global_clock` has additional significance, as it is considered to be the primary system clock (see [F.3.1](#)). Thus, in the following example:

```
global clocking @clk; endclocking
...
assert property (@$global_clock a);
```

the assertion states that `a` is true at each tick of the global clock. This assertion is logically equivalent to:

```
assert property (@clk a);
```

An example of a concurrent assertion is as follows:

```
base_rule1: assert property (cont_prop(rst,in1,in2)) $display("%m, passing");  
           else $display("%m, failed");
```

The keyword **property** distinguishes a concurrent assertion from an immediate assertion. The syntax of concurrent assertions is discussed in [16.14](#).

16.6 Boolean expressions

The outcome of the evaluation of an expression is Boolean and is interpreted the same way as an expression is interpreted in the condition of a procedural **if** statement. In other words, if the expression evaluates to *x*, *z*, or 0, then it is interpreted as being false. Otherwise, it is true.

Expressions that appear in concurrent assertions shall satisfy the following requirements:

- An expression shall result in a type that is cast compatible with an integral type. Subexpressions need not meet this requirement as long as the overall expression is cast compatible with an integral type.
- Elements of dynamic arrays, queues, and associative arrays that are sampled for assertion expression evaluation may get removed from the array or the array may get resized before the assertion expression is evaluated. These specific array elements sampled for assertion expression evaluation shall continue to exist within the scope of the assertion until the assertion expression evaluation completes.
- Expressions that appear in procedural concurrent assertions may reference automatic variables as described in [16.14.6.1](#). Otherwise, expressions in concurrent assertions shall not reference automatic variables.
- Expressions shall not reference non-static class properties or methods.
- Expressions shall not reference variables of the **chandle** data type.
- Sequence match items with a local variable as the *variable_lvalue* may use the C assignment, increment, and decrement operators. Otherwise, evaluation of an expression shall not have any side effects (e.g., the increment and decrement operators are not allowed).
- Functions that appear in expressions shall not contain output or **ref** arguments (**const ref** is allowed).
- Functions shall be automatic (or preserve no state information) and have no side effects.

Care should be taken when accessing large data structures, especially large dynamic data structures, in concurrent assertions. Some types of access may require creating a copy of the entire data structure, which could incur a significant performance penalty. The following example illustrates how the need to copy an entire data structure may arise. In *p1* only a single byte of *q* must be sampled by the assertion, and the location of that byte is constant. However, in *p2* there will be multiple active threads with potentially different values of *l_b*. This increases the difficulty of determining which bytes of *q* to sample and likely results in sampling all of *q*.

```
bit a;  
integer b;  
byte q[$];  
  
property p1;  
  $rose(a) |-> q[0];  
endproperty  
  
property p2;  
  integer l_b;  
  ($rose(a), l_b = b) |-> ##[3:10] q[l_b];  
endproperty
```

There are two places where Boolean expressions occur in concurrent assertions. They are as follows:

- In a sequence or property expression
- In the disable condition inferred for an assertion, specified either in a top-level **disable iff** clause (see [16.12](#)) or in a **default disable iff** declaration (see [16.15](#))

The Boolean expressions used in defining a sequence or property expression shall be evaluated over the sampled values of all variables. The preceding rule shall not, however, apply to expressions in a clocking event (see [16.5](#)).

The expressions in a disable condition are evaluated using the current values of variables (not sampled) and may contain the sequence Boolean method `triggered`. They shall not contain any reference to local variables or to the sequence method `matched`.

Assertions that perform checks based on time values should capture these values in the same context. It is not recommended to capture time outside of the assertion. Time should be captured within the assertion using local variables. The following example illustrates how a problem may arise when capturing time in different contexts. In property `p1`, a time value, `t`, is captured in a procedural context based on the current value of `count`. Within the assertion, a comparison is made between the time value `t` and the time value returned by `$realtime` in the assertion context based on the sampled value of `count`. In both contexts, `$realtime` returns the current time value. As a result, the comparison between values of time captured in the different contexts yields an inconsistent result. The inconsistency results in the computation for `p1` checking the amount of time that elapses between 8 periods of `clk` instead of the intended 7. In property `p2`, both time values are captured within the assertion context. This strategy yields a consistent result.

```

bit [2:0] count;
realtime t;

initial count = 0;
always @(posedge clk) begin
    if (count == 0) t = $realtime; //capture t in a procedural context
    count++;
end

property p1;
    @(posedge clk)
    count == 7 |-> $realtime - t < 50.5;
endproperty

property p2;
    realtime l_t;
    @(posedge clk)
    (count == 0, l_t = $realtime) ##1 (count == 7) [->1] |->
        $realtime - l_t < 50.5;
endproperty

```

16.7 Sequences

```

sequence_expr ::= //from A.2.10
    cycle_delay_range sequence_expr { cycle_delay_range sequence_expr }
  | sequence_expr cycle_delay_range sequence_expr { cycle_delay_range sequence_expr }
  | expression_or_dist [ boolean_abbrev ]
  | sequence_instance [ sequence_abbrev ]
  | ( sequence_expr { , sequence_match_item } ) [ sequence_abbrev ]

```

```

| sequence_expr and sequence_expr
| sequence_expr intersect sequence_expr
| sequence_expr or sequence_expr
| first_match ( sequence_expr { , sequence_match_item } )
| expression_or_dist throughout sequence_expr
| sequence_expr within sequence_expr
| clocking_event sequence_expr
cycle_delay_range ::=
    ## constant_primary
    | ## [ cycle_delay_const_range_expression ]
    | ## [*]
    | ## [+]
sequence_match_item ::=
    operator_assignment
    | inc_or_dec_expression
    | subroutine_call
sequence_instance ::=
    ps_or_hierarchical_sequence_identifier [ ( [ sequence_list_of_arguments ] ) ]
sequence_list_of_arguments ::=
    [sequence_actual_arg] { , [sequence_actual_arg] } { , . identifier ( [sequence_actual_arg] ) }
    | . identifier ( [sequence_actual_arg] ) { , . identifier ( [sequence_actual_arg] ) }
sequence_actual_arg ::=
    event_expression
    | sequence_expr
boolean_abbrev ::=
    consecutive_repetition
    | non_consecutive_repetition
    | goto_repetition
sequence_abbrev ::= consecutive_repetition
consecutive_repetition ::=
    [* const_or_range_expression ]
    | [*]
    | [+]
non_consecutive_repetition ::= [= const_or_range_expression ]
goto_repetition ::= [-> const_or_range_expression ]
const_or_range_expression ::=
    constant_expression
    | cycle_delay_const_range_expression
cycle_delay_const_range_expression ::=
    constant_expression : constant_expression
    | constant_expression : $
expression_or_dist ::= expression [ dist { dist_list } ]

```

Syntax 16-3—Sequence syntax (excerpt from [Annex A](#))

Properties are often constructed out of sequential behaviors. The **sequence** feature provides the capability to build and manipulate sequential behaviors. The simplest sequential behaviors are linear. A linear sequence is a finite list of SystemVerilog Boolean expressions in a linear order of increasing time. The linear sequence is said to match along a finite interval of consecutive clock ticks provided the first Boolean expression evaluates to true at the first clock tick, the second Boolean expression evaluates to true at the second clock tick, and so forth, up to and including the last Boolean expression evaluating to true at the last

clock tick. A single Boolean expression is an example of a simple linear sequence, and it matches at a single clock tick provided the Boolean expression evaluates to true at that clock tick.

More complex sequential behaviors are described by SystemVerilog sequences. A sequence is a regular expression over the SystemVerilog Boolean expressions that concisely specifies a set of zero, finitely many, or infinitely many linear sequences. If at least one of the linear sequences from this set matches along a finite interval of consecutive clock ticks, then the sequence is said to match along that interval.

A property may involve checking of one or more sequential behaviors beginning at various times. An attempted evaluation of a sequence is a search for a match of the sequence beginning at a particular clock tick. To determine whether such a match exists, appropriate Boolean expressions are evaluated beginning at the particular clock tick and continuing at each successive clock tick until either a match is found or it is deduced that no match can exist.

A sequence may admit an *empty match*, a match that occurs over an interval of length 0. (See a formal definition at [16.12.22](#), and see [16.9.2.1](#) for more discussion of empty matches). An *end point* of a sequence is the time step of any nonempty match of the sequence. An end point is reached whenever the ending clock tick of a match of the sequence is reached, regardless of the starting clock tick of the match. A *match point* includes both empty and nonempty matches, and is reached either at an end point or, in the case of an empty match, at the length-0 time interval at the beginning of the time step when sequence evaluation begins. A sequence that admits only empty matches is referred to as an *empty sequence*.

Sequences can be composed by concatenation, analogous to a concatenation of lists. The concatenation specifies a delay, using ##, from the end of the first sequence until the beginning of the second sequence.

The syntax for sequence concatenation is shown in [Syntax 16-4](#).

```
sequence_expr ::=                                     // from A.2.10
    cycle_delay_range sequence_expr { cycle_delay_range sequence_expr }
    | sequence_expr cycle_delay_range sequence_expr { cycle_delay_range sequence_expr }
    ...
cycle_delay_range ::=
    ## constant_primary
    | ## [ cycle_delay_const_range_expression ]
    | ## [*]
    | ## [+]
cycle_delay_const_range_expression ::=
    constant_expression : constant_expression
    | constant_expression : $
```

Syntax 16-4—Sequence concatenation syntax (excerpt from [Annex A](#))

In this syntax, the following statements apply:

- *constant_primary* is a *constant_expression*, which is computed at compile time and shall result in an integer value. Furthermore, *constant_expression* and the bounds in *cycle_delay_const_range_expression* can only be 0 or greater.
- The \$ token is used to indicate a finite, but unbounded, maximum.
- ## [*] is used as an equivalent representation of ## [0:\$].
- ## [+] is used as an equivalent representation of ## [1:\$].
- When a range is specified with two expressions, the second expression shall be greater than or equal to the first expression.

- In a *cycle_delay_range*, it shall be illegal for a *constant_primary* to contain a *constant_mintypmax_expression* that is not also a *constant_expression*.

The context in which a sequence occurs determines when the sequence is evaluated. The first expression in a sequence is checked at the first occurrence of the clock tick at or after the expression that triggered evaluation of the sequence. Each successive element (if any) in the sequence is checked at the next subsequent occurrence of the clock.

A *##* followed by a number or range specifies the delay from the current clock tick to the beginning of the sequence that follows. The delay *##1* indicates that the beginning of the sequence that follows is one clock tick later than the current clock tick. The delay *##0* indicates that the beginning of the sequence that follows is at the same clock tick as the current clock tick.

When used as a concatenation between two sequences, the delay is from the end of the first sequence to the beginning of the second sequence. The delay *##1* indicates that the beginning of the second sequence is one clock tick later than the end of the first sequence. The delay *##0* indicates that the beginning of the second sequence is at the same clock tick as the end of the first sequence.

In the examples in this clause, *`true* is a Boolean expression that always evaluates to 1'b1 and is used for visual clarity. It is defined as follows:

```
`define true 1'b1

##0 a      // means a
##1 a      // means `true ##1 a
##2 a      // means `true ##1 `true ##1 a
##[0:3]a   // means (a) or (`true ##1 a) or (`true ##1 `true ##1 a) or
           // (`true ##1 `true ##1 `true ##1 a)
a ##2 b    // means a ##1 `true ##1 b
```

The sequence

```
req ##1 gnt ##1 !req
```

specifies that *req* be true on the current clock tick, *gnt* shall be true on the first subsequent tick, and *req* shall be false on the next clock tick after that. The *##1* operator specifies one clock tick separation. A delay of more than one clock tick can be specified, as in the following:

```
req ##2 gnt
```

This specifies that *req* shall be true on the current clock tick, and *gnt* shall be true on the second subsequent clock tick, as shown in [Figure 16-2](#).

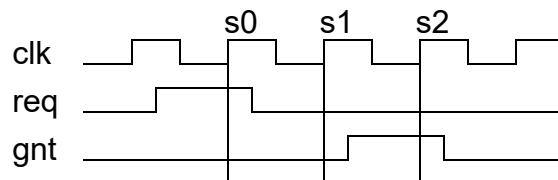


Figure 16-2—Concatenation of sequences

The following specifies that signal *b* shall be true on the *N*th clock tick after signal *a*:

```
a ##N b      // check b on the Nth sample
```

To specify a concatenation of overlapped sequences, where the end point of one sequence coincides with the start of the next sequence, a value of 0 is used, as follows:

```
a ##1 b ##1 c // first sequence seq1
d ##1 e ##1 f // second sequence seq2
(a ##1 b ##1 c) ##0 (d ##1 e ##1 f) // overlapped concatenation
```

In the preceding example, *c* must be true at the end point of sequence *seq1*, and *d* must be true at the start of sequence *seq2*. When concatenated with 0 clock tick delay, *c* and *d* must be true at the same time, resulting in a concatenated sequence equivalent to the following:

```
a ##1 b ##1 c&&d ##1 e ##1 f
```

It should be noted that no other form of overlapping between the sequences can be expressed using the concatenation operation.

In cases where the delay can be any value in a range, a time window can be specified as follows:

```
req ##[4:32] gnt
```

In the preceding case, signal *req* must be true at the current clock tick, and signal *gnt* must be true at some clock tick between the 4th and the 32nd clock tick after the current clock tick.

The time window can extend to a finite, but unbounded, range by using \$ as in the following example:

```
req ##[4:$] gnt
```

A sequence can be unconditionally extended by concatenation with ``true`.

```
a ##1 b ##1 c ##3 `true
```

After satisfying signal *c*, the sequence length is extended by three clock ticks. Such adjustments in the length of sequences can be required when complex sequences are constructed by combining simpler sequences.

16.8 Declaring sequences

A named **sequence** may be declared in the following:

- A module
- An interface
- A program
- A **clocking** block
- A package
- A compilation-unit scope
- A checker
- A generate block

Named sequences are declared using [Syntax 16-5](#).

```

assertion_item_declaration ::=                                     //from A.2.10
    ...
    | sequence_declaration
sequence_declaration ::=
    sequence sequence_identifier [ ( [ sequence_port_list ] ) ] ;
    { assertion_variable_declaration }
    sequence_expr [ ; ]
    endsequence [ : sequence_identifier ]
sequence_port_list ::=
    sequence_port_item { , sequence_port_item }
sequence_port_item ::=
    { attribute_instance } [ local [ sequence_lvar_port_direction ] ] sequence_formal_type
    formal_port_identifier {variable_dimension} [ = sequence_actual_arg ]
sequence_lvar_port_direction ::= input | inout | output
sequence_formal_type ::=
    data_type_or_implicit
    | sequence
    | untyped
formal_port_identifier ::= identifier                             //from A.9.3
sequence_instance ::=                                           //from A.2.10
    ps_or_hierarchical_sequence_identifier [ ( [ sequence_list_of_arguments ] ) ]
sequence_list_of_arguments ::=
    [sequence_actual_arg] { , [sequence_actual_arg] } { , . identifier ([sequence_actual_arg] ) }
    | . identifier ([sequence_actual_arg] ) { , . identifier ([sequence_actual_arg] ) }
sequence_actual_arg ::=
    event_expression
    | sequence_expr
assertion_variable_declaration ::=
    var_data_type list_of_variable_decl_assignments ;

```

Syntax 16-5—Sequence declaration syntax (excerpt from [Annex A](#))

A named sequence may be declared with formal arguments in the optional *sequence_port_list*.

A formal argument may be typed by specifying the type prior to the *formal_port_identifier* of the formal argument. A type shall apply to all formal arguments whose identifiers both follow the type and precede the next type, if any, specified in the port list. Rules particular to the specification and use of typed formal arguments are discussed in [16.8.1](#).

Rules particular to the specification and use of local variable formal arguments are discussed in [16.8.2](#).

A formal argument is said to be *untyped* if there is no type specified prior to its *formal_port_identifier* in the port list. There is no default type for a formal argument.

The supported data types for sequence formal arguments are the types that are allowed for operands in assertion expressions (see [16.6](#)) and the keyword **untyped**.

A default actual argument may be specified for a formal argument in the optional associated declaration assignment. The *default_expression* is resolved in the scope containing the sequence declaration. Requirements for the type of the default actual argument of a typed formal argument are described in [16.8.1](#).

The default actual argument of an untyped formal argument may be of any type provided its substitution results in a valid sequence as described in the rewriting algorithm (see [F.4.1](#)).

A formal argument may be referenced in the body of the declaration of the named sequence. A reference to a formal argument may be written in place of various syntactic entities, such as the following:

- identifier
- expression
- `sequence_expr`
- `event_expression`
- terminal \$ in a *cycle_delay_const_range_expression*

A named sequence may be instantiated by referencing its name. The reference may be a hierarchical name (see [23.6](#)). A named sequence may be instantiated anywhere that a *sequence_expr* may be written, including prior to its declaration. A named sequence may also be instantiated as part of a *sequence_method_call* (see [16.9.11](#), [16.13.5](#)) or as an *event_expression* (see [9.4.2.4](#)). It shall be an error if a cyclic dependency among named sequences results from their instantiations. A cyclic dependency among named sequences results if, and only if, there is a cycle in the directed graph whose nodes are the named sequences and whose edges are defined by the following rule: there is a directed edge from one named sequence to a second named sequence if, and only if, either the first named sequence instantiates the second named sequence within its declaration, including an instance within the declaration of a default actual argument, or there is an instance of the first named sequence that instantiates the second named sequence within an actual argument.

In an instance of a named sequence, actual arguments may be passed to formal arguments. The instance shall provide an actual argument in the list of arguments for each formal argument that does not have a default actual argument declared. The instance may provide an actual argument for a formal argument that has a default actual argument, thereby overriding the default. Actual arguments in the list of arguments may be bound to formal arguments by name or by position.

The terminal \$ may be an actual argument in an instance of a named sequence, either declared as a default actual argument or passed in the list of arguments of the instance. If \$ is an actual argument, then the corresponding formal argument shall be untyped and each of its references either shall be an upper bound in a *cycle_delay_const_range_expression* or shall itself be an actual argument in an instance of a named sequence.

If an instance of a named sequence is within the scope of a local variable (see [16.10](#)), then an actual argument in the list of arguments of the instance may reference the local variable.

Names other than formal arguments that appear in the declaration of a named sequence, including those that appear in default actual arguments, shall be resolved according to the scoping rules from the scope of the declaration of the named sequence. Names appearing in actual arguments in the list of arguments of the instance shall be resolved according to the scoping rules from the scope of the instance of the named sequence.

The sequential behavior and matching semantics of an instance of a named sequence are the same as those of the flattened sequence that is obtained from the body of the declaration of the named sequence by the rewriting algorithm defined in [F.4.1](#). The rewriting algorithm substitutes actual arguments for references to the corresponding formal arguments in the body of the declaration of the named sequence. The rewriting algorithm does not itself account for name resolution and assumes that names have been resolved prior to the substitution of actual arguments. If the flattened sequence is not legal, then the instance is not legal and there shall be an error.

The substitution of an actual argument for a reference to the corresponding untyped formal argument in the rewriting algorithm retains the actual as an expression term. An actual argument shall be enclosed in

parentheses and shall be cast to its self-determined type before being substituted for a reference to the corresponding formal argument unless one of the following conditions holds:

- The actual argument is \$.
- The actual argument is a *variable_lvalue*.

If the result of the rewriting algorithm is an invalid sequence, an error shall occur.

For example, a reference to an untyped formal argument may appear in the specification of a *cycle_delay_range*, a *boolean_abbrev*, or a *sequence_abbrev* (see [16.9.2](#)) only if the actual argument is an elaboration-time constant. The following example illustrates such usage of formal arguments:

```
sequence delay_example(x, y, min, max, delay1);
    x ##delay1 y[*min:max];
endsequence

// Legal
a1: assert property (@(posedge clk) delay_example(x, y, 3, $, 2));

int z, d;

// Illegal: z and d are not elaboration-time constants
a2_illegal: assert property (@(posedge clk) delay_example(x, y, z, $, d));
```

In the following example, named sequences *s1* and *s2* are evaluated on successive **posedge** events of *clk*. The named sequence *s3* is evaluated on successive **negedge** events of *clk*. The named sequence *s4* is evaluated on successive alternating **posedge** and **negedge** events of *clk*.

```
sequence s1;
    @(posedge clk) a ##1 b ##1 c;
endsequence
sequence s2;
    @(posedge clk) d ##1 e ##1 f;
endsequence
sequence s3;
    @(negedge clk) g ##1 h ##1 i;
endsequence
sequence s4;
    @(edge clk) j ##1 k ##1 l;
endsequence
```

Another example of named sequence declaration, which includes arguments, follows:

```
sequence s20_1(data,en);
    (!frame && (data==data_bus)) ##1 (c_be[0:3] == en);
endsequence
```

Named sequence *s20_1* does not specify a clock. In this case, a clock would be inherited from some external source, such as a **property** or an **assert** statement. An example of instantiating a named sequence is shown as follows:

```
sequence s;
    a ##1 b ##1 c;
endsequence
sequence rule;
    @(posedge sysclk)
    trans ##1 start_trans ##1 s ##1 end_trans;
endsequence
```

Named sequence rule in the preceding example is equivalent to the following:

```
sequence rule;
  @(posedge sysclk)
    trans ##1 start_trans ##1 (a ##1 b ##1 c) ##1 end_trans ;
endsequence
```

The following example illustrates an illegal cyclic dependency among the named sequences *s1* and *s2*:

```
sequence s1;
  @(posedge sysclk) (x ##1 s2);
endsequence
sequence s2;
  @(posedge sysclk) (y ##1 s1);
endsequence
```

16.8.1 Typed formal arguments in sequence declarations

The data type specified for a formal argument of a sequence may be the keyword **untyped**. A formal argument shall be untyped (see 16.8) if its data type is **untyped**. The semantics of binding an actual argument expression to a formal with a data type of **untyped** shall be the same as the semantics for an untyped formal. The keyword **untyped** shall be used if an untyped formal argument follows a data type in the formal argument list.

If a formal argument of a named sequence is typed, then the type shall be **sequence** or one of the types allowed in 16.6. The following rules apply to typed formal arguments and their corresponding actual arguments, including default actual arguments declared in a named sequence:

- If the formal argument is of type **sequence**, then the actual argument shall be a *sequence_expr*. A reference to the formal argument of type **sequence** shall either be in a place where a *sequence_expr* is legal, or as an operand of sequence methods *triggered* and *matched*.
- If the formal argument is of type **event**, then the actual argument shall be an *event_expression* and each reference to the formal argument shall be in a place where an *event_expression* may be written.
- Otherwise, the self-determined result type of the actual argument shall be cast compatible (see 6.22.4) with the type of the formal argument. If the actual argument is a *variable_lvalue*, references to the formal shall be considered as having the formal's type with any assignment to the formal being treated as though there was a subsequent assignment from the formal to the actual argument. If the actual argument is not a *variable_lvalue*, the actual argument shall be cast to the type of the formal argument before being substituted for a reference to the formal argument in the rewriting algorithm (see F.4.1).

For example, a Boolean expression may be passed as an actual argument to a formal argument of type **sequence** because a Boolean expression is a *sequence_expr*. A formal argument of type **sequence** may not be referenced as the *expression_or_dist* operand of a *goto_repetition* (see 16.9.2), regardless of the corresponding actual argument, because a *sequence_expr* may not be written in that position.

A reference to a typed formal argument within a *sequence_match_item* (see 16.10) shall not stand as the *variable_lvalue* in either an *operator_assignment* or an *inc_or_dec_expression* unless the formal argument is a local variable argument (see 16.8.2, 16.12.19).

Two examples of declaring formal arguments follow. All of the formal arguments of *s1* are untyped. The formal arguments *w* and *y* of *s2* are untyped, while the formal argument *x* has type **bit**.

```
sequence s1(w, x, y);
  w ##1 x ##[2:10] y;
endsequence
```

```
sequence s2(w, y, bit x);
    w ##1 x ##[2:10] y;
endsequence
```

The following instances of `s1` and `s2` are equivalent:

```
s1(.w(a), .x(bit'(b)), .y(c))
s2(.w(a), .x(b), .y(c))
```

In the instance of `s2` above, if `b` happens to be 8 bits wide then it will be cast to **bit** by truncation since it is being passed to a formal argument of type **bit**. Similarly, if an expression of type **bit** is passed as actual argument to a formal argument of type **byte**, then the expression is extended to a **byte**.

If a reference to a typed formal argument appears in the specification of a *cycle_delay_range*, a *boolean_abbrev*, or a *sequence_abbrev* (see [16.9.2](#)), then the type of the formal argument shall be **shortint**, **int**, or **longint**. The following example illustrates such usage of formal arguments:

```
sequence delay_arg_example (max, shortint delay1, delay2, min);
    x ##delay1 y[*min:max] ##delay2 z;
endsequence

parameter my_delay=2;
cover property (delay_arg_example($, my_delay, my_delay-1, 3));
```

The cover property in the preceding example is equivalent to the following:

```
cover property (x ##2 y[*3:$] ##1 z);
```

The following shows an example of a formal argument with **event** type:

```
sequence event_arg_example (event ev);
    @(ev) x ##1 y;
endsequence

cover property (event_arg_example(posedge clk));
```

The cover property in the preceding example is equivalent to the following:

```
cover property (@(posedge clk) x ##1 y);
```

If the intent is to pass as actual argument an expression that will be combined with an *edge_identifier* to create an *event_expression*, then the formal argument shall not be typed with type **event**. The following example illustrates such usage:

```
sequence event_arg_example2 (reg sig);
    @(posedge sig) x ##1 y;
endsequence

cover property (event_arg_example2(clk));
```

The cover property in the preceding example is equivalent to the following:

```
cover property (@(posedge clk) x ##1 y);
```

Another example, in which a local variable is used to sample a formal argument, shows how to get the effect of “pass by value.” Pass by value is not currently supported as a mode of argument passing.

```
sequence s(bit a, bit b);
  bit loc_a;
  (1'b1, loc_a = a) ##0
  (t == loc_a) [*0:$] ##1 b;
endsequence
```

16.8.2 Local variable formal arguments in sequence declarations

This subclause describes mechanisms for declaring local variable formal arguments and rules specific to their use. Local variable formal arguments are special cases of local variables (see [16.10](#)).

A formal argument of a named sequence may be designated as a local variable argument by specifying the keyword **local** in the port item, followed optionally by one of the directions **input**, **inout**, or **output**. If no direction is specified explicitly, then the direction **input** shall be inferred. If the keyword **local** is specified in a port item, then the type of that argument shall be specified explicitly in that port item and shall not be inferred from a previous argument. The type of a local variable argument shall be one of the types allowed in [16.6](#). If one of the directions **input**, **inout**, or **output** is specified in a port item, then the keyword **local** shall be specified in that port item.

The designation of a formal argument as a local variable argument of a given direction and type shall apply to subsequent identifiers in the port list as long as none of the subsequent port items specifies the keyword **local** or an explicit type. In other words, if a port item consists only of an identifier and if the nearest preceding argument with an explicitly specified type also specifies the keyword **local**, then the port item is a local variable argument with the same direction and type as that preceding argument.

If a local variable formal argument has direction **input**, then a default actual argument may be specified for that argument in the optional declaration assignment in the port item, subject to the rules for default actual arguments described in [16.8](#). It shall be illegal to specify a default actual argument for a local variable argument of direction **inout** or **output**.

An example showing legal declaration of a named sequence using local variable formal arguments is as follows:

```
logic b_d, d_d;
sequence legal_loc_var_formal (
  local inout logic a,
  local logic b = b_d, // input inferred, default actual argument b_d
                      c, // local input logic inferred, no default
                      // actual argument
  d = d_d, // local input logic inferred, default actual
          // argument d_d
  logic e, f // e and f are not local variable formal arguments
);
  logic g = c, h = g || d;
  ...
endsequence
```

An example showing illegal declaration of a named sequence using local variable formal arguments is shown as follows:

```
sequence illegal_loc_var_formal (
  output logic a, // illegal: local must be specified with
                 // direction
```

```

local inout logic b,
                                c = 1'b0, // default actual argument illegal for inout
local                d = expr, // illegal: type must be specified explicitly
local event         e,        // illegal: event is a type disallowed in
                                // 16.6
local logic         f = g      // g shall not refer to the local variable
                                // below and must be resolved upward from
                                // this declaration
);
logic g = b;
...
endsequence

```

In general, a local variable formal argument behaves in the same way as a local variable declared in an *assertion_variable_declaration*. The rules in 16.10 for assigning to and referencing local variables, including the rules of local variable flow, apply to local variable formal arguments with the following provisions:

- Without further specification, the term *local variable* shall mean either a local variable formal argument or a local variable declared in an *assertion_variable_declaration*.
- At the beginning of each evaluation attempt of an instance of a named sequence, a new copy of each of its local variable formal arguments shall be created.
- A local variable formal argument with direction **input** or **inout** shall be treated like a local variable declared in an *assertion_variable_declaration* with a declaration assignment. The initial value for the local variable formal argument is provided by the associated actual argument for the instance. The self-determined result type of the actual argument shall be cast compatible (see 6.22.4) with the type of the local variable formal argument. The value of the actual argument shall be cast to the type of the local variable formal argument before being assigned as initial value to the local variable formal argument. This assignment is referred to as the *initialization assignment* of the local variable formal argument. Initialization of all input and inout local variable formal arguments shall be performed before initialization of any local variable declared in an *assertion_variable_declaration*. The expression of a declaration assignment to a local variable declared in an *assertion_variable_declaration* may refer to a local variable formal argument of direction **input** or **inout**.
- If a local variable formal argument of direction **input** or **inout** is bound to an actual argument in the argument list of an instance and if the actual argument references a local variable, then it shall be an error if that local variable is unassigned at the point of the reference in the context of the instance.
- A local variable formal argument of direction **output** shall be unassigned at the beginning of the evaluation attempt of the instance.
- The entire actual argument expression bound to an **inout** or **output** local variable formal argument shall itself be a reference to a local variable whose scope includes the instance and with whose type the type of the local variable formal argument is cast compatible. It shall be an error if references to the same local variable are bound as actual arguments to two or more local variable formal arguments of direction **inout** or **output**. It shall be an error if there exists a match of the named sequence for which an inout or output local variable formal argument is unassigned at the completion of the match. At the completion of a match of the instance of the named sequence, the value of the inout or output local variable formal argument shall be cast to the type of and assigned to the local variable whose reference is the associated actual argument. If multiple threads of evaluation of the instance of the named sequence match, then multiple threads of evaluation shall continue in the instantiation context, each with its own copy of the actual argument local variable. For each matching thread of the instance of the named sequence, at the completion of the match of that thread the value of the local variable formal argument in that thread shall be cast to the type of and assigned to the associated copy of the actual argument local variable.

- It shall be an error for an instance of a named sequence with an **inout** or **output** local variable formal argument to admit an empty match (see [16.12.22](#)).
- It shall be an error to apply any of the sequence methods **triggered** (see [16.9.11](#), [16.13.6](#)) or **matched** (see [16.13.5](#)) to an instance of a named sequence with an input or inout local variable formal argument.

The following example illustrates legal usage of a local variable formal argument:

```
sequence sub_seq2(local inout int lv);  
  (a ##1 !a, lv += data_in)  
  ##1 !b[*0:$] ##1 b && (data_out == lv);  
endsequence  
sequence seq2;  
  int v1;  
  (c, v1 = data)  
  ##1 sub_seq2(v1) // lv is initialized by assigning it the value of v1;  
                  // when the instance sub_seq2(v1) matches, v1 is  
                  // assigned the value of lv  
  ##1 (do1 == v1);  
endsequence
```

The matching behavior of `seq2` is equivalent to that of `seq2_inlined` as follows:

```
sequence seq2_inlined;  
  int v1, lv;  
  (c, v1 = data) ##1  
  (  
    (1, lv = v1) ##0  
    (a ##1 !a, lv += data_in)  
    ##1 (!b[*0:$] ##1 b && (data_out == lv), v1 = lv)  
  )  
  ##1 (do1 == v1);  
endsequence
```

Untyped arguments provide an alternative mechanism for passing local variables to an instance of a subsequence, including the capability to assign to the local variable in the subsequence and later reference the value assigned in the instantiation context (see [16.10](#)).

16.9 Sequence operations

16.9.1 Operator precedence

Operator precedence and associativity are listed in [Table 16-1](#). The highest precedence is listed first.

Table 16-1—Operator precedence and associativity

SystemVerilog expression operators	Associativity
[*] [=] [->]	—
##	Left
throughout	Right
within	Left
intersect	Left

Table 16-1—Operator precedence and associativity (*continued*)

SystemVerilog expression operators	Associativity
and	Left
or	Left

16.9.2 Repetition in sequences

The syntax for sequence repetition is shown in [Syntax 16-6](#).

```

sequence_expr ::=                                     //from A.2.10
...
| expression_or_dist [ boolean_abbrev ]
| sequence_instance [ sequence_abbrev ]
| ( sequence_expr { , sequence_match_item } ) [ sequence_abbrev ]
...
boolean_abbrev ::=
    consecutive_repetition
    | non_consecutive_repetition
    | goto_repetition
sequence_abbrev ::= consecutive_repetition
consecutive_repetition ::=
    [* const_or_range_expression ]
    | [*]
    | [+]
non_consecutive_repetition ::= [= const_or_range_expression ]
goto_repetition ::= [-> const_or_range_expression ]
const_or_range_expression ::=
    constant_expression
    | cycle_delay_const_range_expression
cycle_delay_const_range_expression ::=
    constant_expression : constant_expression
    | constant_expression : $

```

Syntax 16-6—Sequence repetition syntax (excerpt from [Annex A](#))

The number of iterations of a repetition can either be specified by exact count or be required to fall within a finite range. If specified by exact count, then the number of iterations is defined by a non-negative integer constant expression. If required to fall within a finite range, then the minimum number of iterations is defined by a non-negative integer constant expression; and the maximum number of iterations either is defined by a non-negative integer constant expression or is \$, indicating a finite, but unbounded, maximum.

If both the minimum and maximum numbers of iterations are defined by non-negative integer constant expressions (see [11.2.1](#)), then the minimum number shall be less than or equal to the maximum number.

See [16.9.2.1](#) for discussion of the special case where the number of iterations is 0.

The following three kinds of repetition are provided:

- *Consecutive repetition* ([*const_or_range_expression]): Consecutive repetition specifies finitely many iterative matches of the operand sequence, with a delay of one clock tick from the end of one

match to the beginning of the next. The overall repetition sequence matches at the end of the last iterative match of the operand. `[*]` is an equivalent representation of `[*0:$]` and `[+]` is an equivalent representation of `[*1:$]`.

- *Goto repetition* (`[->const_or_range_expression]`): Goto repetition specifies finitely many iterative matches of the operand Boolean expression, with a delay of one or more clock ticks from one match of the operand to the next successive match and no match of the operand strictly in between. The overall repetition sequence matches at the last iterative match of the operand.
- *Nonconsecutive repetition* (`[=const_or_range_expression]`): Nonconsecutive repetition specifies finitely many iterative matches of the operand Boolean expression, with a delay of one or more clock ticks from one match of the operand to the next successive match and no match of the operand strictly in between. The overall repetition sequence matches at or after the last iterative match of the operand, but before any later match of the operand.

The effect of consecutive repetition of a subsequence within a sequence can be achieved by explicitly iterating the subsequence, as follows:

```
a ##1 b ##1 b ##1 b ##1 c
```

Using the consecutive repetition operator `[*3]`, which indicates three iterations, this sequential behavior is specified more succinctly:

```
a ##1 b [*3] ##1 c
```

A consecutive repetition specifies that the operand sequence shall match a specified number of times. The consecutive repetition operator `[*N]` specifies that the operand sequence must match *N* times in succession. For example:

```
a [*3] means a ##1 a ##1 a
```

The syntax allows the combination of a delay and repetition in the same sequence. The following are both allowed:

```
a ##3 (b[*3]) // means a ##1 `true ##1 `true ##1 (b ##1 b ##1 b)
(a ##2 b)[*3] // means (a ##2 b) ##1 (a ##2 b) ##1 (a ##2 b),
               // which in turn means
               // (a ##1 `true ##1 b) ##1 (a ##1 `true ##1 b) ##1 (a ##1 `true ##1 b)
```

A repetition with a range of minimum `min` and maximum `max` number of iterations can be expressed with the consecutive repetition operator `[*min:max]`.

For example:

```
(a ##2 b) [*1:5]
```

is equivalent to

```
(a ##2 b)
or (a ##2 b ##1 a ##2 b)
or (a ##2 b ##1 a ##2 b ##1 a ##2 b)
or (a ##2 b ##1 a ##2 b ##1 a ##2 b ##1 a ##2 b)
or (a ##2 b ##1 a ##2 b ##1 a ##2 b ##1 a ##2 b ##1 a ##2 b)
```

Similarly,

```
(a[*0:3] ##1 b ##1 c)
```

is equivalent to

```
(b ##1 c)
or (a ##1 b ##1 c)
or (a ##1 a ##1 b ##1 c)
or (a ##1 a ##1 a ##1 b ##1 c)
```

To specify a finite, but unbounded, number of iterations, the dollar sign (\$) is used. For example, the repetition

```
a ##1 b [*1:$] ##1 c
```

matches over an interval of three or more consecutive clock ticks if *a* is true on the first clock tick, *c* is true on the last clock tick, and *b* is true at every clock tick strictly in between the first and the last.

Specifying the number of iterations of a repetition by exact count is equivalent to specifying a range in which the minimum number of repetitions is equal to the maximum number of repetitions. In other words, `seq[*n]` is equivalent to `seq[*n:n]`.

The goto repetition (nonconsecutive exact repetition) takes a Boolean expression rather than a sequence as operand. It specifies the iterative matching of the Boolean expression at clock ticks that are not necessarily consecutive and ends at the last iterative match. For example:

```
a ##1 b [->2:10] ##1 c
```

matches over an interval of consecutive clock ticks provided *a* is true on the first clock tick, *c* is true on the last clock tick, *b* is true on the penultimate clock tick, and, including the penultimate, there are at least 2 and at most 10 not necessarily consecutive clock ticks strictly in between the first and last on which *b* is true. This sequence is equivalent to the following:

```
a ##1 ((!b[*0:$] ##1 b) [*2:10]) ##1 c
```

The nonconsecutive repetition is like the goto repetition except that a match does not have to end at the last iterative match of the operand Boolean expression. The use of nonconsecutive repetition instead of goto repetition allows the match to be extended by arbitrarily many clock ticks provided the Boolean expression is false on all of the extra clock ticks. For example:

```
a ##1 b [=2:10] ##1 c
```

matches over an interval of consecutive clock ticks provided *a* is true on the first clock tick, *c* is true on the last clock tick, and there are at least 2 and at most 10 not necessarily consecutive clock ticks strictly in between the first and last on which *b* is true. This sequence is equivalent to the following:

```
a ##1 ((!b [*0:$] ##1 b) [*2:10]) ##1 !b[*0:$] ##1 c
```

The consecutive repetition operator can be applied to general sequence expressions, but the goto repetition and nonconsecutive repetition operators can be applied only to Boolean expressions. In particular, goto repetition and nonconsecutive repetition cannot be applied to a Boolean expression to which a sequence match item (see [16.10](#), [16.11](#)) has been attached. For example, the following is a legal sequence expression:

```
(b[->1], v = e) [*2]
```

but the following is illegal:

```
(b, v = e) [->2]
```

16.9.2.1 Repetition, concatenation, and empty matches

Using 0 as a sequence repetition number, an empty sequence (see [16.7](#)) results, as in this example:

```
a [*0]
```

Because empty matches occur over an interval of zero clock ticks and are thus of length 0, they follow the set of concatenation rules specified below. In the following rules, an empty sequence is denoted as *empty*, and another sequence (which may be empty or nonempty) is denoted as *seq*.

- (empty ##0 seq) does not result in a match.
- (seq ##0 empty) does not result in a match.
- (empty ##n seq), where n is greater than 0, is equivalent to (##(n-1) seq).
- (seq ##n empty), where n is greater than 0, is equivalent to (seq ##(n-1) `true).

For example, compare the following two sequences:

```
a[*0] ##0 b
`true ##0 b
```

As defined by the preceding rules, the first sequence can never be matched: there is no point in time when the end point of the length-0 sequence `a[*0]` and the length-1 sequence `##0 b` are aligned. In contrast, the second is a well-defined sequence representing the fusion of two sequences of length 1. It will match during any time step when the sampled value of `b` is true.

To apply these rules to a sequence admitting both empty and nonempty matches, rewrite the sequence as the OR of its empty and nonempty cases. Consider the multiple concatenation example:

```
b ##1 a[*0:1] ##2 c
```

This is equivalent to:

```
(b ##1 a[*0] ##2 c) or (b ##1 a[*1] ##2 c)
```

which can be rewritten as:

```
(b ##1 ##1 c) or (b ##1 a ##2 c)
```

or, more concisely:

```
(b ##2 c) or (b ##1 a ##2 c)
```

From this example, we can see that when matching the 0-tick interval specified by the empty case `a[*0]`, the total execution time of the sequence is one less than when using the 1-tick interval specified by `a[*1]`.

16.9.3 Sampled value functions

This subclause describes the system functions available for accessing sampled values of an expression. These functions include the capability to access current sampled value, access sampled value in the past, or detect changes in sampled value of an expression. Sampling of an expression is explained in [16.5.1](#). Local variables (see [16.10](#)) and the sequence method `matched` are not allowed in the argument expressions passed to these functions. The following functions are provided:

```

$sampled ( expression )
$rose ( expression [ , [clocking_event] ] )
$fell ( expression [ , [clocking_event] ] )
$stable ( expression [ , [clocking_event] ] )
$changed ( expression [ , [ clocking_event ] ] )
$past ( expression1 [ , [number_of_ticks] [ , [expression2] [ , [clocking_event]] ] ] )

```

The use of these functions is not limited to assertion features; they may be used as expressions in procedural code as well. The clocking event, although optional as an explicit argument to the functions `$past`, `$rose`, `$stable`, `$changed`, and `$fell`, is required for their semantics. The clocking event is used to sample the value of the argument expression.

The function `$sampled` does not use a clocking event.

For a sampled value function other than `$sampled`, the clocking event shall be explicitly specified as an argument or inferred from the code where the function is called. The following rules are used to infer the clocking event:

- If called in an assertion, sequence, or property, the appropriate clocking event as determined by clock flow rules (see [16.13.3](#)) is used.
- Otherwise, if called in a disable condition or a clock expression in an assertion, sequence, or property, it shall be explicitly clocked.
- Otherwise, if called in an action block of an assertion, the leading clock of the assertion is used.
- Otherwise, if called in a procedure, the inferred clock, if any, from the procedural context (see [16.14.6](#)) is used.
- Otherwise, if called outside an assertion, default clocking (see [14.12](#)) is used.

The function `$sampled` returns the sampled value of its argument (see [16.5.1](#)). The use of `$sampled` in concurrent assertions, although allowed, is redundant, as the result of the function is identical to the sampled value of the expression itself used in the assertion. The use of `$sampled` in a **disable iff** clause is meaningful since the disable condition by default is not sampled (see [16.12](#)).

The function `$sampled` is useful to access the value of expressions used in concurrent assertions in their action blocks. Consider the following example:

```

logic a, b, clk;
// ...
a1_bad: assert property (@clk a == b)
    else $error("Different values: a = %b, b = %b", a, b);
a2_ok: assert property (@clk a == b)
    else $error("Different values: a = %b, b = %b",
        $sampled(a), $sampled(b));

```

If in some clock tick the sampled value of `a` is 0 and of `b` is 1, but their current values in the Reactive region of this tick are 0, then assertion `a1_bad` will report `Different values: a = 0, b = 0`. This is because action blocks are evaluated in the Reactive region (see [16.14.1](#)). Assertion `a2_ok` reports the intended message `Different values: a = 0, b = 1` because the values of `a` and `b` in its action block are evaluated in the same context as in the assertion.

The following functions are called value change functions and are provided to detect changes in sampled values: `$rose`, `$fell`, `$stable`, and `$changed`.

A value change function detects a change (or, in the case of `$stable`, lack of change) in the sampled value of an expression. The change (or lack of change) is determined by comparing the sampled value of the

expression with the sampled value of the expression from the most recent strictly prior time step in which the clocking event occurred (see [16.5.1](#) for the definition of sampling in past clock ticks and the following description of `$past` for how past values are evaluated). The result of a value change function is true or false and a call to a value change function may be used as a Boolean expression. The results of value change functions shall be determined as follows:

- `$rose` returns true (1'b1) if the LSB of the expression changed to 1. Otherwise, it returns false (1'b0).
- `$fell` returns true (1'b1) if the LSB of the expression changed to 0. Otherwise, it returns false (1'b0).
- `$stable` returns true (1'b1) if the value of the expression did not change. Otherwise, it returns false (1'b0).
- `$changed` returns true (1'b1) if the value of the expression changed. Otherwise, it returns false (1'b0).

When these functions are called at or before the simulation time step in which the first clocking event occurs, the results are computed by comparing the sampled value of the expression with its default sampled value (see [16.5.1](#)).

[Figure 16-3](#) illustrates two examples of value changes:

- Value change expression `e1` is defined as `$rose(req)`.
- Value change expression `e2` is defined as `$fell(ack)`.

The clock ticks used for sampling the variables are derived from the clock for the property, which is different from the simulation time steps. Assume, for now, that this clock is defined elsewhere. At clock tick 3, `e1` occurs because the value of `req` at clock tick 2 was low and the value at clock tick 3 is high. Similarly, `e2` occurs at clock tick 6 because the value of `ack` was sampled as high at clock tick 5 and sampled as low at clock tick 6.

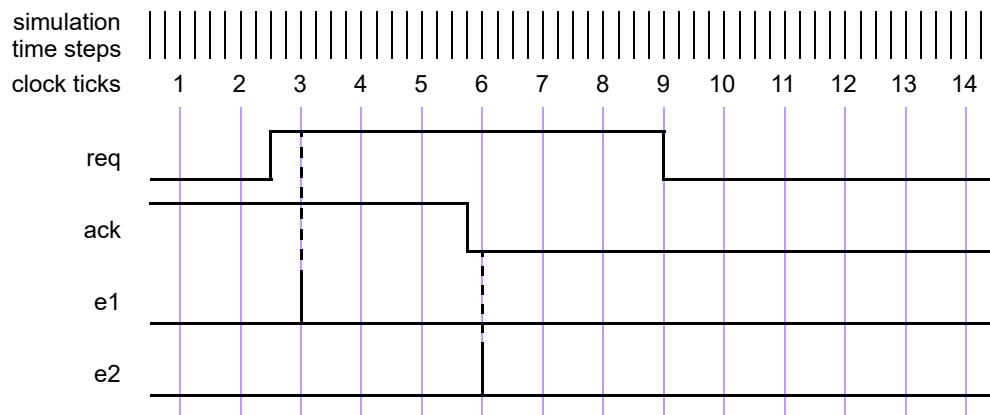


Figure 16-3—Value change expressions

The following example illustrates the use of `$rose` in SystemVerilog code outside assertions:

```
always @(posedge clk)
    reg1 <= a & $rose(b);
```

In this example, the clocking event `@(posedge clk)` is applied to `$rose`. `$rose` is true whenever the sampled value of `b` changed to 1 from its sampled value at the previous tick of the clocking event.

Past sampled values can be accessed with the `$past` function. The following three optional arguments are provided:

- *expression2* is used as a gating expression for the clocking event.
- *number_of_ticks* specifies the number of clock ticks in the past.
- *clocking_event* specifies the clocking event for sampling *expression1*.

expression1 and *expression2* may be any expression allowed in assertions. If *expression2* is not specified, then it defaults to 1'b1.

number_of_ticks shall be 1 or greater and shall be an elaboration-time constant expression. If *number_of_ticks* is not specified, then it defaults to 1.

`$past` returns the sampled value of *expression1* in a particular time step strictly prior to the one in which `$past` is evaluated (see [16.5.1](#) for the definition of sampling in past clock ticks). If *number_of_ticks* equals *k* and if *ev* is the event expression underlying *clocking_event*, then the particular time step is the *k*th strictly prior time step in which the event *ev* **iff** *expression2* occurred. If there do not exist *k* strictly prior time steps in which the event *ev* **iff** *expression2* occurred, then the value returned from the `$past` function is the default sampled value of *expression1* (see [16.5.1](#)).

The clocking event for `$past` shall be explicitly specified through the *clocking_event* argument or inferred from the code where `$past` is called. The rules for inferring the clocking event are described previously.

When intermediate optional arguments between two arguments are not needed, a comma shall be placed for each omitted argument. For example:

```
$past(in1, , enable);
```

Here, a comma is specified to omit *number_of_ticks*. The default of 1 is used for the empty *number_of_ticks* argument. There is no need to include a comma for the omitted *clocking_event* argument, as it does not fall within the specified arguments.

`$past` can be used in any SystemVerilog expression. An example follows.

```
always @(posedge clk)
    reg1 <= a & $past(b);
```

In this example, the inferred clocking event `@(posedge clk)` is applied to `$past`. `$past` is evaluated in the current occurrence of `(posedge clk)` and returns the value of *b* sampled at the previous occurrence of `(posedge clk)`.

The function `$past` may refer to automatic variables, e.g., to procedural loop variables, as follows:

```
always @(posedge clk)
    for (int i = 0; i < 4; i++)
        if (cond[i])
            reg1[i] <= $past(b[i]);
```

According to the definition of the past sampled value (see [16.5.1](#)), `$past` returns at each loop iteration the past value of the *i*-th bit of *b*.

When *expression2* is specified, the sampling of *expression1* is performed based on its clock gated with *expression2*. For example:

```
always @(posedge clk)
  if (enable) q <= d;

always @(posedge clk)
  assert property (done ==> (out == $past(q, 2,enable)) );
```

In this example, the sampling of *q* for evaluating *\$past* is based on the following clocking expression:

```
posedge clk iff enable
```

The clocking event argument of a sampled value function may be different from the clocking event of the context in which it is called, as determined by the clock resolution (see [16.16](#)).

Consider the following assertions:

```
bit clk, fclk, req, gnt, en;
...
a1: assert property
    (@(posedge clk) en && $rose(req) ==> gnt);

a2: assert property
    (@(posedge clk) en && $rose(req, @(posedge fclk)) ==> gnt);
```

Both assertions *a1* and *a2* read: “whenever *en* is high and *req* rises, at the next cycle *gnt* must be asserted.” In both assertions, the rise of *req* occurs if and only if the sampled value of *req* at the current *posedge* of *clk* is 1'b1 and the sampled value of *req* at a particular prior point is distinct from 1'b1. The assertions differ in the specification of the prior point. In *a1* the prior point is the preceding *posedge* of *clk*, while in *a2* the prior point is the most recent prior *posedge* of *fclk*.

As another example,

```
always_ff @(posedge clk1)
  reg1 <= $rose(b, @(posedge clk2));
```

Here, *reg1* is updated in each time step in which *posedge clk1* occurs, using the value returned from the *\$rose* sampled value function in that time step. *\$rose* compares the sampled value of the LSB of *b* from the current time step (one in which *posedge clk1* occurs) with the sampled value of the LSB of *b* in the strictly prior time step in which *posedge clk2* occurs.

The following example is illegal if it is not within the scope of a default clocking because no clock can be inferred:

```
always @(posedge clk) begin
  ...
  @(negedge clk2);
  x = $past(y, 5); // illegal if not within default clocking
end
```

This example is legal if it is within the scope of a default clocking.

16.9.4 Global clocking past and future sampled value functions

This subclause describes the system functions available for accessing the nearest past and future values of an expression as sampled by the global clock. They may be used only if global clocking is defined (see [14.14](#)). These functions include the capability to access the sampled value at the global clock tick that immediately

precedes or follows the time step at which the function is called. Sampled value is explained in [16.5.1](#). The following functions are provided.

Global clocking past sampled value functions are as follows:

```
$past_gclk ( expression )
$rose_gclk ( expression )
$fell_gclk ( expression )
$stable_gclk ( expression )
$changed_gclk ( expression )
```

Global clocking future sampled value functions are as follows:

```
$future_gclk ( expression )
$rising_gclk ( expression )
$falling_gclk ( expression )
$steady_gclk ( expression )
$changing_gclk ( expression )
```

The behavior of the global clocking past sampled value functions can be defined using the sampled value functions as follows (the symbol $\equiv$ means here “is equivalent by definition”):

```
$past_gclk(v)       $\equiv$  $past(v,, @ $global_clock)
$rose_gclk(v)       $\equiv$  $rose(v, @ $global_clock)
$fell_gclk(v)       $\equiv$  $fell(v, @ $global_clock)
$stable_gclk(v)      $\equiv$  $stable(v, @ $global_clock)
$changed_gclk(v)     $\equiv$  $changed(v, @ $global_clock)
```

The global clocking future sampled value functions are similar except that they use the subsequent value of the expression.

$\$future_gclk(v)$ is the sampled value of v at the next global clock tick (see [16.5.1](#) for the definition of sampling in future clock ticks).

The other functions are defined as follows:

- $\$rising_gclk(expression)$ returns true (1'b1) if the sampled value of the LSB of the expression is changing to 1 at the next global clocking tick. Otherwise, it returns false (1'b0).
- $\$falling_gclk(expression)$ returns true (1'b1) if the sampled value of the LSB of the expression is changing to 0 at the next global clocking tick. Otherwise, it returns false (1'b0).
- $\$steady_gclk(expression)$ returns true (1'b1) if the sampled value of the expression does not change at the next global clock tick. Otherwise, it returns false (1'b0).
- $\$changing_gclk(expression)$ is the complement of $\$steady_gclk$, i.e., $!\$steady_gclk(expression)$.

The global clocking future sampled value functions may be invoked only in *property_expr* or in *sequence_expr*; this implies that they shall not be used in assertion action blocks. The global clocking past sampled value functions are a special case of the sampled value functions, and therefore the regular restrictions imposed on the sampled value functions and their arguments apply (see [16.9.3](#)). In particular, the global clocking past sampled value functions are usable in general procedural code and action blocks. Additional restrictions are imposed on the usage of the global clocking future sampled value functions: they shall not be nested and they shall not be used in assertions containing sequence match items (see [16.10](#), [16.11](#)).

The following example illustrates the illegal usage of the global clocking future sampled value functions:

```
// Illegal: global clocking future sampled value functions
// shall not be nested
a1: assert property (@clk $future_gclk(a || $rising_gclk(b));
sequence s;
    bit v;
    (a, v = a) ##1 (b == v) [->1];
endsequence : s

// Illegal: a global clocking future sampled value function shall not
// be used in an assertion containing sequence match items
a2: assert property (@clk s | => $future_gclk(c));
```

Even though global clocking future sampled value functions depend on future values of their arguments, the interval of simulation time steps for an evaluation attempt of an assertion containing global clocking future sampled value functions is defined as though the future sampled values were known in advance. The end of the evaluation attempt is defined to be the last tick of the assertion clock and is not delayed any additional time steps up to the next global clocking tick.

The behavior of **disable iff** and other asynchronous assertion related controls such as `$assertcontrol` (see [20.12](#)) is with respect to the interval of the evaluation attempt previously defined. If, for example, `$assertcontrol` with `control_type 5` (Kill) is executed in a time step strictly after the last tick of the assertion clock for the evaluation attempt, then it shall not affect that attempt, even if `$assertcontrol` is executed no later than the next global clocking tick.

Execution of the action block of an assertion containing global clocking future sampled value functions shall be delayed until the global clocking tick that follows the last tick of the assertion clock for the attempt. If the evaluation attempt fails and `$error` is called by default (see [16.14.1](#)), then `$error` shall be called at the global clocking tick that follows the last tick of the assertion clock.

A tool specific message that reports the starting or ending time step of an evaluation attempt of an assertion containing global clocking future sampled functions shall be consistent with the preceding definition of the interval of simulation time steps for the evaluation attempt. The message may also report the time step in which it is written, which may be that of the global clocking tick that follows the last tick of the assertion clock.

Example 1:

[Table 16-2](#) shows the values returned by the global clocking future sampled value functions for `sig` at different time moments.

The following assertion states that the signal may change only on falling clock:

```
a1: assert property (@$global_clock $changing_gclk(sig)
                    |-> $falling_gclk(clk))
    else $error("sig is not stable");
```

In [Figure 16-4](#), the vertical arrows indicate the ticks of the global clock. The assertion `a1` is violated at time 80 because `$changing_gclk(sig)` is true and `$falling_gclk(clk)` is false. Because the assertion contains global clocking future sampled value functions, the error task `$error("sig is not stable")` in the action block is executed at time 90. If, as part of the tool-specific message printed by `$error`, a tool reports the ending or failing time of this evaluation attempt, the time reported is 80.

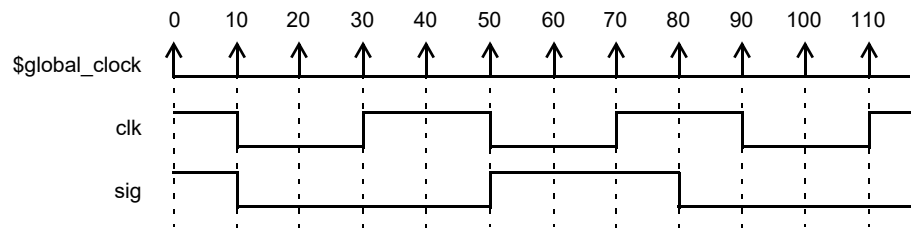


Figure 16-4—Future value change

Table 16-2—Global clocking future sampled value functions

Time	\$sampled(sig)	\$future_gclk(sig)	\$rising_gclk(sig)	\$falling_gclk(sig)	\$changing_gclk(sig)	\$steady_gclk(sig)
10	1'b1	1'b0	1'b0	1'b1	1'b1	1'b0
30	1'b0	1'b0	1'b0	1'b0	1'b0	1'b1
40	1'b0	1'b0	1'b0	1'b0	1'b0	1'b1
50	1'b0	1'b1	1'b1	1'b0	1'b1	1'b0
80	1'b1	1'b0	1'b0	1'b1	1'b1	1'b0

Example 2:

The following assumption states that a signal `sig` must remain stable between two falling edges of a clock `clk` as sampled by global clocking. This differs from the property in Example 1 in the case where the first falling edge of `clk` has not yet occurred. In Example 1, `sig` is not allowed to change in that case, but in this example `sig` can toggle freely while waiting for `clk` to begin.

```
a2: assume property (@$global_clock
    $falling_gclk(clk) ##1 (!$falling_gclk(clk) [*1:$]) |->
        $steady_gclk(sig));
```

Example 3:

Assume that the signal `rst` is high between times 82 and 84, and is low at all other time moments. Then the following assertion:

```
a3: assert property (@$global_clock disable iff (rst) $changing_gclk(sig)
    |-> $falling_gclk(clk))
    else $error("sig is not stable");
```

fails at time 80 (see [Figure 16-4](#)) since `rst` is inactive at time 80. The interval of the failing evaluation attempt starts and ends at time 80. Although `rst` is active prior to the execution of the action block at time 90, the attempt is not disabled.

Example 4:

In some cases, the global clocking future value functions provide a more natural expression of a property than the past value functions. For example, the following two assertions are equivalent:

```
// A ##1 is needed in a4 due to the corner case at cycle 0
a4: assert property (##1 $stable_gclk(sig));
```

```
// In a5, there is no issue at cycle 0
a5: assert property ($steady_gclk(sig));
```

16.9.5 AND operation

The binary operator **and** is used when both operands are expected to match, but the end times of the operand sequences can be different (see [Syntax 16-7](#)).

```
sequence_expr ::= // from A.2.10
...
| sequence_expr and sequence_expr
```

Syntax 16-7—And operator syntax (excerpt from [Annex A](#))

The two operands of **and** are sequences. The requirement for the match of the **and** operation is that both the operands shall match. The operand sequences start at the same time. When one of the operand sequences matches, it waits for the other to match. The end time of the composite sequence is the end time of the operand sequence that completes last.

When *te1* and *te2* are sequences, then the composite sequence

```
te1 and te2
```

matches if *te1* and *te2* match. The end time is the end time of either *te1* or *te2*, whichever matches last.

The following example is a sequence with operator **and**, where the two operands are sequences:

```
(te1 ##2 te2) and (te3 ##2 te4 ##2 te5)
```

The operation as illustrated in [Figure 16-5](#) shows the evaluation attempt at clock tick 8. Here, the two operand sequences are (*te1 ##2 te2*) and (*te3 ##2 te4 ##2 te5*). The first operand sequence requires that first *te1* evaluates to true followed by *te2* two clock ticks later. The second sequence requires that first *te3* evaluates to true followed by *te4* two clock ticks later, followed by *te5* two clock ticks later.

This attempt results in a match because both operand sequences match. The end times of matches for the individual sequences are clock ticks 10 and 12. The end time for the composite sequence is the later of the two end times; therefore, a match is recognized for the composite sequence at clock tick 12.

In the following example, the first operand sequence has a concatenation operator with range from 1 to 5:

```
(te1 ##[1:5] te2) and (te3 ##2 te4 ##2 te5)
```

The first operand sequence requires that *te1* evaluate to true and that *te2* evaluate to true 1, 2, 3, 4, or 5 clock ticks later. The second operand sequence is the same as in the previous example. To consider all possibilities of a match of the composite sequence, the following steps can be taken:

- a) Five threads of evaluation are started for the five possible linear sequences associated with the first sequence operand.
- b) The second operand sequence has only one associated linear sequence; therefore, only one thread of evaluation is started for it.

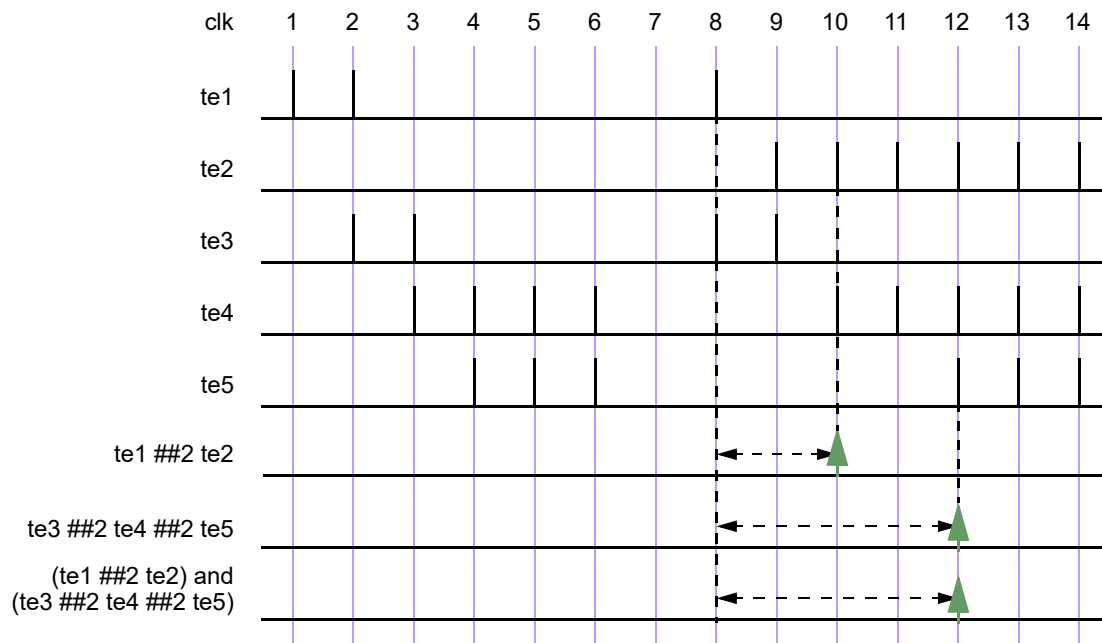


Figure 16-5—ANDing (and) two sequences

- c) [Figure 16-6](#) shows the evaluation attempt beginning at clock tick 8. All five linear sequences for the first operand sequence match, as shown in a time window; therefore, there are five matches of the first operand sequence, ending at clock ticks 9, 10, 11, 12, and 13, respectively. The second operand sequence matches at clock tick 12.
- d) Each match of the first operand sequence is combined with the single match of the second operand sequence, and the rules of the AND operation determine the end time of the resulting match of the composite sequence.

The result of this computation is five matches of the composite sequence, four of them ending at clock tick 12, and the fifth ending at clock tick 13. [Figure 16-6](#) shows the matches of the composite sequence ending at clock ticks 12 and 13.

If `te1` and `te2` are sampled expressions (not sequences), the sequence (`te1 and te2`) matches if `te1` and `te2` both evaluate to true.

An example is illustrated in [Figure 16-7](#), which shows the results for attempts at every clock tick. The sequence matches at clock tick 1, 3, 8, and 14 because both `te1` and `te2` are simultaneously true. At all other clock ticks, match of the AND operation fails because either `te1` or `te2` is false.

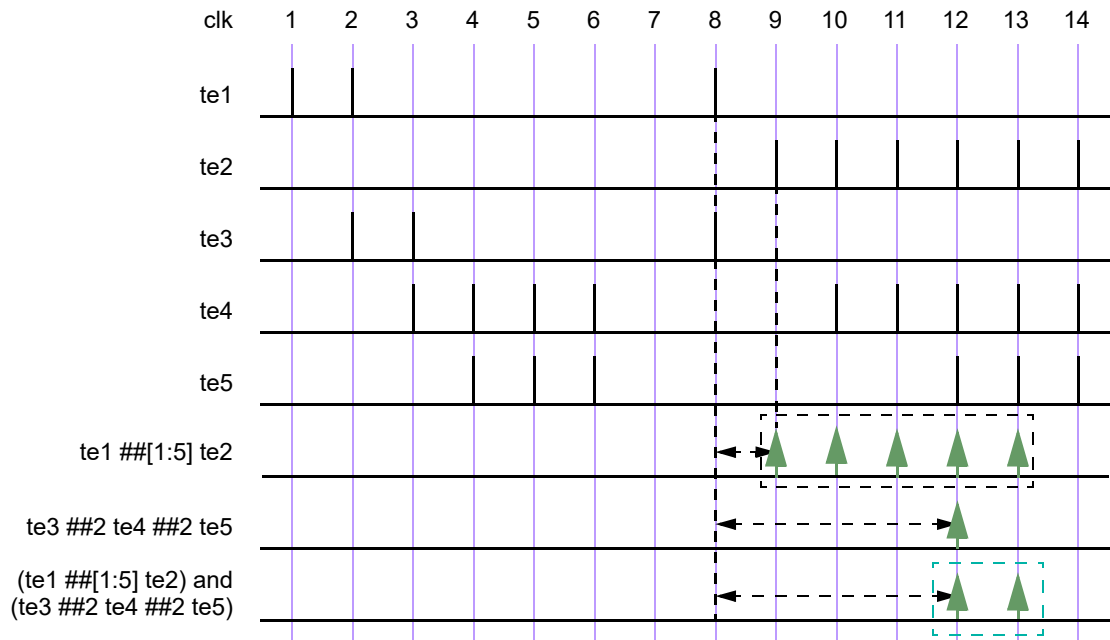


Figure 16-6—ANDing (and) two sequences, including a time range

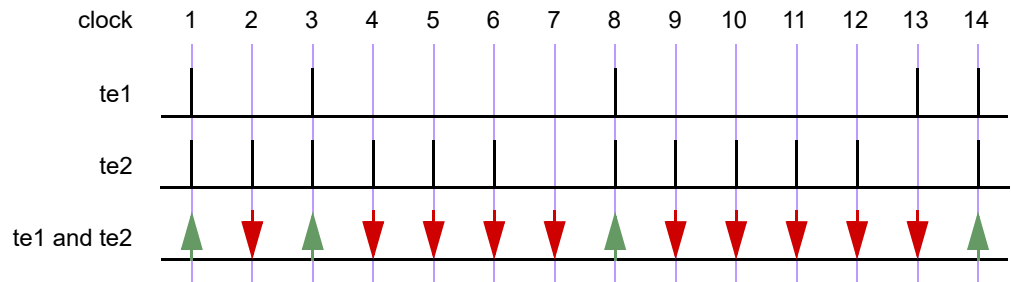


Figure 16-7—ANDing (and) two Boolean expressions

16.9.6 Intersection (AND with length restriction)

The binary operator **intersect** is used when both operand sequences are expected to match, and the end times of the operand sequences must be the same (see [Syntax 16-8](#)).

```
sequence_expr ::= // from A.2.10  
...  
| sequence_expr intersect sequence_expr
```

Syntax 16-8—Intersect operator syntax (excerpt from [Annex A](#))

The two operands of **intersect** are sequences. The requirements for match of the **intersect** operation are as follows:

- Both the operands shall match.
- The lengths of the two matches of the operand sequences shall be the same.

The additional requirement on the length of the sequences is the basic difference between **and** and **intersect**.

An attempted evaluation of an **intersect** sequence can result in multiple matches. The results of such an attempt can be computed as follows:

- Matches of the first and second operands that are of the same length are paired. Each such pair results in a match of the composite sequence, with length and match point equal to the shared length and match point of the paired matches of the operand sequences.
- If no such pair is found, then there is no match of the composite sequence.

[Figure 16-8](#) is similar to [Figure 16-6](#), except that **and** is replaced by **intersect**. In this case, unlike in [Figure 16-6](#), there is only a single match at clock tick 12.

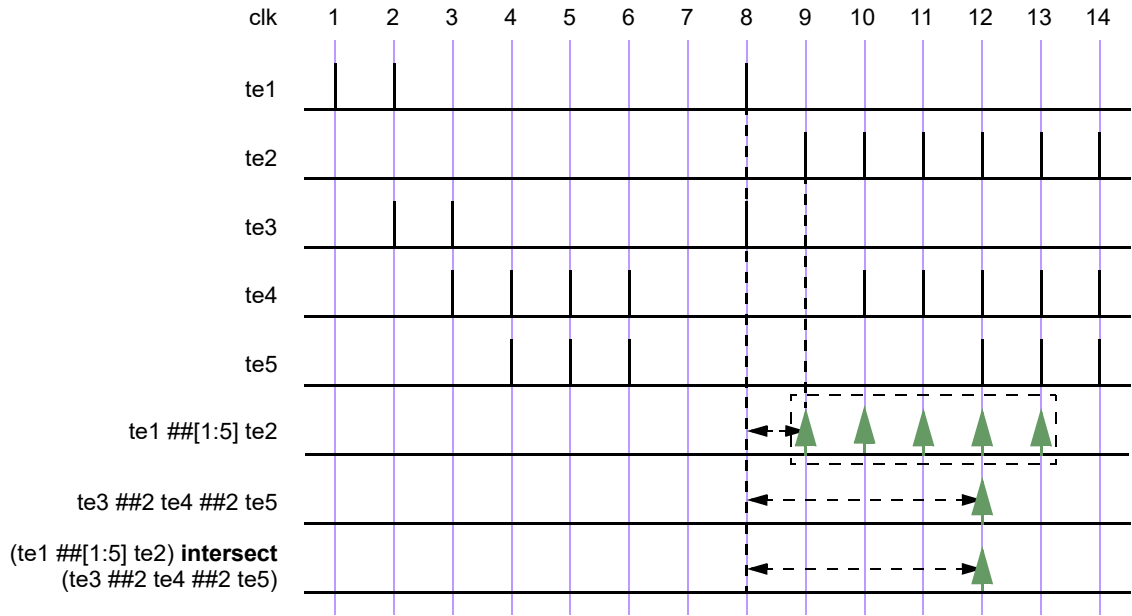


Figure 16-8—Intersecting two sequences

16.9.7 OR operation

The operator **or** is used when at least one of the two operand sequences is expected to match ([Syntax 16-9](#)).

```
sequence_expr ::= // from A.2.10  
...  
| sequence_expr or sequence_expr
```

Syntax 16-9—Or operator syntax (excerpt from [Annex A](#))

The two operands of **or** are sequences.

If the operands `te1` and `te2` are expressions, then

```
te1 or te2
```

matches at any clock tick on which at least one of `te1` and `te2` evaluates to true.

[Figure 16-9](#) illustrates an OR operation for which the operands `te1` and `te2` are expressions. The composite sequence does not match at clock ticks 7 and 13 because `te1` and `te2` are both false at those times. At all other clock ticks, the composite sequence matches, as at least one of the two operands evaluates to true.

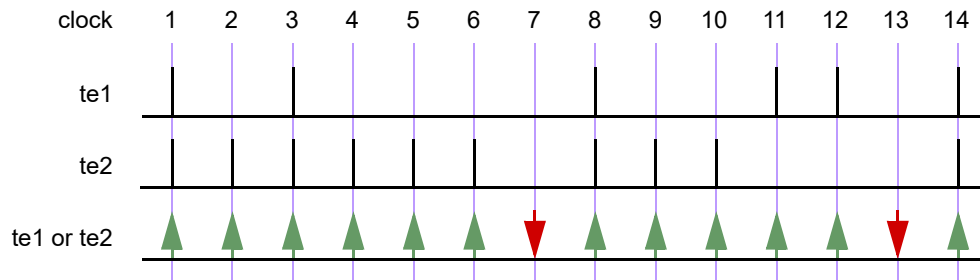


Figure 16-9—ORing (or) two Boolean expressions

When `te1` and `te2` are sequences, then the sequence

```
te1 or te2
```

matches if at least one of the two operand sequences `te1` and `te2` matches. Each match of either `te1` or `te2` constitutes a match of the composite sequence, and its end time as a match of the composite sequence is the same as its end time as a match of `te1` or of `te2`. In other words, the set of matches of `te1 or te2` is the union of the set of matches of `te1` with the set of matches of `te2`.

The following example shows a sequence with operator `or` where the two operands are sequences. [Figure 16-10](#) illustrates this example.

```
(te1 ##2 te2) or (te3 ##2 te4 ##2 te5)
```

The two operand sequences in the preceding example are `(te1 ##2 te2)` and `(te3 ##2 te4 ##2 te5)`. The first sequence requires that `te1` first evaluates to true, followed by `te2` two clock ticks later. The second sequence requires that `te3` evaluates to true, followed by `te4` two clock ticks later, followed by `te5` two clock ticks later. In [Figure 16-10](#), the evaluation attempt for clock tick 8 is shown. The first sequence matches at clock tick 10, and the second sequence matches at clock tick 12. Therefore, two matches for the composite sequence are recognized.

In the following example, the first operand sequence has a concatenation operator with range from 1 to 5:

```
(te1 ##[1:5] te2) or (te3 ##2 te4 ##2 te5)
```

The first operand sequence requires that `te1` evaluate to true and that `te2` evaluate to true 1, 2, 3, 4, or 5 clock ticks later. The second operand sequence requires that `te3` evaluate to true, that `te4` evaluate to true two clock ticks later, and that `te5` evaluate to true another two clock ticks later. The composite sequence matches at any clock tick on which at least one of the operand sequences matches. As shown in [Figure 16-11](#), for the attempt at clock tick 8, the first operand sequence matches at clock ticks 9, 10, 11, 12, and 13, while the second operand matches at clock tick 12. The composite sequence, therefore, has one match at each of clock ticks 9, 10, 11, and 13 and has two matches at clock tick 12.

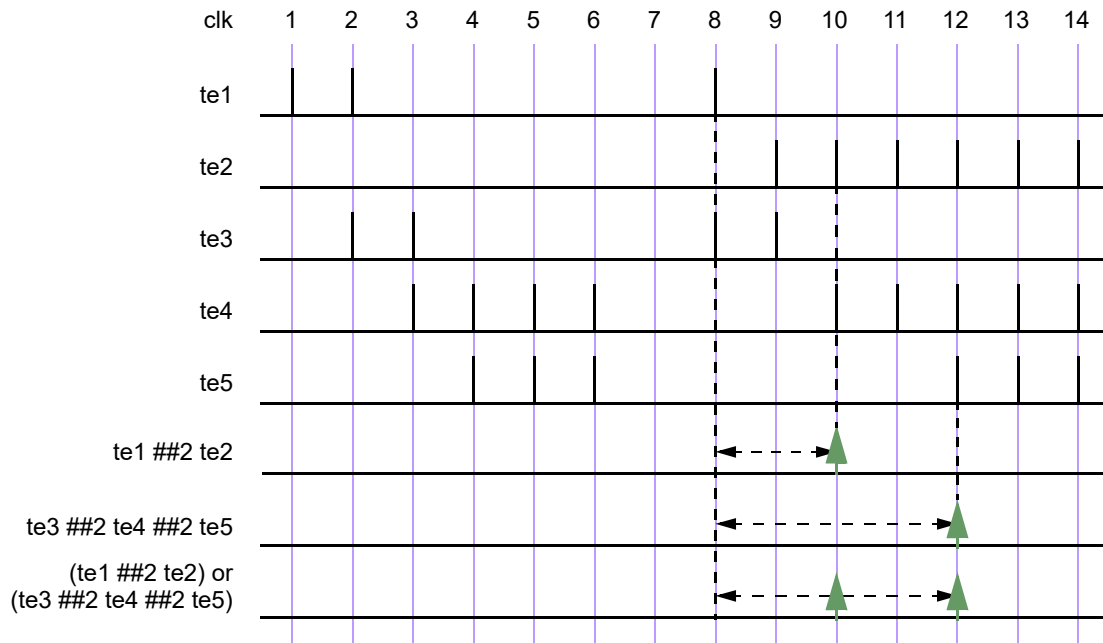


Figure 16-10—ORing (or) two sequences

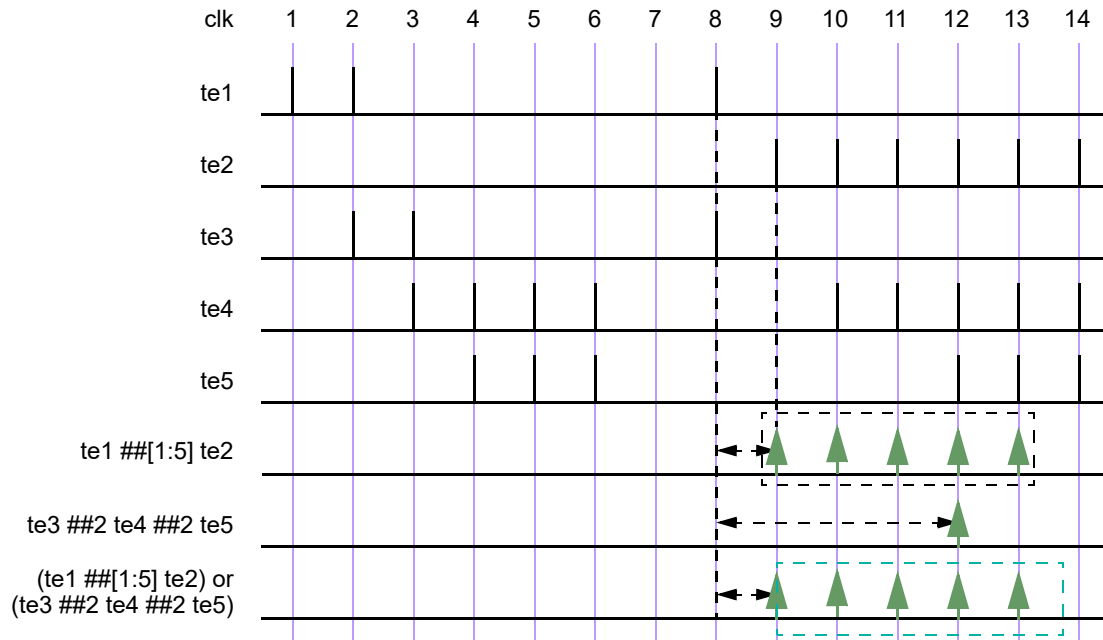


Figure 16-11—ORing (or) two sequences, including a time range

16.9.8 First_match operation

The **first_match** operator matches only the first of possibly multiple matches for an evaluation attempt of its operand sequence. This allows all subsequent matches to be discarded from consideration. In particular, when a sequence is a subsequence of a larger sequence, then applying the **first_match** operator has significant effect on the evaluation of the enclosing sequence (see [Syntax 16-10](#)).

```
sequence_expr ::= // from A.2.10  
...  
| first_match ( sequence_expr { , sequence_match_item } )
```

Syntax 16-10—First_match operator syntax (excerpt from [Annex A](#))

An evaluation attempt of **first_match**(*seq*) results in an evaluation attempt for the operand *seq* beginning at the same clock tick. If the evaluation attempt for *seq* produces no match, then the evaluation attempt for **first_match**(*seq*) produces no match. Otherwise, the match of *seq* with the earliest ending clock tick is a match of **first_match**(*seq*). If there are multiple matches of *seq* with the same ending clock tick as the earliest one, then all those matches are matches of **first_match**(*seq*).

The following example shows a variable delay specification:

```
sequence t1;  
    t1 ## [2:5] te2;  
endsequence  
sequence ts1;  
    first_match(t1 ## [2:5] te2);  
endsequence
```

Here, *te1* and *te2* are expressions. Each attempt of sequence *t1* can result in matches for up to four of the following sequences:

```
t1 ##2 te2  
t1 ##3 te2  
t1 ##4 te2  
t1 ##5 te2
```

However, sequence *ts1* can result in a match for only one of the preceding four sequences. Whichever match of the preceding four sequences ends first is a match of sequence *ts1*.

For example:

```
sequence t2;  
    (a ##[2:3] b) or (c ##[1:2] d);  
endsequence  
sequence ts2;  
    first_match(t2);  
endsequence
```

Each attempt of sequence *t2* can result in matches for up to four of the following sequences:

```
a ##2 b  
a ##3 b  
c ##1 d  
c ##2 d
```

Sequence `ts2` matches only the earliest ending match of these sequences. If `a`, `b`, `c`, and `d` are expressions, then it is possible to have matches ending at the same time for both.

```
a ##2 b
c ##2 d
```

If both of these sequences match and `(c ##1 d)` does not match, then evaluation of `ts2` results in these two matches.

Sequence match items can be attached to the operand sequence of the **first_match** operator. The sequence match items are placed within the same set of parentheses that enclose the operand. Thus, for example, the local variable assignment `x = e` can be attached to the first match of `seq` via

```
first_match(seq, x = e)
```

which is equivalent to the following:

```
first_match((seq, x = e))
```

See [16.10](#) and [16.11](#) for discussion of sequence match items.

16.9.9 Conditions over sequences

Sequences often occur under the assumptions of some conditions for correct behavior. A logical condition must hold true, for instance, while processing a transaction. Also, occurrence of certain values is prohibited while processing a transaction. Such situations can be expressed directly using the construct shown in [Syntax 16-11](#).

```
sequence_expr ::=                                     //from A.2.10
...
| expression_or_dist throughout sequence_expr
```

Syntax 16-11—Throughout construct syntax (excerpt from [Annex A](#))

The construct `exp throughout seq` is an abbreviation for the following:

```
(exp) [*0:$] intersect seq
```

The composite sequence, `exp throughout seq`, matches along a finite interval of consecutive clock ticks provided `seq` matches along the interval and `exp` evaluates to true at each clock tick of the interval.

The following example is illustrated in [Figure 16-12](#).

```
sequence burst_rule1;
  @(posedge mclk)
    $fell(burst_mode) ##0
    ((!burst_mode) throughout (##2 ((trdy==0)&&(irdy==0)) [*7]));
endsequence
```

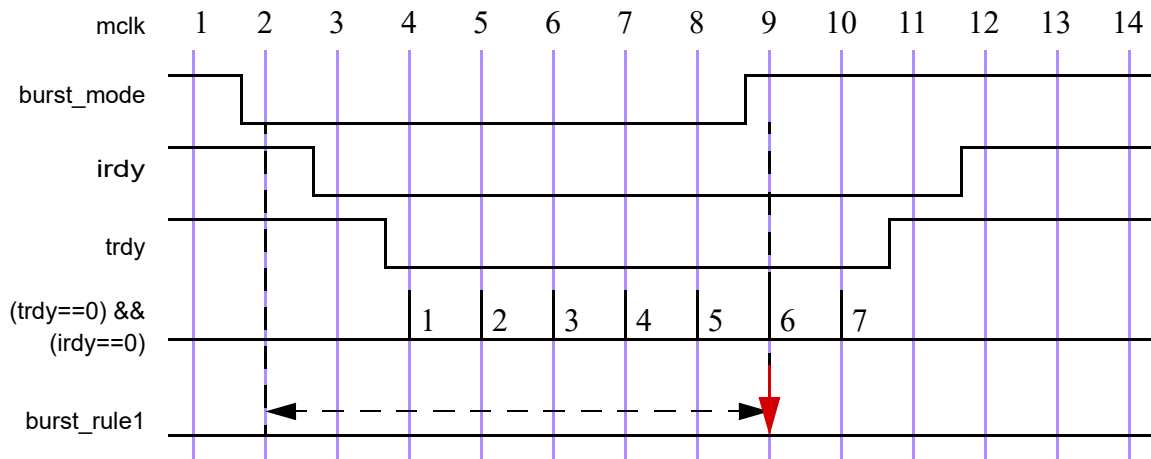


Figure 16-12—Match with throughout restriction fails

Figure 16-13 illustrates the evaluation attempt for sequence `burst_rule1` beginning at clock tick 2. Because signal `burst_mode` is high at clock tick 1 and low at clock tick 2, `$fell(burst_mode)` is true at clock tick 2. To complete the match of `burst_rule1`, the value of `burst_mode` is required to be low throughout a match of the subsequence `(##2 ((trdy==0) && (irdy==0)) [*7])` beginning at clock tick 2. This subsequence matches from clock tick 2 to clock tick 10. However, at clock tick 9 `burst_mode` becomes high, thereby failing to match according to the rules for **throughout**.

If signal `burst_mode` were instead to remain low through at least clock tick 10, then there would be a match of `burst_rule1` from clock tick 2 to clock tick 10, as shown in Figure 16-13.

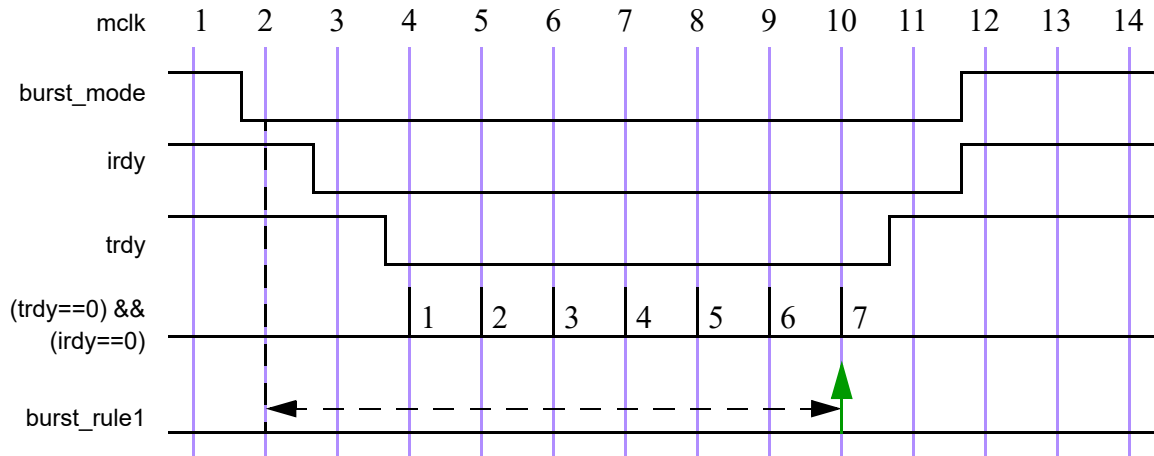


Figure 16-13—Match with throughout restriction succeeds

16.9.10 Sequence contained within another sequence

The containment of a sequence within another sequence is expressed as follows in Syntax 16-12.

```
sequence_expr ::= // from A.2.10
...
| sequence_expr within sequence_expr
```

Syntax 16-12—Within construct syntax (excerpt from [Annex A](#))

The construct `seq1 within seq2` is an abbreviation for the following:

```
(1[*0:$] ##1 seq1 ##1 1[*0:$]) intersect seq2
```

The composite sequence `seq1 within seq2` matches along a finite interval of consecutive clock ticks provided `seq2` matches along the interval and `seq1` matches along some subinterval of consecutive clock ticks. In other words, the matches of `seq1` and `seq2` must satisfy the following:

- The start point of the match of `seq1` shall be no earlier than the start point of the match of `seq2`.
- The match point of `seq1` shall be no later than the match point of `seq2`.

For example, the sequence

```
!trdy[*7] within ($fell(irdy) ##1 !irdy[*8])
```

matches from clock tick 3 to clock tick 11 on the trace shown in [Figure 16-13](#).

16.9.11 Composing sequences from simpler subsequences

There are two ways in which a complex sequence can be composed using simpler subsequences.

One is to instantiate a named sequence by referencing its name. Evaluation of such a reference requires the named sequence to match starting from the clock tick at which the reference is reached during the evaluation of the enclosing sequence. For example:

```
sequence s;
  a ##1 b ##1 c;
endsequence
sequence rule;
  @(posedge sysclk)
    trans ##1 start_trans ##1 s ##1 end_trans;
endsequence
```

Sequence `s` is evaluated beginning one tick after the evaluation of `start_trans` in the sequence `rule`.

Another way to use a sequence is to detect its end point in another sequence. The reaching of the end point (see [16.7](#)) can be tested by using the method `triggered`.

To detect the end point, the `triggered` method may be applied to a named sequence instance, with or without arguments, an untyped formal argument, or a formal argument of type **sequence**, where such is allowed, as follows:

```
sequence_instance.triggered
or
formal_argument_sequence.triggered
```

`triggered` is a method on a sequence. The result of its operation is true (1'b1) or false (1'b0). When method `triggered` is evaluated in an expression, it tests whether its operand sequence has reached its end point at that particular point in time. The result of `triggered` does not depend upon the starting point of the match of its operand sequence. An example is shown as follows:

```
sequence e1;
    @(posedge sysclk) $rose(ready) ##1 proc1 ##1 proc2 ;
endsequence
sequence rule;
    @(posedge sysclk) reset ##1 inst ##1 e1.triggered ##1 branch_back;
endsequence
```

In this example, sequence `e1` must match one clock tick after `inst`. If the method `triggered` is replaced with an instance of sequence `e1`, a match of `e1` must start one clock tick after `inst`. Notice that method `triggered` only tests for the end point of `e1` and has no bearing on the starting point of `e1`.

The following example demonstrates an application of the method `triggered` on a named sequence instance with arguments:

```
sequence e2(a,b,c);
    @(posedge sysclk) $rose(a) ##1 b ##1 c;
endsequence
sequence rule2;
    @(posedge sysclk) reset ##1 inst ##1 e2(ready,proc1,proc2).triggered
    ##1 branch_back;
endsequence
```

`rule2` is equivalent to `rule2a` as follows:

```
sequence e2_instantiated;
    e2(ready,proc1,proc2);
endsequence
sequence rule2a;
    @(posedge sysclk) reset ##1 inst ##1 e2_instantiated.triggered ##1
    branch_back;
endsequence
```

The following example demonstrates an application of method `triggered` on a formal argument of type `sequence`:

```
sequence e3(sequence a, untyped b);
    @(posedge sysclk) a.triggered ##1 b;
endsequence

sequence rule3;
    @(posedge sysclk) reset ##1 e3(ready ##1 proc1, proc2) ##1 branch_back;
endsequence
```

There are additional restrictions on passing local variables into an instance of a sequence to which `triggered` is applied (see [16.10](#)).

The method `triggered` can be used in the presence of multiple clocks. However, the ending clock of the sequence instance to which `triggered` is applied shall always be the same as the clock in the context where the application of method `triggered` appears (see [16.13.5](#)).

If a sequence admits an empty match, such empty matches shall not activate the `.triggered` method. For example, consider the following sequence, which admits both empty and nonempty matches:

```
sequence zero_or_one_req;
    (req==1'b1) [*0:1];
endsequence
```

The method `zero_or_one_req.triggered()` will only return true (1'b1) when the sampled value of `req` is 1'b1, resulting in a nonempty match.

16.10 Local variables

Data can be manipulated within named sequences (see [16.8](#)) and properties (see [16.12](#)) using dynamically created local variables. The use of a static SystemVerilog variable implies that only one copy exists. If data values need to be checked in pipelined designs, then for each quantum of data entering the pipeline, a separate variable can be used to store the predicted output of the pipeline for later comparison when the result actually exits the pipe. This storage can be built by using an array of variables arranged in a shift register to mimic the data propagating through the pipeline. However, in more complex situations where the latency of the pipe is variable and out of order, this construction could become very complex and error prone. Therefore, variables are needed that are local to and are used within a particular transaction check that can span an arbitrary interval of time and can overlap with other transaction checks. Such a variable will thus be dynamically created when needed within an instance of a sequence and removed when the end of the sequence is reached.

The dynamic creation of a local variable and its assignment is achieved by either using a local variable formal argument declaration (see [16.8.2](#), [16.12.18](#)) or using an assertion variable declaration within the declaration of a named sequence or property (see [16.12](#)). Without further specification, the term *local variable* shall mean either a local variable formal argument or a local variable declared in an *assertion_variable_declaration*. Without further specification, the term *local variable initialization assignment* shall mean either an initialization assignment to a local variable formal argument of direction **input** or **inout** of the value of the corresponding actual argument or a declaration assignment to a local variable declared in an *assertion_variable_declaration* (see [Syntax 16-13](#)).

```
assertion_variable_declaration ::=                                     // from A.2.10
    var_data_type list_of_variable_decl_assignments ;
```

Syntax 16-13—Assertion variable declaration syntax (excerpt from [Annex A](#))

The data type of an assertion variable declaration shall be specified explicitly. The data type shall be one of the types allowed within assertions as defined in [16.6](#). The data type shall be followed by a comma-separated list of one or more identifiers with optional declaration assignments. A declaration assignment, if present, defines the initial value to be placed in the corresponding local variable. The initial value is defined by an expression, which need not be constant.

The sampled value of a local variable is defined as the current value (see [16.5.1](#)).

At the beginning of each evaluation attempt of an instance of a named sequence or property, a new copy of each of its local variables shall be created and, if present, the corresponding initialization assignment shall be performed. Initialization assignments shall be performed in the Observed region in the order that they appear in the sequence or property declaration. For the purposes of this rule, all initialization assignments to local variable formal arguments shall be performed before any initialization assignment to a local variable declared in an *assertion_variable_declaration*. An initialization assignment to a local variable uses the

sampled value of its expression in the time slot in which the evaluation attempt begins. The expression of an initialization assignment to a given local variable may refer to a previously declared local variable. In this case the previously declared local variable shall itself have an initialization assignment, and the initial value assigned to the previously declared local variable shall be used in the evaluation of the expression assigned to the given local variable. Local variables do not have default initial values. A local variable without an initialization assignment shall be unassigned at the beginning of the evaluation attempt.

For example, at the beginning of an evaluation attempt of an instance of

```
sequence s;
  logic u, v = a, w = v || b;
  ...
endsequence
```

the assignment of *a* to *v* is performed first, and the assignment of *v* || *b* to *w* is performed second. The value assigned to *w* is the same as would result from the declaration assignment *w* = *a* || *b*. The local variable *u* is unassigned at the beginning of the evaluation attempt.

Local variables may be assigned and reassigned within the body of the sequence or property in which they are declared.

```
sequence_expr ::= // from A.2.10
  ...
  | ( sequence_expr { , sequence_match_item } ) [ sequence_abbrev ]
  ...
sequence_match_item ::=
  operator_assignment
  | inc_or_dec_expression
  ...
```

Syntax 16-14—Variable assignment syntax (excerpt from [Annex A](#))

One or more local variables may be assigned at the end point of a syntactic subsequence by placing the subsequence, comma-separated from the list of local variable assignments, in parentheses. At the end of any nonempty match of the subsequence, the local variable assignments are performed in the order that they appear in the list. For example, if in

```
a ##1 b[->1] ##1 c[*2]
```

it is desired to assign *x* = *e* and then *y* = *x* && *f* at the match of *b*[->1], the sequence can be rewritten as

```
a ##1 (b[->1], x = e, y = x && f) ##1 c[*2]
```

A local variable may be reassigned later in the sequence or property, as in

```
a ##1 (b[->1], x = e, y = x && f) ##1 (c[*2], x &= g)
```

The subsequence to which a local variable assignment is attached shall not admit an empty match (see [16.12.22](#)). For example, the sequence

```
a ##1 (b[*0:1], x = e) ##1 c[*2] // illegal
```

is illegal because the subsequence *b*[*0:1] can match the empty word. The sequence

```
(a ##1 b[*0:1], x = e) ##1 c[*2] // legal
```

is legal because the concatenated subsequence `a ##1 b[*0:1]` cannot match the empty word.

A local variable may be referenced within the sequence or property in which it is declared. The sequence or property shall assign a value to the local variable prior to the point at which the reference is made. The prior assignment may be an initialization assignment or an assignment attached to a subsequence. There is an implicit reference associated with the use of an *inc_or_dec_operator* or an assignment operator other than “=”. Therefore, a local variable shall be assigned a value prior to being updated with an *inc_or_dec_operator* or with an assignment operator other than “=”.

Under certain circumstances, a local variable that is assigned later becomes unassigned. If a local variable does not flow out of a subsequence (see the following), then the local variable shall become unassigned at the end of that subsequence, regardless of whether it was assigned a value prior to that point. The local variable shall not be referenced after the point from which it does not flow until after it has again been assigned a value. See [F.5.4](#) for precise conditions defining local variable flow.

Hierarchical references to a local variable are not allowed.

As an example of local variable usage, assume a pipeline that has a fixed latency of five clock cycles. The data enter the pipe on `pipe_in` when `valid_in` is true, and the value computed by the pipeline appears five clock cycles later on the signal `pipe_out1`. The data as transformed by the pipe are predicted by a function that increments the data. The following property verifies this behavior:

```
property e;
  int x;
  (valid_in, x = pipe_in) |-> ##5 (pipe_out1 == (x+1));
endproperty
```

Property `e` is evaluated as follows:

- When `valid_in` is true, `x` is assigned the value of `pipe_in`. If five cycles later, `pipe_out1` is equal to `x+1`, then property `e` is true. Otherwise, property `e` is false.
- When `valid_in` is false, property `e` evaluates to true.

A local variable can be used to form expressions in the same way that a static variable of the same type can be used. This includes the use of local variables in expressions for bit-selects and part-selects of vectors or for indices of arrays. A local variable shall not be used in a clocking event expression.

Local variables may be used in sequences or properties.

```
sequence data_check;
  int x;
  a ##1 (!a, x = data_in) ##1 !b[*0:$] ##1 b && (data_out == x);
endsequence
property data_check_p
  int x;
  a ##1 (!a, x = data_in) |=> !b[*0:$] ##1 b && (data_out == x);
endproperty
```

Local variable assignments may be attached to the operand sequence of a repetition and accomplish accumulation of values.

```
sequence rep_v;
  int x = 0;
```

```
(a[>1], x += data)[*4] ##1 b ##1 c && (data_out == x);
endsequence
```

An accumulating local variable may be used to count the number of times a condition is repeated, as in the following example:

```
sequence count_a_cycles;
  int x;
  ($rose(a), x = 1)
  ##1 (a, x++)[*0:$]
  ##1 !a && (x <= MAX);
endsequence
```

The local variables declared within a sequence or property are not visible in the context where the sequence or property is instantiated. The following example illustrates an illegal access to local variable `v1` of sequence `sub_seq1` in sequence `seq1`.

```
sequence sub_seq1;
  int v1;
  (a ##1 !a, v1 = data_in) ##1 !b[*0:$] ##1 b && (data_out == v1);
endsequence
sequence seq1;
  c ##1 sub_seq1 ##1 (d1 == v1); // error because v1 is not visible
endsequence
```

It can be useful to assign a value to a local variable within an instance of a named sequence and reference the local variable in the instantiating context at or after the completion of a match of the instance. The rules for assigning values to a local variable within an instance of a named sequence are described in [16.8.2](#). This capability is also supported under the following conditions:

- The local variable shall be declared outside the named sequence, and its scope shall include both the instance of the named sequence and the desired reference in the instantiating context.
- The local variable shall be passed as an entire actual argument in the list of arguments of the instance of the named sequence.
- The corresponding formal argument shall be untyped.

The named sequence may specify assignments to the formal argument in one or more *sequence_match_items*.

The following example illustrates this usage:

```
sequence sub_seq2(lv);
  (a ##1 !a, lv = data_in) ##1 !b[*0:$] ##1 b && (data_out == lv);
endsequence
sequence seq2;
  int v1;
  c ##1 sub_seq2(v1) // v1 is bound to lv
  ##1 (d1 == v1); // v1 holds the value that was assigned to lv
endsequence
```

An alternative way to achieve a similar capability is by using local variable formal arguments (see [16.8.2](#)).

Local variables can be passed into an instance of a named sequence to which `triggered` is applied and accessed in a similar manner. For example:

```
sequence seq2a;
    int v1; c ##1 sub_seq2(v1).triggered ##1 (do1 == v1);
    // v1 is now bound to lv
endsequence
```

There are additional restrictions when passing local variables into an instance of a named sequence to which `triggered` is applied:

- Local variables can be passed in only as entire actual arguments, not as proper subexpressions of actual arguments.
- In the declaration of the named sequence, the formal argument to which the local variable is bound shall not be referenced before it is assigned.

The second restriction is met by `sub_seq2` because the assignment `lv = data_in` occurs before the reference to `lv` in `data_out == lv`.

If a local variable is assigned before being passed into an instance of a named sequence to which `triggered` is applied, then the restrictions prevent this assigned value from being visible within the named sequence. The restrictions are important because the use of `triggered` means that there is no guaranteed relationship between the point in time at which the local variable is assigned outside the named sequence and the beginning of the match of the instance.

A local variable that is passed in as actual argument to an instance of a named sequence to which `triggered` is applied will flow out of the application of `triggered` to that instance provided both of the following conditions are met:

- The local variable flows out of the end of the named sequence instance, as defined by the local variable flow rules for sequences. (See the following and [F.5.4](#).)
- The application of `triggered` to this instance is a maximal Boolean expression. In other words, the application of `triggered` cannot have negation or any other expression operator applied to it.

Both conditions are satisfied by `sub_seq2` and `seq2a`. Thus, in `seq2a`, the value in `v1` in the comparison `do1 == v1` is the value assigned to `lv` in `sub_seq2` by the assignment `lv = data_in`. However, in

```
sequence seq2b;
    int v1; c ##1 !sub_seq2(v1).triggered ##1 (do1 == v1); // v1 unassigned
endsequence
```

the second condition is violated because of the negation applied to `sub_seq2(v1).triggered`. Therefore, `v1` does not flow out of the application of `triggered` to this instance, and the reference to `v1` in `do1 == v1` is to an unassigned variable.

In a single cycle, there can be multiple matches of a sequence instance to which `triggered` is applied, and these matches can have different valuations of the local variables. The multiple matches are treated semantically the same way as matching both disjuncts of an `or` (see the following). In other words, the thread evaluating the instance to which `triggered` is applied will fork to account for such distinct local variable valuations.

When a local variable is a formal argument of a sequence declaration, it is illegal to declare the variable, as shown in the following example:

```
sequence sub_seq3(lv);
    int lv; // illegal because lv is a formal argument
    (a ##1 !a, lv = data_in) ##1 !b[*0:$] ##1 b && (data_out == lv);
endsequence
```

There are special considerations when using local variables in sequences involving the branching operators **or**, **and**, and **intersect**. The evaluation of a composite sequence constructed from one of these operators can be thought of as forking two threads to evaluate the operand sequences in parallel. A local variable may have been assigned a value before the start of the evaluation of the composite sequence, either from an initialization assignment or from an assignment attached to a preceding subsequence. Such a local variable is said to *flow in* to each of the operand sequences. The local variable may be assigned or reassigned in one or both of the operand sequences. In general, there is no guarantee that evaluation of the two threads results in consistent values for the local variable, or even that there is a consistent view of whether the local variable has been assigned a value. Therefore, the values assigned to the local variable before and during the evaluation of the composite sequence are not always allowed to be visible after the evaluation of the composite sequence.

In some cases, inconsistency in the view of the local variable's value does not matter, while in others it does. Precise conditions are given in [F.5.4](#) to define static (i.e., compile-time computable) conditions under which a sufficiently consistent view of the local variable's value after the evaluation of the composite sequence is provided. If these conditions are satisfied, then the local variable is said to *flow out* of the composite sequence. Otherwise, the local variable shall become unassigned at the end of the composite sequence. An intuitive description of the conditions for local variable flow follows:

- a) Variables assigned on parallel threads cannot be accessed in sibling threads. For example:

```
sequence s4;
  int x;
  (a ##1 (b, x = data) ##1 c) or (d ##1 (e==x)); // illegal
endsequence
```

- b) In the case of **or**, a local variable flows out of the composite sequence if, and only if, it flows out of each of the operand sequences. If the local variable is not assigned before the start of the composite sequence and it is assigned in only one of the operand sequences, then it does not flow out of the composite sequence.
- c) Each thread for an operand of an **or** that matches its operand sequence continues as a separate thread, carrying with it its own latest assignments to the local variables that flow out of the composite sequence. These threads do not have to have consistent valuations for the local variables. For example:

```
sequence s5;
  int x,y;
  ((a ##1 (b, x = data, y = data1) ##1 c)
   or (d ##1 (`true, x = data) ##0 (e==x))) ##1 (y==data2);
  // illegal because y is not in the intersection
endsequence
sequence s6;
  int x,y;
  ((a ##1 (b, x = data, y = data1) ##1 c)
   or (d ##1 (`true, x = data) ##0 (e==x))) ##1 (x==data2);
  // legal because x is in the intersection
endsequence
```

- d) In the case of **and** and **intersect**, a local variable that flows out of at least one operand shall flow out of the composite sequence unless it is blocked. A local variable is blocked from flowing out of the composite sequence if either of the following statements applies:
- 1) The local variable is assigned in and flows out of each operand of the composite sequence, or
 - 2) The local variable is blocked from flowing out of at least one of the operand sequences.

The value of a local variable that flows out of the composite sequence is the latest assigned value. The threads for the two operands are merged into one at completion of evaluation of the composite sequence.

```
sequence s7;
  int x,y;
  ((a ##1 (b, x = data, y = data1) ##1 c)
   and (d ##1 (`true, x = data) ##0 (e==x))) ##1 (x==data2);
  // illegal because x is common to both threads
endsequence
sequence s8;
  int x,y;
  ((a ##1 (b, x = data, y = data1) ##1 c)
   and (d ##1 (`true, x = data) ##0 (e==x))) ##1 (y==data2);
  // legal because y is in the difference
endsequence
```

16.11 Calling subroutines on match of a sequence

Tasks, task methods, void functions, void function methods, and system tasks can be called at the end of a successful nonempty match of a sequence. The subroutine calls, like local variable assignments, appear in the comma-separated list that follows the sequence. The subroutine calls are said to be attached to the sequence. It shall be an error to attach a subroutine call or any *sequence_match_item* to a sequence that admits an empty match (see [16.12.22](#)). The sequence and the list that follows are enclosed in parentheses (see [Syntax 16-15](#)).

```
sequence_expr ::= // from A.2.10
  ...
  | ( sequence_expr { , sequence_match_item } ) [ sequence_abbrev ]
  ...
sequence_match_item ::=
  operator_assignment
  | inc_or_dec_expression
  | subroutine_call
```

Syntax 16-15—Subroutine call in sequence syntax (excerpt from [Annex A](#))

For example:

```
sequence s1;
  logic v, w;
  (a, v = e) ##1
  (b[->1], w = f, $display("b after a with v = %h, w = %h\n", v, w));
endsequence
```

defines a sequence `s1` that matches at the first occurrence of `b` strictly after an occurrence of `a`. At the match, the system task `$display` is executed to write a message that announces the match and shows the values assigned to the local variables `v` and `w`.

All subroutine calls attached to a sequence are executed at every end point of the sequence. For each end point, the attached calls are executed in the order they appear in the list. Assertion evaluation does not wait on or receive data back from any attached subroutine. The subroutines are scheduled in the Reactive region, like an action block.

Each argument of a subroutine call attached to a sequence shall either be passed by value as an input or be passed by reference (either **ref** or **const ref**; see [13.5.2](#)). Actual argument expressions that are passed by value use sampled values of the underlying variables and are consistent with the variable values used to

evaluate the sequence match. The variable passed by value as an input shall be of a type allowed in [16.6](#). An automatic variable may be passed as a constant input for a subroutine call from an assertion statement in procedural code (see [16.14.6.1](#)). An automatic variable shall not be passed by reference nor passed as a non-constant input to a subroutine call from an assertion statement in procedural code. The rules for passing elements of dynamic arrays, queues, and associative arrays as ref arguments are described in [13.5.2](#).

Local variables can be passed into subroutine calls attached to a sequence. Any local variable that flows out of the sequence or that is assigned in the list following the sequence, but before the subroutine call, can be used in an actual argument expression for the call. If a local variable appears in an actual argument expression, then that argument shall be passed by value.

16.12 Declaring properties

A property defines a behavior of the design. A named property may be used for verification as an assumption, an obligation, or a coverage specification. In order to use the behavior for verification, an **assert**, **assume**, or **cover** statement must be used. A property declaration by itself does not produce any result.

A named **property** may be declared in any of the following:

- A module
- An interface
- A program
- A **clocking** block
- A package
- A compilation-unit scope
- A generate block
- A checker

To declare a named property, the **property** construct is used as shown in [Syntax 16-16](#).

```

assertion_item_declaration ::=                                     // from A.2.10
    property_declaration
    ...
property_declaration ::=
    property property_identifier [ ( [ property_port_list ] ) ] ;
    { assertion_variable_declaration }
    property_spec [ ; ]
    endproperty [ : property_identifier ]
property_port_list ::=
    property_port_item { , property_port_item }
property_port_item ::=
    { attribute_instance } [ local [ property_lvar_port_direction ] ] property_formal_type
    formal_port_identifier { variable_dimension } [ = property_actual_arg ]
property_lvar_port_direction ::= input
property_formal_type ::=
    sequence_formal_type
    | property
property_spec ::=
    [ clocking_event ] [ disable iff ( expression_or_dist ) ] property_expr

```

```

property_expr ::=
    sequence_expr
  | strong ( sequence_expr )
  | weak ( sequence_expr )
  | ( property_expr )
  | not property_expr
  | property_expr or property_expr
  | property_expr and property_expr
  | sequence_expr |-> property_expr
  | sequence_expr ==> property_expr
  | if ( expression_or_dist ) property_expr [ else property_expr ]
  | case ( expression_or_dist ) property_case_item { property_case_item } endcase
  | sequence_expr #- property_expr
  | sequence_expr #= property_expr
  | nexttime property_expr
  | nexttime [ constant_expression ] property_expr
  | s_nexttime property_expr
  | s_nexttime [ constant_expression ] property_expr
  | always property_expr
  | always [ cycle_delay_const_range_expression ] property_expr
  | s_always [ constant_range ] property_expr
  | s_eventually property_expr
  | eventually [ constant_range ] property_expr
  | s_eventually [ cycle_delay_const_range_expression ] property_expr
  | property_expr until property_expr
  | property_expr s_until property_expr
  | property_expr until_with property_expr
  | property_expr s_until_with property_expr
  | property_expr implies property_expr
  | property_expr iff property_expr
  | accept_on ( expression_or_dist ) property_expr
  | reject_on ( expression_or_dist ) property_expr
  | sync_accept_on ( expression_or_dist ) property_expr
  | sync_reject_on ( expression_or_dist ) property_expr
  | property_instance
  | clocking_event property_expr

property_case_item ::=
    expression_or_dist { , expression_or_dist } : property_expr ;
  | default [ : ] property_expr ;

assertion_variable_declaration ::=
    var_data_type list_of_variable_decl_assignments ;

property_instance ::=
    ps_or_hierarchical_property_identifier [ ( [ property_list_of_arguments ] ) ]

property_list_of_arguments ::=
    [property_actual_arg] { , [property_actual_arg] } { , . identifier ( [property_actual_arg] ) }
  | . identifier ( [property_actual_arg] ) { , . identifier ( [property_actual_arg] ) }

property_actual_arg ::=
    property_expr
  | sequence_actual_arg

```

Syntax 16-16—Property construct syntax (excerpt from [Annex A](#))

A named property may be declared with formal arguments in the optional *property_port_list*.

Except as described in [16.12.18](#), [16.12.19](#), and [16.12.17](#), the rules for declaring formal arguments and default actual arguments in named properties and for instantiating named properties with actual arguments are the same as those for named sequences as described in [16.8](#), [16.8.1](#), and [16.8.2](#).

Rules particular to the specification and use of typed formal arguments in named properties are discussed in [16.12.18](#).

Rules particular to the specification and use of local variable formal arguments in named properties are discussed in [16.12.19](#).

A formal argument may be referenced in the body *property_spec* of the declaration of the named property. A reference to a formal argument may be written in place of various syntactic entities, including, in addition to those listed in [16.8](#), the following:

- *property_expr*
- *property_spec*

A named property may be instantiated prior to its declaration. A named property may be instantiated anywhere a *property_spec* may be written. A named property may be instantiated in a place where a *property_expr* may be written provided the instance does not produce an illegal **disable iff** clause (see the following). There may be cyclic dependencies among named properties resulting from their instantiations. A cyclic dependency among named properties results if, and only if, there is a cycle in the directed graph whose nodes are the named properties and whose edges are defined by the following rule: there is a directed edge from one named property to a second named property if, and only if, either the first named property instantiates the second named property within its declaration, including an instance within the declaration of a default actual argument, or there is an instance of the first named property that instantiates the second named property within an actual argument. Named properties with such cyclic dependencies are called *recursive* and are discussed in [16.12.17](#).

If \$ is an actual argument to an instance of a named property, then the corresponding formal argument shall be untyped and each of its references either shall be an upper bound in a *cycle_delay_const_range_expression* or shall itself be an actual argument in an instance of a named sequence or property.

The behavior and semantics of an instance of a nonrecursive named property are the same as those of the flattened property that is obtained from the body of the declaration of the named property by the rewriting algorithm defined in [F.4.1](#). The rewriting algorithm substitutes actual arguments for references to the corresponding formal arguments in the body of the declaration of the named property. The rewriting algorithm does not itself account for name resolution and assumes that names have been resolved prior to the substitution of actual arguments. If the flattened property is not legal, then the instance is not legal and there shall be an error.

The result of property evaluation is either true or false. Properties may be built from other properties or sequences using instantiation and the operators described in the following subclauses.

[Table 16-3](#) lists the sequence and property operators from highest to lowest precedence and shows the associativity of the non-unary operators. The precedence for the strong and weak sequence operators is not defined because these operators require parentheses. The operators described in [Table 11-2](#) have higher precedence than the sequence and property operators.

Table 16-3—Sequence and property operator precedence and associativity

Sequence operators	Property operators	Associativity
[*], [=], [->]		—
##		Left
throughout		Right
within		Left
intersect		Left
	not, nexttime, s_nexttime	—
and	and	Left
or	or	Left
	iff	Right
	until, s_until, until_with, s_until_with, implies	Right
	->, =>, #-#, #=#	Right
	always, s_always, eventually, s_eventually, if-else, case, accept_on, reject_on, sync_accept_on, sync_reject_on	—

A **disable iff** clause can be attached to a *property_expr* to yield a *property_spec*.

```
disable iff (expression_or_dist) property_expr
```

The expression of the **disable iff** is called the *disable condition*. The **disable iff** clause allows preemptive resets to be specified. For an evaluation of the *property_spec*, there is an evaluation of the underlying *property_expr*. If the disable condition is true at anytime between the start of the attempt in the Observed region, inclusive, and the end of the evaluation attempt, inclusive, then the overall evaluation of the property results in disabled. A property has disabled evaluation if it was preempted due to a **disable iff** condition. A disabled evaluation of a property does not result in success or failure. Otherwise, the evaluation of the *property_spec* is the same as that of the *property_expr*. The disable condition is tested independently for different evaluation attempts of the *property_spec*. The values of variables used in the disable condition are those in the current simulation cycle, i.e., not sampled. The expression may contain a reference to an end point of a sequence by using the method `triggered` of that sequence. The disable conditions shall not contain any reference to local variables or the sequence method `matched`. If a sampled value function other than `$sampled` is used in the disable condition, the sampling clock shall be explicitly specified in its actual argument list as described in [16.9.3](#). Nesting of **disable iff** clauses, explicitly or through property instantiations, is not allowed.

16.12.1 Property instantiation

An instance of a named property can be used as a *property_expr* or *property_spec*. In general, the instance is legal provided the body *property_spec* of the named property can be substituted in place of the instance, with actual arguments substituted for formal arguments, and result in a legal *property_expr* or *property_spec*. For example, if an instance of a named property is used as a *property_expr* operand for any property-building operator, then the named property must not have a **disable iff** clause.

16.12.2 Sequence property

Sequence properties have three forms: *sequence_expr*, **weak**(*sequence_expr*), and **strong**(*sequence_expr*). The **strong** and **weak** operators are called *sequence operators*. **strong**(*sequence_expr*) evaluates to true if, and only if, there is a nonempty match of the *sequence_expr*. **weak**(*sequence_expr*) evaluates to true if, and only if, there is no finite prefix that witnesses inability to match the *sequence_expr*. The *sequence_expr* of a sequential property shall not admit an empty match (see [16.12.22](#)).

If the **strong** or **weak** operator is omitted, then the evaluation of the *sequence_expr* depends on the assertion statement in which it is used. If the assertion statement is **assert property** or **assume property**, then the *sequence_expr* is evaluated as **weak**(*sequence_expr*). Otherwise, the *sequence_expr* is evaluated as **strong**(*sequence_expr*).

NOTE—The semantics for a *sequence_expr* definition in IEEE Std 1800-2009 and on is not backward compatible with IEEE Std 1800-2005. The current equivalent to a *sequence_expr* as defined in IEEE Std 1800-2005 is **strong**(*sequence_expr*).

Since only one match of a *sequence_expr* is needed for **strong**(*sequence_expr*) to hold, a property of the form **strong**(*sequence_expr*) evaluates to true if, and only if, the property **strong**(**first_match**(*sequence_expr*)) evaluates to true.

Similarly, a property of the form **weak**(*sequence_expr*) evaluates to true if, and only if, the property **weak**(**first_match**(*sequence_expr*)) evaluates to true. This is because a prefix witnesses inability to match *sequence_expr* if, and only if, it witnesses inability to match **first_match**(*sequence_expr*).

The following examples illustrate the sequential property forms:

```
property p3;
    b ##1 c;
endproperty

c1: cover property (@(posedge clk) a #-# p3);
a1: assert property (@(posedge clk) a |-> p3);
```

The sequential property p3 is interpreted as strong in the cover property c1. An evaluation attempt of c1 returns true if, and only if, a is true at the tick of **posedge** clk at which the attempt begins and both of the following conditions are satisfied:

- b is true at the tick of **posedge** clk at which the attempt begins.
- There exists a subsequent tick of **posedge** clk and c is true at the first such tick.

The sequential property p3 is interpreted as weak in the **assert property** a1. An evaluation attempt of a1 returns true if, and only if, either a is false at the tick of **posedge** clk at which the attempt begins or both of the following conditions are satisfied:

- b is true at the tick of **posedge** clk at which the attempt begins.
- If there exists a subsequent tick of **posedge** clk, then c is true at the first such tick.

16.12.3 Negation property

A property is a *negation* if it has the form **not** *property_expr*. For each evaluation attempt of the property, there is an evaluation attempt of *property_expr*. The keyword **not** states that the evaluation of the property returns the opposite of the evaluation of the underlying *property_expr*. Thus, if *property_expr* evaluates to true, then **not** *property_expr* evaluates to false; and if *property_expr* evaluates to false, then **not** *property_expr* evaluates to true.

The **not** operator switches the strength of a property. In particular, one should be careful when negating a sequence. For example, consider the following assertion:

```
a1: assert property (@clk not a ##1 b);
```

Since the sequential property `a ##1 b` is used in an assertion, it is weak. This means that if `clk` stops ticking and `a` holds at the last tick of `clk`, the weak sequential property `a ##1 b` will also hold beginning at that tick, and so the assertion `a1` will fail. In this case it is more reasonable to use:

```
a2: assert property (@clk not strong(a ##1 b));
```

16.12.4 Disjunction property

A property is a *disjunction* if it has the following form:

```
property_expr or property_expr
```

The property evaluates to true if, and only if, at least one of *property_expr1* and *property_expr2* evaluates to true.

16.12.5 Conjunction property

A property is a *conjunction* if it has the following form:

```
property_expr and property_expr
```

The property evaluates to true if, and only if, both *property_expr1* and *property_expr2* evaluate to true.

16.12.6 If-else property

A property is an *if-else* if it has either the following form:

```
if ( expression_or_dist ) property_expr
```

or the following form:

```
if ( expression_or_dist ) property_expr else property_expr
```

A property of the first form evaluates to true if, and only if, either *expression_or_dist* evaluates to false or *property_expr* evaluates to true. A property of the second form evaluates to true if, and only if, either *expression_or_dist* evaluates to true and *property_expr1* evaluates to true or *expression_or_dist* evaluates to false and *property_expr2* evaluates to true.

16.12.7 Implication

The implication construct specifies that the checking of a property is performed conditionally on the match of a sequential antecedent (see [Syntax 16-17](#)).

```
property_expr ::= // from A.2.10
...
| sequence_expr -> property_expr
| sequence_expr => property_expr
```

Syntax 16-17—Implication syntax (excerpt from [Annex A](#))

This construct is used to precondition monitoring of a property expression and is allowed at the property level. The result of the implication is either true or false. The left-hand operand *sequence_expr* is called the *antecedent*, while the right-hand operand *property_expr* is called the *consequent*.

The following points should be noted for $\rightarrow$ implication:

- From a given start point, the antecedent *sequence_expr* can have zero, one, or more than one successful match.
- If there is no match of the antecedent *sequence_expr* from a given start point, then evaluation of the implication from that start point succeeds and returns true.
- For each successful match of the antecedent *sequence_expr*, the consequent *property_expr* is separately evaluated. The end point of the match of the antecedent *sequence_expr* is the start point of the evaluation of the consequent *property_expr*.
- From a given start point, evaluation of the implication succeeds and returns true if, and only if, for every match of the antecedent *sequence_expr* beginning at the start point, the evaluation of the consequent *property_expr* beginning at the end point of the match succeeds and returns true.

Two forms of implication are provided: overlapped using operator $\rightarrow$ and nonoverlapped using operator $\Rightarrow$. For overlapped implication, if there is a match for the antecedent *sequence_expr*, then the end point of the match is the start point of the evaluation of the consequent *property_expr*. For nonoverlapped implication, the evaluation of the consequent is described by two cases, depending on whether the implication is triggered by a nonempty match or by an empty match:

- If triggered by a nonempty match, the start point of the evaluation of the consequent *property_expr* is the clock tick after the end point of the match.
- If triggered by an empty match, the start point of the evaluation of the consequent *property_expr* is its nearest clock tick, starting from the tick when evaluation of the *sequence_expr* begins. For a singly clocked property, this coincides with the current clock tick.

Therefore,

```
sequence_expr  $\Rightarrow$  property_expr
```

is equivalent to the following:

```
sequence_expr ##1 `true  $\rightarrow$  property_expr
```

The use of implication when multiclock sequences and properties are involved is explained in [16.13](#).

The following example illustrates a bus operation for data transfer from a master to a target device. When the bus enters a data transfer phase, multiple data phases can occur to transfer a block of data. During the data transfer phase, a data phase completes on any rising clock edge on which *irdy* is asserted and either *trdy* or *stop* is asserted. In this example, an asserted signal implies a value of low. The requirement for the end of a data phase can be expressed as follows:

```
let ready_exp = (irdy == 0) && ($fell(trdy) || $fell(stop));
property data_end;
  @(posedge mclk)
  $rose(data_phase)  $\rightarrow$  ##[1:5] ready_exp;
endproperty
a1: assert property (data_end);
```

Each time the sequence *\$rose(data_phase)* matches, an evaluation of the consequent property begins. In [Figure 16-14](#), a match for *\$rose(data_phase)* occurs at clock tick 2. This begins the evaluation of the

consequent property. Then, at clock tick 6, the assertion attempt evaluates to true because `$fell(stop)` and `irdy==0` both evaluate to true.

In another example, `data_end_exp` is used to verify that `frame` is deasserted (value high) within two clock ticks after `data_end_exp` occurs. Further, it is also required that `irdy` is deasserted (value high) one clock tick after `frame` is deasserted.

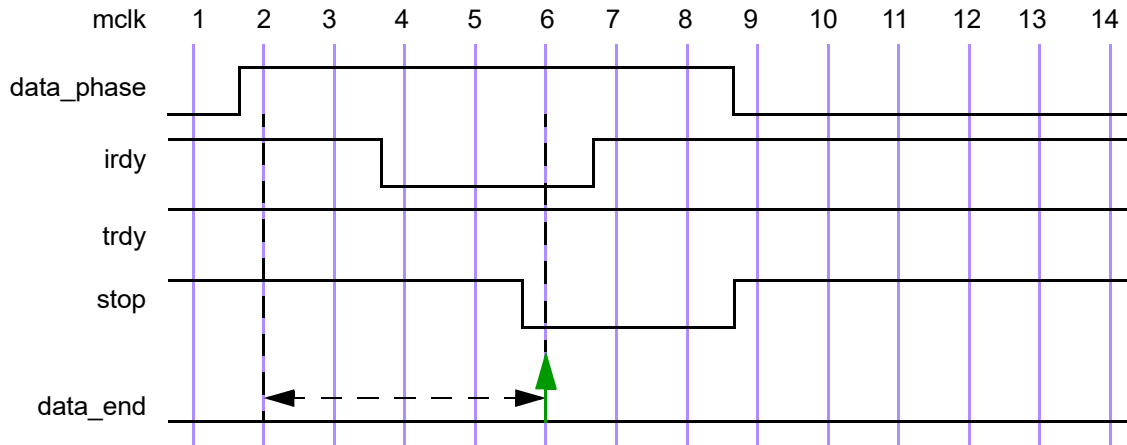


Figure 16-14—Conditional sequence matching

A property written to express this condition is as follows:

```
let data_end_exp = data_phase && ready_exp;
property data_end_rule1;
  @(posedge mclk)
    data_end_exp |-> ##[1:2] $rose(frame) ##1 $rose(irdy);
endproperty
a2: assert property(data_end_rule1);
```

Property `data_end_rule1` first evaluates `data_end_exp` at every clock tick to test if its value is true. If the value is false, then that particular attempt to evaluate `data_end_rule1` is considered true. Otherwise, the following sequence is evaluated:

```
##[1:2] $rose(frame) ##1 $rose(irdy)
```

that specifies looking for the rising edge of `frame` within two clock ticks in the future. After `frame` toggles high, `irdy` must also toggle high after one clock tick. This is illustrated in [Figure 16-15](#) for the evaluation attempt at clock tick 6. `data_end_exp` is acknowledged at clock tick 6. Next, `frame` toggles high at clock tick 7. Because this falls within the timing constraint imposed by `[1:2]`, it satisfies the sequence and continues to evaluate further. At clock tick 8, `irdy` is evaluated. Signal `irdy` transitions to high at clock tick 8, matching the sequence specification completely for the attempt that began at clock tick 6.

Generally, assertions are associated with preconditions so that the checking is performed only under certain specified conditions. As seen from the previous example, the `|->` operator provides this capability to specify preconditions with sequences that must be satisfied before evaluating their consequent properties. The next example modifies the preceding example to see the effect on the results of the assertion by removing the precondition for the consequent. This is shown below and illustrated in [Figure 16-16](#).

```
property data_end_rule2;  
  @(posedge mclk) ##[1:2] $rose(frame) ##1 $rose(irdy);  
endproperty  
a3: assert property(data_end_rule2);
```

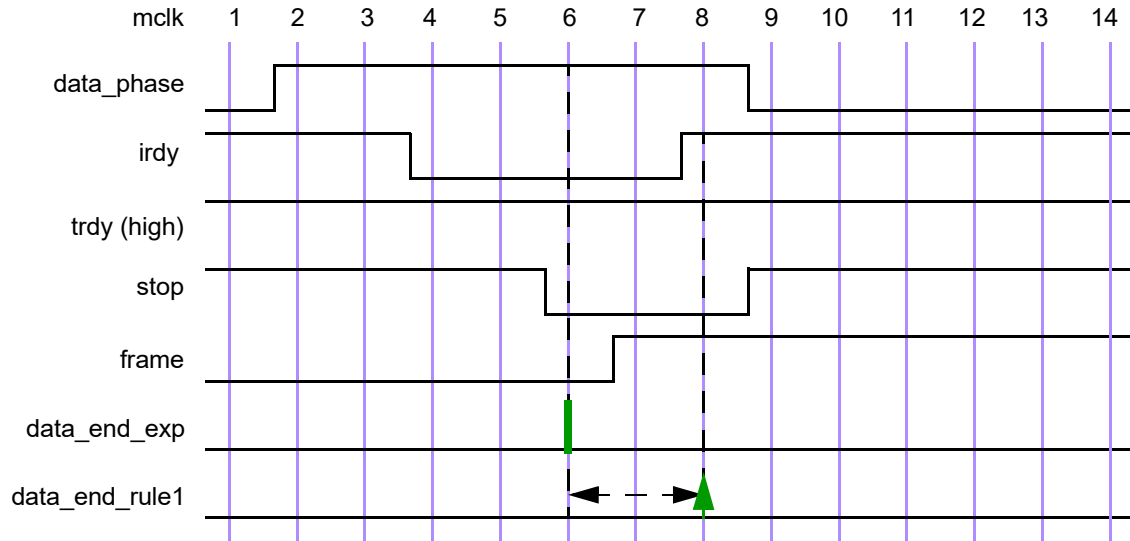


Figure 16-15—Conditional sequences

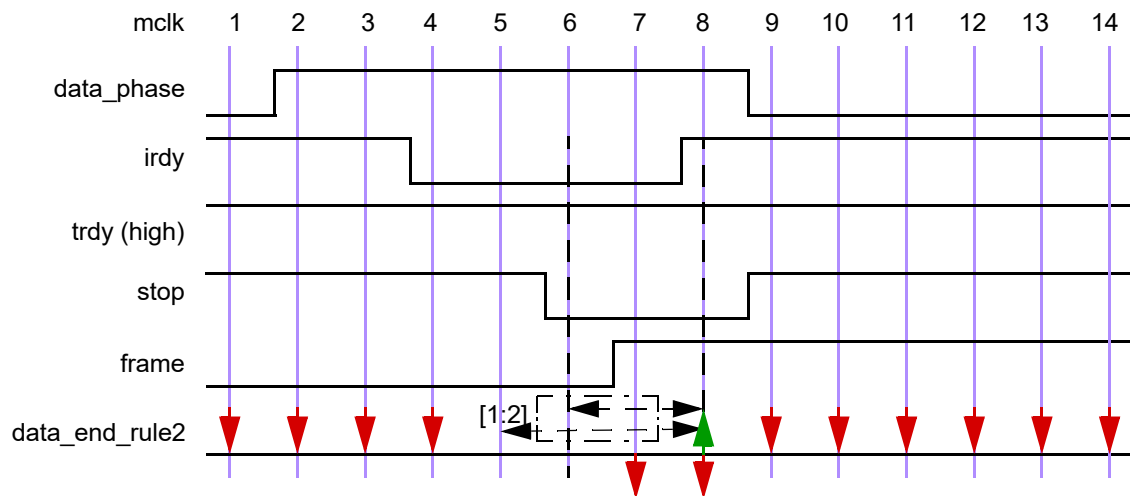


Figure 16-16—Results without the condition

The property is evaluated at every clock tick. For the evaluation at clock tick 1, the rising edge of signal `frame` does not occur at clock tick 2 or 3; therefore, the property fails at clock tick 1. Similarly, there is a failure at clock ticks 2, 3, and 4. For attempts starting at clock ticks 5 and 6, the rising edge of signal `frame` at clock tick 7 allows checking further. At clock tick 8, the sequences complete according to the specification, resulting in a match for attempts starting at clock ticks 5 and 6. All later attempts to match the sequence fail because `$rose(frame)` does not occur again.

[Figure 16-16](#) shows that removing the precondition of checking `data_end_exp` from the assertion causes failures that are not relevant to the verification objective. It is important from the validation standpoint to determine these preconditions and use them to filter out inappropriate or extraneous situations.

An example of implication where the antecedent is a sequence follows:

```
(a ##1 b ##1 c) |-> (d ##1 e)
```

If the sequence `(a ##1 b ##1 c)` matches, then the sequence `(d ##1 e)` must also match. On the other hand, if the sequence `(a ##1 b ##1 c)` does not match, then the result is true.

Another example of implication is as follows:

```
property write_to_addr;
  (write_en & data_valid) ##0
  (write_en && (retire_address[0:4]==addr)) [*2] |->
  ##[3:8] write_en && !data_valid && (write_address[0:4]==addr);
endproperty
```

This property can be coded alternatively as a nested implication:

```
property write_to_addr_nested;
  (write_en & data_valid) |->
  (write_en && (retire_address[0:4]==addr)) [*2] |->
  ##[3:8] write_en && !data_valid && (write_address[0:4]==addr);
endproperty
```

16.12.8 Implies and iff properties

A property is an *implies* if it has the following form:

```
property_expr1 implies property_expr2
```

A property of this form evaluates to true if, and only if, either *property_expr1* evaluates to false or *property_expr2* evaluates to true.

A property is an *iff* if it has the following form:

```
property_expr1 iff property_expr2
```

A property of this form evaluates to true if, and only if, either both *property_expr1* evaluates to false and *property_expr2* evaluates to false or both *property_expr1* evaluates to true and *property_expr2* evaluates to true.

16.12.9 Followed-by property

A property is a *followed-by* if it has one of the following forms that use the followed-by operators shown in [Syntax 16-18](#).

```
property_expr ::=                                     //from A.2.10
...
| sequence_expr ## property_expr
| sequence_expr ==## property_expr
```

Syntax 16-18—Followed-by syntax (excerpt from [Annex A](#))

This clause is used to trigger monitoring of a property expression and is allowed at the property level.

The result of the followed-by is either true or false. The left-hand operand *sequence_expr* is called the *antecedent*, while the right-hand operand *property_expr* is called the *consequent*. For the followed-by property to succeed, the following must hold:

- From a given start point *sequence_expr* shall have at least one successful match.
- *property_expr* shall be successfully evaluated starting from one of the match points of the *sequence_expr*.

From a given start point, evaluation of the followed-by succeeds and returns true if, and only if, there exists a match of the antecedent *sequence_expr* beginning at the start point, and the evaluation of the consequent *property_expr* beginning at the tick specified by the rules in the next paragraph succeeds and returns true.

Two forms of followed-by are provided: overlapped using operator `#-#` and nonoverlapped using operator `##`. For overlapped followed-by, there shall be a match for the antecedent *sequence_expr*, where the end point of this match is the start point of the evaluation of the consequent *property_expr*. For nonoverlapped followed-by, the evaluation of the consequent is described by two cases, depending on whether the antecedent *sequence_expr* attains a nonempty match or an empty match:

- If a nonempty match, the start point of the evaluation of the consequent *property_expr* is the clock tick after the end point of the match.
- If an empty match, the start point of the evaluation of the consequent *property_expr* is its nearest clock tick, starting from the tick when evaluation of the *sequence_expr* begins. For a singly clocked property, this coincides with the current clock tick.

The followed-by operators are the duals of the implication operators. Therefore, *sequence_expr* `#-#` *property_expr* is equivalent to the following:

```
not (sequence_expr |-> not property_expr)
```

and *sequence_expr* `##` *property_expr* is equivalent to the following:

```
not (sequence_expr |=> not property_expr)
```

Examples:

```
property p1;
  ##[0:5] done #-# always !rst;
endproperty

property p2;
  ##[0:5] done ## always !rst;
endproperty
```

Property p1 says that *done* shall be asserted at some clock tick during the first 6 clock ticks, and starting from one of the clock ticks when *done* is asserted, *rst* shall always be low. Property p2 says that *done* shall be asserted at some clock tick during the first 6 clock ticks, and starting the clock tick after one of the clock ticks when *done* is asserted, *rst* shall always be low.

sequence_expr `#-#` **strong**(*sequence_expr1*) is semantically equivalent to **strong**(*sequence_expr* `##0` *sequence_expr1*), and *sequence_expr* `##` **strong**(*sequence_expr1*) is semantically equivalent to **strong**(*sequence_expr* `##1` *sequence_expr1*).

A followed-by operator is especially convenient for specifying a **cover property** directive over a sequence followed by a property.

16.12.10 Nexttime property

A property is a *nexttime* if it has one of the following forms that use the *nexttime* operators:

- Weak *nexttime*
nexttime *property_expr*
The weak *nexttime* property **nexttime** *property_expr* evaluates to true if, and only if, either the *property_expr* evaluates to true beginning at the next clock tick or there is no further clock tick.
- Indexed form of weak *nexttime*
nexttime [*constant_expression*] *property_expr*
The indexed weak *nexttime* property **nexttime** [*constant_expression*] *property_expr* evaluates to true if, and only if, either there are not *constant_expression* clock ticks or *property_expr* evaluates to true beginning at the last of the next *constant_expression* clock ticks.
- Strong *nexttime*
s_nexttime *property_expr*
The strong *nexttime* property **s_nexttime** *property_expr* evaluates to true if, and only if, there exists a next clock tick and *property_expr* evaluates to true beginning at that clock tick.
- Indexed form of strong *nexttime*
s_nexttime [*constant_expression*] *property_expr*
The indexed strong *nexttime* property **s_nexttime** [*constant_expression*] *property_expr* evaluates to true if, and only if, there exist *constant_expression* clock ticks and *property_expr* evaluates to true beginning at the last of the next *constant_expression* clock ticks.

The number of clock ticks given by *constant_expression* shall be a non-negative integer constant expression.

The preceding explanations refer to the case where the *nexttime* property is evaluated in a time step that is a tick of the clock of the *nexttime* property. When the *nexttime* property is evaluated in a time step that is not a tick of the clock of the *nexttime* property, an alignment to the tick of the clock of the *nexttime* property should be applied before the preceding description. Thus, it is more precise to say that **s_nexttime**[*n*] *property_expr* evaluates to true if, and only if, there exist *n*+1 ticks of the clock of the *nexttime* property, including the current time step, and *property_expr* evaluates to true on the *n*+1 clock tick, where counting starts at the current time step. In particular **nexttime**[0] and **s_nexttime**[0] act as alignment operators.

The comments in the following examples describe the conditions for the properties to be evaluated to true:

```
// if the clock ticks once more, then a shall be true at the next clock tick
property p1;
  nexttime a;
endproperty

// the clock shall tick once more and a shall be true at the next clock tick.
property p2;
  s_nexttime a;
endproperty

// as long as the clock ticks, a shall be true at each future clock tick
// starting from the next clock tick
property p3;
  nexttime always a;
```

endproperty

```
// the clock shall tick at least once more and as long as it ticks, a shall
// be true at every clock tick starting from the next one
property p4;
    s_nexttime always a;
endproperty

// if the clock ticks at least once more, it shall tick enough times for a to
// be true at some point in the future starting from the next clock tick
property p5;
    nexttime s_eventually a;
endproperty

// a shall be true sometime in the strict future
property p6;
    s_nexttime s_eventually a;
endproperty

// if there are at least two more clock ticks, a shall be true at the second
// future clock tick
property p7;
    nexttime[2] a;
endproperty

// there shall be at least two more clock ticks, and a shall be true at the
// second future clock tick
property p8;
    s_nexttime[2] a;
endproperty
```

16.12.11 Always property

A property is an *always* if it has one of the following forms that use the always operators:

- Weak always

always property_expr

A property **always** *property_expr* evaluates to true if, and only if, *property_expr* holds at every current or future clock tick.

- Ranged form of weak always

always [cycle_delay_const_range_expression] property_expr

A property **always** [*cycle_delay_const_range_expression*] *property_expr* evaluates to true if, and only if, *property_expr* holds at every current or future clock tick that is within the range of clock ticks specified by *cycle_delay_const_range_expression*. It is not required that all clock ticks within this range exist. The range for a weak always may be unbounded.

- Ranged form of strong always

s_always [constant_range] property_expr

A property **s_always** [*constant_range*] *property_expr* evaluates to true if, and only if, all current or future clock ticks specified by *constant_range* exist and *property_expr* holds at each of these clock ticks. The range for a strong always shall be bounded.

The range of clock ticks given by *constant_range* shall adhere to the following restrictions. The minimum number of clock ticks is defined by a non-negative integer constant expression; and the maximum number of

clock ticks either is defined by a non-negative integer constant expression or is \$, indicating a finite, but unbounded, maximum. If both the minimum and maximum numbers of clock ticks are defined by non-negative integer constant expressions (see [11.2.1](#)), then the minimum number shall be less than or equal to the maximum number.

The preceding explanations refer to the case where the always property is evaluated in a time step that is a tick of the clock of the always property. When the always property is evaluated in a time step that is not a tick of the clock of the always property, an alignment to the tick of the clock of the always property should be applied before the preceding description. Thus, it is more precise to say that **s_always**[n:m] *property_expr* evaluates to true if, and only if, there exist m+1 ticks of the clock of the always property, including the current time step, and *property_expr* evaluates to true beginning in all of the n+1 to m+1 clock ticks, where counting starts at the current time step.

There is also the implicit always that is associated with concurrent assertions (see [16.5](#)). A verification statement that is not placed inside an initial block specifies that an evaluation attempt of its top-level property shall begin at each occurrence of its leading clocking event. In the following two examples, there is a one-to-one correspondence between the evaluation attempts of *p* specified by the implicit always from the verification statement **implicit_always** and the evaluation attempts of *p* specified by the explicit **always** operator in **explicit_always**:

Implicit form:

```
implicit_always: assert property (p);
```

Explicit form:

```
initial explicit_always: assert property (always p);
```

This is not shown as a practical example, but only for illustration of the meaning of **always**.

Examples:

```
initial a1: assume property ( @(posedge clk) reset[*5] ==# always !reset);

property p1;
  a ##1 b | => always c;
endproperty

property p2;
  always [2:5] a;
endproperty

property p3;
  s_always [2:5] a;
endproperty

property p4;
  always [2:$] a;
endproperty

property p5;
  s_always [2:$] a; // Illegal
endproperty
```

The assertion a1 says that **reset** shall be true for the first 5 clock ticks and then remain 0 for the rest of the computation. The assumption is being evaluated once starting at the first clock tick. The property p1

evaluates to true provided that if *a* is true at the first clock tick and *b* is true at the second clock tick, then *c* shall be true at every clock tick that follows the second. The properties *p2* and *p3* evaluate to true provided that *a* is true at each of the second through fifth clock ticks after the starting clock tick of the evaluation attempt. Property *p3* evaluates to true provided that these clock ticks exist, while property *p2* does not require that. The property *p4* evaluates to true if, and only if, *a* is true at every clock tick that is at least two clock ticks after the starting clock tick of the evaluation attempt. These clock ticks are not required to exist. The property *p5* is illegal since specifying an unbounded range is not permitted with the strong form of an always property.

16.12.12 Until property

A property is an *until* if it has one of the following forms that use the until operators:

- Weak non-overlapping form
property_expr1 **until** property_expr2
- Strong non-overlapping form
property_expr1 **s_until** property_expr2
- Weak overlapping form
property_expr1 **until_with** property_expr2
- Strong overlapping form
property_expr1 **s_until_with** property_expr2

An until property of the non-overlapping form evaluates to true if *property_expr1* evaluates to true at every clock tick beginning with the starting clock tick of the evaluation attempt and continuing until at least one tick before a clock tick where *property_expr2* evaluates to true. An until property of one of the overlapping forms evaluates to true if *property_expr1* evaluates to true at every clock tick beginning with the starting clock tick of the evaluation attempt and continuing until and including a clock tick at which *property_expr2* evaluates to true. An until property of one of the strong forms requires a current or future clock tick exist at which *property_expr2* evaluates to true, while an until property of one of the weak forms does not make this requirement. An until property of one of the weak forms evaluates to true if *property_expr1* evaluates to true at each clock tick, even if *property_expr2* never holds.

Examples:

```
property p1;
    a until b;
endproperty

property p2;
    a s_until b;
endproperty

property p3;
    a until_with b;
endproperty

property p4;
    a s_until_with b;
endproperty
```

Property *p1* evaluates to true if, and only if, *a* is true at every clock tick beginning with the starting clock tick of the evaluation attempt and continuing until, but not necessarily including, a clock tick at which *b* is

true. If there is no current or future clock tick at which *b* is true, then *a* shall be true at every current or future clock tick. If *b* is true at the starting clock tick of the evaluation attempt, then *a* need not be true at that clock tick. The property *p2* evaluates to true provided that there exists a current or future clock tick at which *b* is true and that *a* is true at every clock tick beginning with the starting clock tick of the evaluation attempt and continuing until, but not necessarily including, the clock tick at which *b* is true. If *b* is true at the starting clock tick of the evaluation attempt, then *a* need not be true at that clock tick. The property *p3* evaluates to true provided that *a* is true at every clock tick beginning with the starting clock tick of the evaluation attempt and continuing until and including a clock tick at which *b* is true. If there is no current or future clock tick at which *b* is true, then *a* shall be true at every current or future clock tick. The property *p4* evaluates to true provided there exists a current or future clock tick at which *b* is true and that *a* is true at every clock tick beginning with the starting clock tick of the evaluation attempt and continuing until and including the clock tick at which *b* is true. The property *p4* is equivalent to **strong**(*a* [*1:\$] ##0 *b*) (here *a* and *b* are Boolean expressions).

16.12.13 Eventually property

A property is an *eventually* if it has one of the following forms that use the eventually operators:

- Strong eventually

s_eventually property_expr

A property *s_eventually* property_expr evaluates to true if, and only if, there exists a current or future clock tick at which property_expr evaluates to true.

- Ranged form of weak eventually

eventually [constant_range] property_expr

A property eventually [constant_range] property_expr evaluates to true if, and only if, either there exists a current or future clock tick within the range specified by constant_range at which property_expr evaluates to true or not all the current or future clock ticks within the range specified by constant_range exist. The range for a weak eventually shall be bounded.

- Ranged form of strong eventually

s_eventually [cycle_delay_const_range_expression] property_expr

A property *s_eventually* [cycle_delay_const_range_expression] property_expr evaluates to true if, and only if, there exists a current or future clock tick within the range specified by cycle_delay_const_range_expression at which property_expr evaluates to true. The range for a strong eventually may be unbounded.

In the following examples, *a* and *b* are Boolean expressions:

```
property p1;
  s_eventually a;
endproperty

property p2;
  s_eventually always a;
endproperty

property p3;
  always s_eventually a;
endproperty

property p4;
  eventually [2:5] a;
endproperty
```

```
property p5;
    s_eventually [2:5] a;
endproperty

property p6;
    eventually [2:$] a; // Illegal
endproperty

property p7;
    s_eventually [2:$] a;
endproperty
```

The property `p1` evaluates to true if, and only if, there exists a current or future clock tick at which `a` is true. It is equivalent to **strong**(`##[*0:$] a`). The property `p2` evaluates to true if, and only if, there exists a current or future clock tick such that `a` is true both at that clock tick and also at every subsequent clock tick. On a computation with infinitely many clock ticks, the property `p3` evaluates to true if, and only if, `a` is true at infinitely many of those clock ticks. On a computation with finitely many clock ticks, the property `p3` evaluates to true provided that if there is at least one clock tick, then `a` holds at the last clock tick. The property `p4` evaluates to true provided that if the second through fifth clock ticks from the starting clock tick of the evaluation attempt all exist, then `a` is true at one of these clock ticks. `p4` is equivalent to **weak**(`##[2:5] a`). The property `p5` evaluates to true if, and only if, there exists a clock tick at which `a` is true and that it is between the second and fifth clock ticks, inclusive, from the starting clock tick of the evaluation attempt. `p5` is equivalent to **strong**(`##[2:5] a`). The property `p7` evaluates to true if, and only if, there exists a clock tick at which `a` is true and that it is no earlier than the second clock tick after the starting clock tick of the evaluation attempt.

The preceding explanations refer to the case where the eventually property is evaluated in a time step that is a tick of the clock of the eventually property. When the eventually property is evaluated in a time step that is not a tick of the clock of the eventually property, an alignment to the tick of the clock of the eventually property should be applied before the preceding description. Thus, it is more precise to say that **s_eventually**`[n:m] property_expr` evaluates to true if, and only if, there exist at least `n+1` ticks of the clock of the eventually property, including the current time step, and `property_expr` evaluates to true beginning in one of the `n+1` to `m+1` clock ticks, where counting starts at the current time step.

16.12.14 Abort properties

A property is an *abort* if it has one of the following forms:

```
accept_on ( expression_or_dist ) property_expr
reject_on ( expression_or_dist ) property_expr
sync_accept_on ( expression_or_dist ) property_expr
sync_reject_on ( expression_or_dist ) property_expr
```

where the `expression_or_dist` is called the *abort condition*. The properties **accept_on** and **reject_on** are called *asynchronous abort properties*, and the properties **sync_accept_on** and **sync_reject_on** are called *synchronous abort properties*.

For an evaluation of **accept_on**(`expression_or_dist`) `property_expr` and of **sync_accept_on**(`expression_or_dist`) `property_expr`, there is an evaluation of the underlying `property_expr`. If during the evaluation, the abort condition becomes true, then the overall evaluation of the property results in true. Otherwise, the overall evaluation of the property is equal to the evaluation of the `property_expr`.

For an evaluation of **reject_on** (*expression_or_dist*) *property_expr* and of **sync_reject_on** (*expression_or_dist*) *property_expr*, there is an evaluation of the underlying *property_expr*. If during the evaluation, the abort condition becomes true, then the overall evaluation of the property results in false. Otherwise, the overall evaluation of the property is equal to the evaluation of the *property_expr*.

The operators **accept_on** and **reject_on** are evaluated at the granularity of the simulation time step like **disable iff**, but their abort condition is evaluated using sampled value as a regular Boolean expression in assertions. The operators **accept_on** and **reject_on** represent asynchronous resets.

The operators **sync_accept_on** and **sync_reject_on** are evaluated at the simulation time step when the clocking event happens, unlike **disable iff**, **accept_on**, and **reject_on**. Their abort condition is evaluated using sampled value as for **accept_on** and **reject_on**. The operators **sync_accept_on** and **sync_reject_on** represent synchronous resets.

The semantics of **accept_on** is similar to **disable iff**, except for the following differences:

- **accept_on** operates at the property level rather than the concurrent assertion level.
- **accept_on** uses sampled values.
- While a disable condition of a **disable iff** in a *property_spec* may cause an evaluation of the *property_spec* to be disabled, an abort condition of **accept_on** in a *property_expr* may cause the evaluation of the *property_expr* to be true.

The semantics of **reject_on**(*expression_or_dist*) *property_expr* is the same as **not**(**accept_on**(*expression_or_dist*) **not**(*property_expr*)).

The semantics of **sync_accept_on** is similar to **accept_on**, except that it evaluates only at the time steps when the clocking event happens.

The semantics of **sync_reject_on**(*expression_or_dist*) *property_expr* is the same as **not**(**sync_accept_on**(*expression_or_dist*) **not**(*property_expr*)).

Any nesting of abort operators **accept_on**, **reject_on**, **sync_accept_on**, and **sync_reject_on** is allowed.

For example, whenever *go* is high, followed by two occurrences of *get* being high, then *stop* cannot be high until after *put* is asserted twice (not necessarily consecutive).

```
assert property (@(clk) go ##1 get[*2] |-> reject_on(stop) put[->2]);
```

In this example the *stop* is an asynchronous abort, its value is checked even between ticks of *clk*. The following is the synchronous version of the same example:

```
assert property (@(clk) go ##1 get[*2] |-> sync_reject_on(stop) put[->2]);
```

Here *stop* is checked only at the *clk* ticks. The latter assertion can also be written as follows:

```
assert property (@(clk) go ##1 get[*2] |-> !stop throughout put[->2]);
```

When the abort condition occurs at the same time step where the evaluation of the *property_expr* ends, the abort condition takes precedence. For example:

```
property p; (accept_on(a) p1) and (reject_on(b) p2); endproperty
```

If *a* becomes true during the evaluation of *p*₁, the first term is ignored in deciding the truth of *p*. On the other hand, if *b* becomes true during the evaluation of *p*₂ then *p* evaluates to false.

```
property p; (accept_on(a) p1) or (reject_on(b) p2); endproperty
```

If *a* becomes true during the evaluation of *p*₁ then *p* evaluates to true. On the other hand, if *b* becomes true during the evaluation of *p*₂, then the second term is ignored in deciding the truth of *p*.

```
property p; not (accept_on(a) p1); endproperty
```

not inverts the effect of the abort operator. Therefore, if *a* becomes true while evaluating *p*₁, property *p* evaluates to false.

Nested **accept_on**, **reject_on**, **sync_accept_on**, and **sync_reject_on** operators are evaluated in the lexical order (left to right). Therefore, if two nested operator conditions become true in the same time step during the evaluation of the argument property, then the outermost operator takes precedence. For example:

```
property p; accept_on(a) reject_on(b) p1; endproperty
```

If *a* becomes true in the same time step as *b* and during the evaluation of *p*₁, then *p* succeeds in that time step. If *b* becomes true before *a* and during the evaluation of *p*₁, then *p* fails.

The abort conditions may contain sampled value functions (see [16.9.3](#)). When sampled value functions other than `$sampled` are used in the abort condition, the clock argument shall be explicitly specified. Abort conditions shall not contain any reference to local variables and the sequence methods `triggered` and `matched`.

16.12.15 Weak and strong operators

The property operators **s_nexttime**, **s_always**, **s_eventually**, **s_until**, **s_until_with**, and sequence operator **strong** are strong: they require that some terminating condition happen in the future, and this includes the requirement that the property clock ticks enough time to enable the condition to happen.

The property operators **nexttime**, **always**, **until**, **eventually**, **until_with**, and sequence operator **weak** are weak: they do not impose any requirement on the terminating condition, and do not require the clock to tick.

The concept of weak and strong operators is closely related to an important notion of safety properties. Safety properties have the characteristic that all their failures happen at a finite time. For example, the property **always** *a* is a safety property since it is violated only if after finitely many clock ticks there is a clock tick at which *a* is false, even if there are infinitely many clock ticks in the computation. To the contrary, a failure of the property **s_eventually** *a* on a computation with infinitely many clock ticks cannot be identified at a finite time; if it is violated, the value of *a* must be false at each of the infinitely many clock ticks.

16.12.16 Case

The *case* property statement is a multiway decision that tests whether a Boolean expression matches one of a number of other Boolean expressions and branches accordingly (see [Syntax 16-19](#)).

```
property_expr ::= // from A.2.10  
...  
| case ( expression_or_dist ) property_case_item { property_case_item } endcase
```

```
...
property_case_item ::=
    expression_or_dist { , expression_or_dist } : property_expr ;
| default [ : ] property_expr ;
```

Syntax 16-19—Property statement case syntax (excerpt from [Annex A](#))

The *default* statement shall be optional. Use of multiple default statements in one property case statement shall be illegal.

A simple example of the use of the case property statement is the decoding of variable delay to produce a delay between the check of two signals as follows:

```
property p_delay(logic [1:0] delay);
case (delay)
    2'd0    : a && b;
    2'd1    : a ##2 b;
    2'd2    : a ##4 b;
    2'd3    : a ##8 b;
    default: 0;           // cause a failure if delay has x or z values
endcase
endproperty
```

During the linear search, if one of the case item expressions matches the case expression given in parentheses, then the property statement associated with that case item shall be evaluated, and the linear search shall terminate. If there is a default case item, it is ignored during this linear search. If all comparisons fail and the default item is given, then the default item property statement shall be executed. If the default property statement is not given and all of the comparisons fail, then none of the case item property statements shall be evaluated and the evaluation of the case property statement from that start point succeeds and returns true (vacuously).

The rules for comparing the case expression to the case item expressions are described in [12.5](#).

16.12.17 Recursive properties

SystemVerilog allows recursive properties. A named property is recursive if its declaration involves an instantiation of itself. Recursion provides a flexible framework for coding properties to serve as ongoing assumptions, obligations, or coverage monitors.

For example:

```
property prop_always(p);
    p and (1'b1 | => prop_always(p));
endproperty
```

is a recursive property that says that the formal argument property *p* must hold at every cycle. This example is useful if the ongoing requirement that property *p* hold applies after a complicated triggering condition encoded in sequence *s*:

```
property p1(s,p);
    s | => prop_always(p);
endproperty
```

As another example, the recursive property

```
property prop_weak_until(p,q);
    q or (p and (1'b1 |>= prop_weak_until(p,q)));
endproperty
```

says that formal argument property *p* must hold at every cycle up to, but not including, the first cycle at which formal argument property *q* holds. Formal argument property *q* is not required ever to hold, however. This example is useful if *p* must hold at every cycle after a complicated triggering condition encoded in sequence *s*, but the requirement on *p* is lifted by *q*:

```
property p2(s,p,q);
    s |>= prop_weak_until(p,q);
endproperty
```

More generally, several properties can be mutually recursive. For example:

```
property check_phase1;
    s1 |>= (phase1_prop and (1'b1 |>= check_phase2));
endproperty
property check_phase2;
    s2 |>= (phase2_prop and (1'b1 |>= check_phase1));
endproperty
```

There are four restrictions on recursive property declarations, as follows:

- *Restriction 1:* The negation operator **not** and strong operators **s_nexttime**, **s_eventually**, **s_always**, **s_until**, and **s_until_with** cannot be applied to any property expression that instantiates a recursive property. In particular, the negation of a recursive property cannot be asserted or used in defining another property.

The following are examples of illegal property declarations that violate Restriction 1:

```
property illegal_recursion_1(p);
    not prop_always(not p);
endproperty

property illegal_recursion_2(p);
    p and (1'b1 |>= not illegal_recursion_2(p));
endproperty
```

Furthermore, **not** cannot be applied to any property expression that instantiates a property that depends on a recursive property. The precise definition of dependency is given in [F.7](#).

- *Restriction 2:* The operator **disable iff** cannot be used in the declaration of a recursive property. This restriction is consistent with the restriction that **disable iff** cannot be nested.

The following is an example of an illegal property declaration that violates Restriction 2:

```
property illegal_recursion_3(p);
    disable iff (b)
    p and (1'b1 |>= illegal_recursion_3(p));
endproperty
```

The intent of `illegal_recursion_3` can be written legally as follows:

```
property legal_3(p);
    disable iff (b) prop_always(p);
endproperty
```

because `legal_3` is not a recursive property.

- *Restriction 3:* If `p` is a recursive property, then, in the declaration of `p`, every instance of `p` must occur after a positive advance in time. In the case of mutually recursive properties, all recursive instances must occur after positive advances in time.

The following is an example of an illegal property declaration that violates Restriction 3:

```
property illegal_recursion_4(p);
    p and (1'b1 |-> illegal_recursion_4(p));
endproperty
```

If this form were legal, the recursion would be stuck in time, checking `p` over and over again at the same cycle.

- *Restriction 4:* For every recursive instance of property `q` in the declaration of property `p`, each actual argument expression `e` of the instance satisfies at least one of the following conditions:
 - `e` is itself a formal argument of `p`.
 - No formal argument of `p` appears in `e`.
 - `e` is bound to a local variable formal argument of `q`.

For example:

```
property fibonacci1 (local input int a, b, n, int fib_sig);
    (n > 0)
    |->
    (
        (fib_sig == a)
        and
        (1'b1 |=> fibonacci1(b, a + b, n - 1, fib_sig))
    );
endproperty
```

is a legal declaration, but

```
property fibonacci2 (int a, b, n, fib_sig);
    (n > 0)
    |->
    (
        (fib_sig == a)
        and
        (1'b1 |=> fibonacci2(b, a + b, n - 1, fib_sig))
    );
endproperty
```

is not legal because, in the recursive instance `fibonacci2(b, a+b, n-1, fib_sig)`, the actual argument expressions `a+b`, `n-1` are not themselves formal arguments of `fibonacci2`, are not bound to local variable formal arguments, and yet formal arguments of `fibonacci2` appear in these expressions.

The operators **accept_on**, **reject_on**, **sync_accept_on**, and **sync_reject_on** may be used inside a recursive property. For example, the following uses of **accept_on** and **reject_on** in a property are legal:

```
property p3(p, bit b, abort);
    (p and (1'b1 |=> p4(p, b, abort)));
endproperty

property p4(p, bit b, abort);
```

```
accept_on(b) reject_on(abort) p3(p, b, abort);  
endproperty
```

Recursive properties can represent complicated requirements, such as those associated with varying numbers of data beats, out-of-order completions, retries, etc. Following is an example of using a recursive property to check complicated conditions of this kind.

Suppose that write data must be checked according to the following conditions:

- Acknowledgment of a write request is indicated by the signal `write_request` together with `write_request_ack`. When a write request is acknowledged, it gets a 4-bit tag, indicated by signal `write_request_ack_tag`. The tag is used to distinguish data beats for multiple write transactions in flight at the same time.
- It is understood that distinct write transactions in flight at the same time must be given distinct tags. For simplicity, this condition is not a part of what is checked in this example.
- Each write transaction can have between 1 data beat and 16 data beats, and each data beat is 8 bits. There is a model of the expected write data that is available at acknowledgment of a write request. The model is a 128-bit vector. The most significant group of 8 bits represents the expected data for the first beat, the next group of 8 bits represents the expected data for the second beat (if there is a second beat), and so forth.
- Data transfer for a write transaction occurs after acknowledgment of the write request and, barring retry, ends with the last data beat. The data beats for a single write transaction occur in order.
- A data beat is indicated by the `data_valid` signal together with the signal `data_valid_tag` to determine the relevant write transaction. The signal data are valid with `data_valid` and carry the data for that beat. The data for each beat must be correct according to the model of the expected write data.
- The last data beat is indicated by signal `last_data_valid` together with `data_valid` and `data_valid_tag`. For simplicity, this example does not represent the number of data beats and does not check that `last_data_valid` is signaled at the correct beat.
- At any time after acknowledgment of the write request, but not later than the cycle after the last data beat, a write transaction can be forced to retry. Retry is indicated by the signal `retry` together with signal `retry_tag` to identify the relevant write transaction. If a write transaction is forced to retry, then its current data transfer is aborted, and the entire data transfer must be repeated. The transaction does not re-request, and its tag does not change.
- There is no limit on the number of times a write transaction can be forced to retry.
- A write transaction completes the cycle after the last data beat provided it is not forced to retry in that cycle.

The following is code to check these conditions:

```
property check_write;  
  
  logic [0:127] expected_data; // local variable to sample model data  
  logic [3:0] tag;           // local variable to sample tag  
  
  disable iff (reset)  
  (  
    write_request && write_request_ack,  
    expected_data = model_data,  
    tag = write_request_ack_tag  
  )  
  |=>  
  check_write_data_beat(expected_data, tag, 4'h0);
```

```

endproperty

property check_write_data_beat
(
    local input logic [0:127] expected_data,
    local input logic [3:0] tag, i
);
(
    (data_valid && (data_valid_tag == tag))
    ||
    (retry && (retry_tag == tag))
) [->1]
|->
(
    (
        (data_valid && (data_valid_tag == tag))
        |->
        (data == expected_data[i*8+:8])
    )
    and
    (
        if (retry && (retry_tag == tag))
        (
            1'b1 |>= check_write_data_beat(expected_data, tag, 4'h0)
        )
        else if (!last_data_valid)
        (
            1'b1 |>= check_write_data_beat(expected_data, tag, i+4'h1)
        )
        else
        (
            ##1 (retry && (retry_tag == tag))
            |>=
            check_write_data_beat(expected_data, tag, 4'h0)
        )
    )
);

endproperty

```

16.12.18 Typed formal arguments in property declarations

The rules in [16.8.1](#) for typed formal arguments and their corresponding actual arguments apply to named properties, except as described next.

If a formal argument of a named property is typed, then the type shall be **property**, **sequence**, **event**, or one of the types allowed in [16.6](#). If the formal argument is of type **property**, then the corresponding actual argument shall be a *property_expr*, and each reference to the formal argument shall be in a place where a *property_expr* may be written.

For example, a Boolean expression or a *sequence_expr* may be passed as actual argument to a formal argument of type **property** because each is a *property_expr*. A formal argument of type **property** may not be referenced as the antecedent of |-> or |>= (see [16.12.7](#)), regardless of the corresponding actual argument, because a *property_expr* may not be written in that position.

16.12.19 Local variable formal arguments in property declarations

The rules in [16.8.2](#) for local variable formal arguments and their corresponding actual arguments apply to named properties, except as described next.

A local variable formal argument of a named property shall have direction **input**, either specified explicitly or inferred. It shall be illegal to declare a local variable formal argument of a named property with direction **inout** or **output**.

16.12.20 Property examples

The following examples illustrate the property forms:

```
property rule1;
  @(posedge clk) a |-> b ##1 c ##1 d;
endproperty
property rule2;
  @(clk) disable iff (e) a |-> not(b ##1 c ##1 d);
endproperty
```

Property rule2 negates the sequence (b ##1 c ##1 d) in the consequent of the implication. clk specifies the clock for the property.

```
property rule3;
  @(posedge clk) a[*2] |-> ((##[1:3] c) or (d |> e));
endproperty
```

Property rule3 says that if a holds and a also held last cycle, then either c must hold at some point one to three cycles after the current cycle or, if d holds in the current cycle, then e must hold one cycle later.

```
property rule4;
  @(posedge clk) a[*2] |-> ((##[1:3] c) and (d |> e));
endproperty
```

Property rule4 says that if a holds and a also held last cycle, then c must hold at some point one to three cycles after the current cycle and, if d holds in the current cycle, then e must hold one cycle later.

```
property rule5;
  @(posedge clk)
  a ##1 (b || c) [->1] |->
    if (b)
      (##1 d |-> e)
    else // c
      f ;
endproperty
```

Property rule5 has a followed by the next occurrence of either b or c as its antecedent. The consequent uses **if-else** to split cases on which of b or c is matched first.

```
property rule6(x,y);
  ##1 x |-> y;
endproperty
property rule5a;
  @(posedge clk)
  a ##1 (b || c) [->1] |->
    if (b)
```

```

        rule6(d,e)
    else // c
        f ;
endproperty

```

Property `rule5a` is equivalent to `rule5`, but it uses an instance of `rule6` as a property expression.

A property can optionally specify an event control for the clock. The clock derivation and resolution rules are described in [16.16](#).

A named property can be instantiated by referencing its name. A hierarchical name can be used, consistent with the SystemVerilog naming conventions. Like sequence declarations, variables used within a property that are not formal arguments to the property are resolved hierarchically from the scope in which the property is declared.

Properties that use more than one clock are described in [16.13](#).

16.12.21 Finite-length versus infinite-length behavior

The formal semantics in [F.5](#) defines whether a given property holds on a given behavior. How the outcome of this evaluation relates to the design depends on the behavior that was analyzed. In dynamic verification, only behaviors that are finite in length are considered. In such a case, SystemVerilog defines the following four levels of satisfaction of a property:

- Holds strongly
 - No bad states have been seen.
 - All future obligations have been met.
 - The property will hold on any extension of the path.
- Holds (but does not hold strongly)
 - No bad states have been seen.
 - All future obligations have been met.
 - The property may or may not hold on a given extension of the path.
- Pending
 - No bad states have been seen.
 - Future obligations have not been met.
 - The property may or may not hold on a given extension of the path.
- Fails
 - A bad state has been seen.
 - Future obligations may or may not have been met.
 - The property will not hold on any extension of the path.

16.12.22 Nondegeneracy

It is possible to define sequences that can never be matched. For example:

```
(1'b1) intersect (1'b1 ##1 1'b1)
```

It is also possible to define sequences that admit only empty matches. For example:

```
1'b1[*0]
```

A zero consecutive repetition means that there is no sample taken at any clock tick. Therefore, such a sequence can only match on an empty trace (as formally defined in [F.4.3](#)). A sequence may admit both empty and nonempty matches, for example, `a[*0:2]`. This sequence admits an empty match and up to two nonempty matches: `a` and `a[*2]`.

A sequence that admits no match or that admits only empty matches is called *degenerate*. A sequence that admits at least one nonempty match is called *nondegenerate*. A more precise definition of nondegeneracy is given in [F.5.2](#) and [F.5.5](#).

The following restrictions apply:

- a) Any sequence that is used as a property shall be nondegenerate and shall not admit any empty match.
- b) Any sequence that is used as the antecedent of an overlapping implication (`|->`) shall be nondegenerate.
- c) Any sequence that is used as the antecedent of a nonoverlapping implication (`|=>`) shall admit at least one match. Such a sequence can admit only empty matches.

The reason for these restrictions is because the use of degenerate sequences in forbidden ways results in counterintuitive property semantics, especially when the property is combined with a **disable iff** clause.

16.13 Multiclock support

Multiclock sequences and properties can be specified as described in the following subclauses.

16.13.1 Multiclocked sequences

Multiclocked sequences are built by concatenating singly clocked subsequences using the single-delay concatenation operator `##1` or the zero-delay concatenation operator `##0`. The single delay indicated by `##1` is understood to be from the end point of the first sequence, which occurs at a tick of the first clock, to the nearest strictly subsequent tick of the second clock, where the second sequence begins. The zero delay indicated by `##0` is understood to be from the end point of the first sequence, which occurs at a tick of the first clock, to the nearest possibly overlapping tick of the second clock, where the second sequence begins.

Example 1:

```
@(posedge clk0) sig0 ##1 @(posedge clk1) sig1
```

A match of this sequence starts with a match of `sig0` at **posedge** `clk0`. Then `##1` moves the time to the nearest strictly subsequent **posedge** `clk1`, and the match of the sequence ends at that point with a match of `sig1`. If `clk0` and `clk1` are not identical, then the clocking event for the sequence changes after `##1`. If `clk0` and `clk1` are identical, then the clocking event does not change after `##1`, and the preceding sequence is equivalent to the singly clocked sequence

```
@(posedge clk0) sig0 ##1 sig1
```

Example 2:

```
@(posedge clk0) sig0 ##0 @(posedge clk1) sig1
```

A match of this sequence starts with a match of `sig0` at **posedge** `clk0`. Then `##0` moves the time to the nearest possibly overlapping **posedge** `clk1`, and the match of the sequence ends at that point with a match of `sig1`: if **posedge** `clk0` and **posedge** `clk1` happen simultaneously then the time does not move at `##0`, otherwise, it behaves as `##1`. If `clk0` and `clk1` are not identical, then the clocking event for the sequence

changes after `##0`. If `clk0` and `clk1` are identical, then the clocking event does not change after `##0`, and the preceding sequence is equivalent to the following singly clocked sequence:

```
@(posedge clk0) sig0 ##0 sig1
```

which is equivalent to the following:

```
@(posedge clk0) sig0 && sig1
```

When concatenating differently clocked sequences, the maximal singly clocked subsequences are required to admit only nonempty matches. The term *maximal singly clocked subsequence* refers to the largest singly clocked sequence appearing in a multiclock sequence resulting from the application of the rewriting algorithm in [F.4.1](#). Such a sequence cannot be enlarged by absorbing any surrounding operators and their arguments without changing the singly clocked sequence into a multiclock sequence or to a property.

Thus, if `s1`, `s2` are sequence expressions with no clocking events, then the multiclocked sequence

```
@(posedge clk1) s1 ##1 @(posedge clk2) s2
```

is legal only if neither `s1` nor `s2` can match the empty word. The clocking event `@(posedge clk1)` applies throughout the match of `s1`, while the clocking event `@(posedge clk2)` applies throughout the match of `s2`. Because the match of `s1` is nonempty, there is an end point of this match at `posedge clk1`. The `##1` synchronizes between this end point and the first occurrence of `posedge clk2` strictly after it. That occurrence of `posedge clk2` is the start point of the match of `s2`.

A multiclocked sequence has well-defined starting and ending clocking events and well-defined clock changes because of the restriction that maximal singly clocked subsequences not match the empty word. If `clk1` and `clk2` are not identical, then the sequence

```
@(posedge clk0) sig0 ##1 @(posedge clk1) sig1[*0:1]
```

is illegal because of the possibility of an empty match of `sig1[*0:1]`, which would make ambiguous whether the ending clocking event is `@(posedge clk0)` or `@(posedge clk1)`.

Differently clocked or multiclocked sequence operands cannot be combined with any sequence operators other than `##1` and `##0`. For example, if `clk1` and `clk2` are not identical, then the following are illegal:

```
@(posedge clk1) s1 ##2 @(posedge clk2) s2
```

```
@(posedge clk1) s1 intersect @(posedge clk2) s2
```

16.13.2 Multiclocked properties

A clock may be explicitly specified with any property. The property is multiclocked if some of its subproperties have a clock different from the property clock, or some of its subproperties are multiclocked sequences.

As in the case of singly clocked properties, the result of evaluating a multiclocked property is either true or false. Multiclocked sequences are themselves multiclocked properties. For example:

```
@(posedge clk0) sig0 ##1 @(posedge clk1) sig1
```

is a multiclocked property. If a multiclocked sequence is evaluated as a property starting at some point, the evaluation returns true if, and only if, there is a match of the multiclocked sequence beginning at that point.

The following example shows how to form a multiclocked property using Boolean property operators:

```
@(posedge clk0) sig0) and @(posedge clk1) sig1)
```

This is a multiclocked property, but it is not a multiclocked sequence. This property evaluates to true at a point if, and only if, the two sequences

```
@(posedge clk0) sig0
```

and

```
@(posedge clk1) sig1
```

both have matches beginning at the point.

The meaning of multiclocked nonoverlapping implication is similar to that of singly clocked nonoverlapping implication. For example, if *s0* and *s1* are sequences with no clocking event, then in

```
@(posedge clk0) s0 | => @(posedge clk1) s1
```

|=> synchronizes between **posedge** *clk0* and **posedge** *clk1*. Starting at the point at which the implication is being evaluated, for each match of *s0* clocked by *clk0*, time is advanced from the end point of the match to the nearest strictly future occurrence of **posedge** *clk1*, and from that point there must exist a match of *s1* clocked by *clk1*.

The following example shows a combination of differently clocked properties using both implication and Boolean property operators:

```
@(posedge clk0) s0 | => ( @(posedge clk1) s1) and ( @(posedge clk2) s2)
```

The multiclocked overlapping implication *|->* has the following meaning: at the end of the antecedent the nearest tick of the consequent clock is awaited. If the consequent clock happens at the end of the antecedent, the consequent is started checking immediately. Otherwise, the meaning of the multiclocked overlapping implication is the same as the meaning of the multiclock nonoverlapping implication.

For example, if *s0* and *s1* are sequences with no clocking events, then

```
@(posedge clk0) s0 | -> @(posedge clk1) s1
```

means the following: at each match of *s0* the nearest **posedge** *clk1* is awaited. If it happens immediately then *s1* is checked without delay, otherwise its check starts at the next **posedge** *clk1* as in case with |=>. In both cases the evaluation of *s1* is controlled by **posedge** *clk1*.

The semantics of multiclocked **if/if-else** operators is similar to the semantics of the overlapping implication. For example, if *s1* and *s2* are sequences with no clocking events, then

```
@(posedge clk0) if (b) @(posedge clk1) s1 else @(posedge clk2) s2
```

has the following meaning: the condition *b* is checked at **posedge** *clk0*. If *b* is true then *s1* is checked at the nearest, possibly overlapping **posedge** *clk1*, else *s2* is checked at the nearest non-strictly subsequent **posedge** *clk2*.

16.13.3 Clock flow

Throughout this subclause, c and d denote clocking event expressions and v , w , x , y , and z denote sequences with no clocking events.

Clock flow allows the scope of a clocking event to extend in a natural way through various parts of multiclocked sequences and properties and reduces the number of places at which the same clocking event must be specified.

Intuitively, clock flow provides that in a multiclocked sequence or property, the scope of a clocking event flows left to right across linear operators (e.g., repetition, concatenation, negation, implication, followed-by, and the **nexttime**, **always**, **eventually** operators) and distributes to the operands of branching operators (e.g., conjunction, disjunction, intersection, **if-else**, and the until operators) until it is replaced by a new clocking event.

For example:

$$@ (c) \ x \ | => \ @ (c) \ y \ \#\#1 \ @ (d) \ z$$

can be written more simply as

$$@ (c) \ x \ | => \ y \ \#\#1 \ @ (d) \ z$$

because clock c is understood to flow across $|=>$.

Clock flow also makes the adjointness relationships between concatenation and implication clean for multiclocked properties:

$$@ (c) \ x \ \#\#1 \ y \ | => \ @ (d) \ z$$

is equivalent to

$$@ (c) \ x \ | => \ y \ | => \ @ (d) \ z$$

and

$$@ (c) \ x \ \#\#0 \ y \ | => \ @ (d) \ z$$

is equivalent to

$$@ (c) \ x \ | -> \ y \ | => \ @ (d) \ z$$

The scope of a clocking event flows into parenthesized subexpressions and, if the subexpression is a sequence, also flows left to right across the parenthesized subexpression. However, the scope of a clocking event does not flow out of enclosing parentheses.

For example, in the following:

$$@ (c) \ w \ \#\#1 \ (x \ \#\#1 \ @ (d) \ y) \ | => \ z$$

w , x , and z are clocked at c , and y is clocked at d . Clock c flows across $\#\#1$, across the parenthesized subsequence $(x \ \#\#1 \ @ (d) \ y)$, and across $|=>$. Clock c also flows into the parenthesized subsequence, but it does not flow through $@ (d)$. Clock d does not flow out of its enclosing parentheses.

As another example, in the following:

```
@(c) v | => (w ##1 @(d) x) and (y ##1 z)
```

v , w , y , and z are clocked at c , and x is clocked at d . Clock c flows across $|=>$, distributes to both operands of the **and** (which is a property conjunction due to the multiple clocking), and flows into each of the parenthesized subexpressions. Within $(w \##1 \ @(d) \ x)$, c flows across $\##1$ but does not flow through $\ @(d)$. Clock d does not flow out of its enclosing parentheses. Within $(y \##1 \ z)$, c flows across $\##1$.

Similarly, the scope of a clocking event flows into an instance of a named property or sequence, regardless of whether method `triggered` or method `matched` is applied to the instance of the sequence. The scope of a clocking event flows left to right across an instance of a property or sequence. A clocking event in the declaration of a property or sequence does not flow out of an instance of that property or sequence.

The scope of a clocking event does not flow into the disable condition of **disable iff**.

Juxtaposing two clocking events nullifies the first of them; therefore, the following two-clocking-event statement:

```
@(d) @(c) x
```

is equivalent to the following:

```
@(c) x
```

because the flow of clock d is immediately overridden by clock c .

16.13.4 Examples

The following are examples of multiclock specifications:

```
sequence s1;
  a ##1 b; // unclocked sequence
endsequence
sequence s2;
  c ##1 d; // unclocked sequence
endsequence
```

a) Multiclock sequence

```
sequence mult_s;
  @(posedge clk) a ##1 @(posedge clk1) s1 ##1 @(posedge clk2) s2;
endsequence
```

b) Property with a multiclock sequence

```
property mult_p1;
  @(posedge clk) a ##1 @(posedge clk1) s1 ##1 @(posedge clk2) s2;
endproperty
```

c) Property with a named multiclock sequence

```
property mult_p2;
  mult_s;
endproperty
```

- d) Property with multiclock implication

```
property mult_p3;  
  @(posedge clk) a ##1 @(posedge clk1) s1 | => @(posedge clk2) s2;  
endproperty
```

- e) Property with implication, where antecedent and consequent are named multiclocked sequences

```
property mult_p6;  
  mult_s | => mult_s;  
endproperty
```

- f) Property using clock flow and overlapped implication

```
property mult_p7;  
  @(posedge clk) a ##1 b | -> c ##1 @(posedge clk1) d;  
endproperty
```

Here, a, b, and c are clocked at **posedge** clk.

- g) Property using clock flow and **if-else**

```
property mult_p8;  
  @(posedge clk) a ##1 b | ->  
    if (c)  
      (1 | => @(posedge clk1) d)  
    else  
      e ##1 @(posedge clk2) f ;  
endproperty
```

Here, a, b, c, e, and constant 1 are clocked at **posedge** clk.

16.13.5 Detecting and using end point of a sequence in multiclock context

Method `triggered` can be applied to detect the end point of a multiclocked sequence. Method `triggered` can also be applied to detect the end point of a sequence from within a multiclocked sequence. In both cases, the ending clock of the sequence instance to which `triggered` is applied shall be the same as the clock in the context where the application of method `triggered` appears.

To detect the end point of a sequence when the clock of the source sequence is different from the destination sequence, method `matched` on the source sequence is used. The end point of a sequence is reached whenever there is a match on its expression.

To detect the end point, the `matched` method may be applied to a named sequence instance, with or without arguments, an untyped formal argument, or a formal argument of type **sequence**, where such is allowed, as follows:

```
sequence_instance.matched  
or  
formal_argument_sequence.matched
```

`matched` is a method on a sequence that returns true (1'b1) or false (1'b0). Unlike `triggered`, `matched` uses synchronization between the two clocks, by storing the result of the source sequence match until the arrival of the first destination clock tick after the match. The result of `matched` does not depend upon the starting point of the source sequence.

Like `triggered`, `matched` can be used on sequences that have formal arguments. An example is shown as follows:

```
sequence e1(a,b,c);
  @(posedge clk) $rose(a) ##1 b ##1 c ;
endsequence
sequence e2;
  @(posedge sysclk) reset ##1 inst ##1 e1(ready,proc1,proc2).matched [->1]
  ##1 branch_back;
endsequence
```

In this example, source sequence `e1` is evaluated at clock `clk`, while the destination sequence `e2` is evaluated at clock `sysclk`. In `e2`, the end point of the instance `e1(ready,proc1,proc2)` is tested to occur sometime after the occurrence of `inst`. Notice that method `matched` only tests for the end point of `e1(ready,proc1,proc2)` and has no bearing on the starting point of `e1(ready,proc1,proc2)`.

Local variables can be passed into an instance of a named sequence to which `matched` is applied. The same restrictions apply as in the case of `triggered`. Values of local variables sampled in an instance of a named sequence to which `matched` is applied will flow out under the same conditions as for `triggered`. See [16.10](#).

As with `triggered`, a sequence instance to which `matched` is applied can have multiple matches in a single cycle of the destination sequence clock. The multiple matches are treated semantically the same way as matching both disjuncts of an `or`. In other words, the thread evaluating the destination sequence will fork to account for such distinct local variable valuations.

16.13.6 Sequence methods

Methods `triggered` and `matched` are available to identify the end point of a sequence. The operand sequence shall be a named sequence instance, with or without arguments, an untyped formal argument, or a formal argument of type **sequence**, in the contexts where such arguments are legal. These methods are invoked using the following syntax:

```
sequence_instance.sequence_method
or
formal_argument_sequence.sequence_method
```

The results of these operations are true (1'b1) or false (1'b0) and do not depend upon the starting point of the match of their operand sequence. These methods can be invoked on sequences with formal arguments. The sampled values of these methods are defined as the current values (see [16.5.1](#)).

The value of method `triggered` evaluates to true (1'b1) if the operand sequence has reached its end point at that particular point in time and false (1'b0) otherwise. The triggered status of the sequence is set in the Observed region and persists through the remainder of the time step. In addition to using this method in assertion statements, it may be used in `wait` statements (see [9.4.4](#)) or Boolean expressions outside a sequence context. It shall be considered an error to invoke this method outside a sequence context on sequences that treat their formal arguments as local variables. A sequence treats its formal argument as a local variable if the formal argument is used as an lvalue in *operator_assignment* or *inc_or_dec_expression* in *sequence_match_item*. There shall be no circular dependencies between sequences induced by the use of `triggered`.

The method `matched` is used to detect the end point of one sequence (the source sequence) referenced in a multiclocked sequence (the destination sequence). It can only be used in sequence expressions. Unlike `triggered`, `matched` provides synchronization between two clocks by storing the result of the source sequence until the arrival of the first clock tick of the destination sequence after the match. The `matched`

status of the sequence is set in the Observed region and persists until the Observed region following the arrival of the first clock tick of the destination sequence after the match.

It shall be considered an error to use the sequence method `matched` in sampled value functions (see [16.9.3](#)).

An example of using the previous methods on a sequence is shown as follows:

```
sequence e1;
  @(posedge sysclk) $rose(a) ##1 b ##1 c;
endsequence

sequence e2;
  @(posedge sysclk) reset ##1 inst ##1 e1.triggered ##1 branch_back;
endsequence

sequence e3;
  @(posedge clk) reset1 ##1 e1.matched ##1 branch_back1;
endsequence

sequence e2_with_arg(sequence subseq);
  @(posedge sysclk) reset ##1 inst ##1 subseq.triggered ##1 branch_back;
endsequence

sequence e4;
  e2_with_arg(@(posedge sysclk) $rose(a) ##1 b ##1 c);
endsequence

program check;
  initial begin
    wait (e1.triggered || e2.triggered);
    if (e1.triggered)
      $display("e1 passed");
    if (e2.triggered)
      $display("e2 passed");
    L2: ...
  end
endprogram
```

In the preceding example, sequence `e2` tests for the end point of sequence `e1` using method `triggered` because both sequences use the same clock. The sequence `e3` tests for the end point of sequence `e1` using method `matched` because `e1` and `e3` use different clocks. The sequence `e4` is semantically equivalent to `e2` and shows an application of the sequence method `triggered` on a formal argument of type **sequence**. The **initial** procedure in the program waits for the end point of either `e1` or `e2`. When either `e1` or `e2` evaluates to true, the wait statement unblocks the initial process. The process then displays the sequence that caused it to unblock, and then continues to execute at the statement labeled `L2`.

The sequence on which a method is applied shall either be clocked or infer the clock from the context where it is used. The same rules are used to infer the clocking event as specified in [16.9.3](#) for sampled value functions.

If the default actual argument `$inferred_clock` is specified for a formal argument of a sequence, and an actual argument is not provided to the sequence instance to which a method is applied, the same rules as specified in [16.9.3](#) for sampled value functions are used to determine the inferred clocking event expression that is bound to that formal argument.

If a sequence with a method is passed as an actual argument to a checker instantiation, it is substituted in place of the corresponding formal argument. Such a sequence shall be clocked as if it were instantiated inside the checker.

If a sequence with a method is connected to a port of a module instantiation, it shall be clocked as if it were instantiated at the place of module instantiation. The same rule shall apply if a sequence with a method is connected to a port of an interface or program instantiation or passed as an actual argument to a function or task call.

The preceding rules for inferring the clocking event also apply to a sequence instantiated in an event expression.

The following examples illustrate how a clock is inferred by a sequence when a method is applied to it.

```

module mod_sva_checks;
  logic a, b, c, d;
  logic clk_a, clk_d, clk_e1, clk_e2;
  logic clk_c, clk_p;

  clocking cb_prog @(posedge clk_p); endclocking
  clocking cb_checker @(posedge clk_c); endclocking

  default clocking cb @(posedge clk_d); endclocking

  sequence e4;
    $rose(b) ##1 c;
  endsequence

  // e4 infers posedge clk_a as per clock flow rules
  a1: assert property (@(posedge clk_a) a |=> e4.triggered);

  sequence e5;
    // e4 will infer posedge clk_e1 as per clock flow rules
    // wherever e5 is instantiated (with/without a method)
    @(posedge clk_e1) a ##[1:3] e4.triggered ##1 c;
  endsequence

  // e4, used in e5, infers posedge clk_e1 from e5
  a2: assert property (@(posedge clk_a) a |=> e5.matched);

  sequence e6(f);
    @(posedge clk_e2) f;
  endsequence

  // e4 infers posedge clk_e2 as per clock flow rules
  a3: assert property (@(posedge clk_a) a |=> e6(e4.triggered));

  sequence e7;
    e4 ##1 e6(d);
  endsequence

  // Leading clock of e7 is posedge clk_a as per clock flow rules
  a4: assert property (@(posedge clk_a) a |=> e7.triggered);

  // Illegal use in a disable condition, e4 is not explicitly clocked
  a5_illegal: assert property (
    @(posedge clk_a) disable iff (e4.triggered) a |=> b);

```

```

always @(posedge clk_a) begin
    // e4 infers default clocking cb and not posedge clk_a as there is
    // more than one event control in this procedure (16.14.6)
    @(e4);
    d = a;
end

program prog_e4;
    default clocking cb_prog;
    initial begin
        // e4 infers default clocking cb_prog
        wait (e4.triggered);
        $display("e4 passed");
    end
endprogram : prog_e4

checker check(input in1, input sequence s_f);
    default clocking cb_checker;
    always @(s_f)
        $display("sequence triggered");
    a4: assert property (a | => in1);
endchecker : check

// e4 infers checker's default clocking cb_checker
check c1(e4.triggered, e4);

// e4 connected to port of a module instance infers default clocking cb
mod_adder a1(e4.triggered);

endmodule : mod_sva_checks

```

If a sequence admits an empty match, such empty matches shall not activate the `.triggered` or `.matched` methods (see [16.9.11](#)).

More details about sequence methods can be found in [9.4.4](#), [16.9.11](#), and [16.13.5](#).

16.13.7 Local variable initialization assignments

For singly clocked sequences and properties, a local variable initialization assignment for an evaluation attempt of an instance of a named sequence or property is performed when the evaluation attempt begins. Such an evaluation attempt always begins in a time step in which there is a tick of the single governing clock.

For multiclock sequences and properties, a local variable initialization assignment for an evaluation attempt of an instance of a named sequence or property with a single semantic leading clock (see [16.16.1](#)) shall be performed at the earliest tick of the semantic leading clock that is at or after the beginning of the evaluation attempt. If there are two or more distinct semantic leading clocks for an instance of a named property, then a separate copy of the local variable shall be created for each semantic leading clock. For each copy of the local variable, the initialization assignment shall be performed at the earliest tick of the corresponding semantic leading clock that is at or after the beginning of the evaluation attempt, and that copy of the local variable shall be used in the evaluation of the subproperty associated with the corresponding semantic leading clock.

For example, let

```
property p;
  logic v = e;
  (@(posedge clk1) (a == v) [*1:$] |-> b)
  and
  (@(posedge clk2) c [*1:$] |-> d == v)
;
endproperty
a1: assert property (@(posedge clk) f |> p);
```

where *f* is of type **logic**. The instance of *p* in assertion *a1* has two semantic leading clocks, **posedge** *clk1* and **posedge** *clk2*. Separate copies of the local variable *v* are created for the two subproperties governed by these clocks. Let *t0* be a time step in which **posedge** *clk1* occurs and in which the sampled value of *f* is true. According to the structure of *a1*, an evaluation attempt of the instance of *p* starts strictly after *t0*. Let *t1* be the earliest time step after *t0* in which **posedge** *clk1* occurs, and let *t2* be the earliest time step after *t0* in which **posedge** *clk2* occurs. Then a declaration assignment *v* = *e* is performed in *t1*, and the value is assigned to the copy of *v* associated with **posedge** *clk1*. This value is used in the evaluation of the subproperty (a == v) [*1:\$] |-> b. Similarly, a declaration assignment *v* = *e* is performed in *t2*, and the value is assigned to the copy of *v* associated with **posedge** *clk2*. This value is used in the evaluation of the subproperty c [*1:\$] |-> d == v.

An equivalent declaration of *p* that does not use local variable declaration assignments is as follows:

```
property p;
  logic v;
  (@(posedge clk1) (1, v = e) ##0 (a == v) [*1:$] |-> b)
  and
  (@(posedge clk2) (1, v = e) ##0 c [*1:$] |-> d == v)
;
endproperty
```

16.14 Concurrent assertions

A property on its own is never evaluated for checking an expression. It shall be used within an assertion statement (see [16.2](#)) for this to occur.

A concurrent assertion statement may be specified in any of the following:

- An always procedure or initial procedure as a statement, wherever these procedures may appear (see [9.2](#))
- A module
- An interface
- A program
- A generate block
- A checker

```
concurrent_assertion_item ::=                                     // from A.2.10
  [ block_identifier : ] concurrent_assertion_statement
  ...
procedural_assertion_statement ::=                             // from A.6.10
  concurrent_assertion_statement
  ...
concurrent_assertion_statement ::=
  assert_property_statement
```

```

| assume_property_statement
| cover_property_statement
| cover_sequence_statement
| restrict_property_statement
assert_property_statement::=
    assert property ( property_spec ) action_block
assume_property_statement::=
    assume property ( property_spec ) action_block
cover_property_statement::=
    cover property ( property_spec ) statement_or_null
cover_sequence_statement::=
    cover sequence ( [clocking_event] [ disable iff ( expression_or_dist ) ]
        sequence_expr ) statement_or_null
restrict_property_statement::=
    restrict property ( property_spec ) ;

```

Syntax 16-20—Concurrent assert construct syntax (excerpt from [Annex A](#))

The execution of assertion statements can be controlled using assertion control system tasks (see [20.12](#)).

A concurrent assertion statement can be referenced by its optional name. A hierarchical name can be used consistent with the SystemVerilog naming conventions. When a name is not provided, a tool shall assign a name to the statement for the purpose of reporting. Unnamed assertions do not create a scope.

16.14.1 Assert statement

The **assert** statement is used to enforce a **property**. When the property for the **assert** statement is evaluated to be true, the pass statements of the action block are executed. When the property for the **assert** statement is evaluated to be false, the fail statements of the *action_block* are executed. When the property for the **assert** statement is evaluated to be disabled, no *action_block* statement is executed. The execution of pass and fail statements can be controlled by using assertion action control tasks. The assertion action control tasks are described in [20.12](#).

For example:

```

property abc(a, b, c);
    disable iff (a==2) @(posedge clk) not (b ##1 c);
endproperty
env_prop: assert property (abc(rst, in1, in2))
    $display("env_prop passed."); else $display("env_prop failed.");

```

When no action is needed, a null statement (i.e., ;) is specified. If no statement is specified for **else**, then `$error` is used as the statement when the assertion fails.

The *action_block* shall not include any concurrent **assert**, **assume**, or **cover** statement. The *action_block*, however, can contain immediate assertion statements.

The conventions regarding default severity (error) and the use of severity system tasks in concurrent assertion action blocks shall be the same as those specified for immediate assertions in [16.3](#).

The pass and fail statements of an assert statement are executed in the Reactive region. The regions of execution are explained in the scheduling semantics in [Clause 4](#).

16.14.2 Assume statement

The purpose of the **assume** statement is to allow properties to be considered as assumptions for formal analysis as well as for dynamic simulation tools. When a property is assumed, the tools constrain the environment so that the property holds.

For formal analysis, there is no obligation to verify that the assumed properties hold. An assumed property can be considered as a hypothesis to prove the asserted properties.

For simulation, the environment must be constrained so that the properties that are assumed shall hold. Like an asserted property, an assumed property must be checked and reported if it fails to hold. When the property for the **assume** statement is evaluated to be true, the pass statements of the *action_block* are executed. If it evaluates to false, the fail statements of the *action_block* are executed. For example:

```
property abc(a, b, c);
  disable iff (c) @(posedge clk) a | => b;
endproperty
env_prop:
  assume property (abc(req, gnt, rst)) else $error("Assumption failed.");
```

If the property has a disabled evaluation, neither the pass nor fail statements of the *action_block* are executed. The execution of pass and fail statements can be controlled by using assertion action control tasks. The assertion action control tasks are described in [20.12](#).

Additionally, for random simulation, biasing on the inputs provides a way to make random choices. An expression can be associated with biasing as follows:

```
expression dist { dist_list } ; // from A.2.10
```

Distribution sets and the **dist** operator are explained in [18.5.4](#).

The biasing feature is useful when properties are considered as assumptions to drive random simulation. When a property with biasing is used within an **assert** or **cover** assertion statement, the **dist** operator is equivalent to **inside** operator, and the weight specification is ignored. For example:

```
a1:assume property ( @(posedge clk) req dist {0:=40, 1:=60} ) ;
property proto ;
  @(posedge clk) req |-> req[*1:$] ##0 ack;
endproperty
```

This is equivalent to the following:

```
a1_assertion:assert property ( @(posedge clk) req inside {0, 1} ) ;
property proto_assertion ;
  @(posedge clk) req |-> req[*1:$] ##0 ack;
endproperty
```

In the preceding example, signal *req* is specified with distribution in assumption *a1* and is converted to an equivalent assertion *a1_assertion*.

It should be noted that the properties that are assumed must hold in the same way with or without biasing. When using an assume statement for random simulation, the biasing simply provides a means to select values of free variables, according to the specified weights, when there is a choice of selection at a particular time.

Consider an example specifying a simple synchronous request and acknowledge protocol, where variable `req` can be raised at any time and must stay asserted until `ack` is asserted. In the next clock cycle, both `req` and `ack` must be deasserted.

Properties governing `req` are as follows:

```
property pr1;
  @(posedge clk) !reset_n |-> !req;           // when reset_n is asserted (0),
                                              // keep req 0
endproperty
property pr2;
  @(posedge clk) ack |>= !req;               // one cycle after ack, req
                                              // must be deasserted
endproperty
property pr3;
  @(posedge clk) req |-> req[*1:$] ##0 ack; // hold req asserted until
                                              // and including ack asserted
endproperty
```

Properties governing `ack` are as follows:

```
property pa1;
  @(posedge clk) !reset_n || !req |-> !ack;
endproperty
property pa2;
  @(posedge clk) ack |>= !ack;
endproperty
```

When verifying the behavior of a protocol controller that has to respond to requests on `req`, assertions `assert_ack1` and `assert_ack2` should be proven while assuming that statements `a1`, `assume_req1`, `assume_req2`, and `assume_req3` hold at all times.

```
a1:assume property (@(posedge clk) req dist {0:=40, 1:=60} );
assume_req1:assume property (pr1);
assume_req2:assume property (pr2);
assume_req3:assume property (pr3);

assert_ack1:assert property (pa1)
  else $error("ack asserted while req is still deasserted");
assert_ack2:assert property (pa2)
  else $error("ack is extended over more than one cycle");
```

16.14.3 Cover statement

There exist two categories of cover statements: **cover sequence** and **cover property**. The **cover sequence** statement specifies sequence coverage, while the **cover property** statement specifies property coverage. Both monitor behavioral aspects of the design for coverage. Tools shall collect coverage information and report the results at the end of simulation or on demand via an assertion API (refer to [Clause 39](#)). The difference between the two categories is that for sequence coverage, all matches per evaluation attempt are reported, whereas for property coverage the coverage count is incremented at most once per evaluation attempt. A cover statement may have an optional pass statement. The pass statement shall not include any concurrent **assert**, **assume**, or **cover** statement.

For property coverage, the statement appears as follows:

```
cover property ( property_spec ) statement_or_null
```

The results of this coverage statement for a property shall contain the following:

- Number of times attempted
- Number of times succeeded (maximum of one per attempt)
- Number of times succeeded because of vacuity

The pass statement specified in *statement_or_null* shall be executed once for each successful evaluation attempt of the underlying *property_spec*. The pass statement shall be executed in the Reactive region of the time step in which the corresponding evaluation attempt succeeds. The execution of *statement_or_null* can be controlled by using assertion action control tasks. The assertion action control tasks are described in [20.12](#).

The preceding coverage counters for success or vacuous success do not include disabled evaluations. The attempt counter includes the attempts that result in disabled evaluation. See [40.5.3](#) for details on obtaining assertion coverage results.

For sequence coverage, the statement appears as follows:

```
cover sequence (
    [clocking_event] [ disable iff ( expression_or_dist ) ] sequence_expr )
    statement_or_null
```

Results of coverage for a sequence shall include the following:

- Number of times attempted
- Number of times matched (each attempt can generate multiple matches)

For a given attempt of the **cover sequence** statement, all matches of the *sequence_expr* that complete without the occurrence of the **disable iff** condition shall be counted, with multiplicity, toward the total number of times matched for the attempt. No other match shall be counted towards the total for the attempt. The pass statement specified in *statement_or_null* shall be executed, with multiplicity, for each match that is counted toward the total for the attempt. The pass statement shall be executed in the Reactive region of the time step in which the corresponding match completes. The execution of *statement_or_null* can be controlled by using assertion action control tasks. The assertion action control tasks are described in [20.12](#).

For a given attempt of the **cover sequence** statement, the total number of times matched for the attempt is equal to the number of times `increment_match_coverage()` is executed in the corresponding attempt of

```
assert property (
    [clocking_event] [ disable iff ( expression_or_dist ) ]
    sequence_expr |-> ( 1'b1, increment_match_coverage() ) );
```

For each execution of `increment_match_coverage()`, the pass statement of the cover sequence statement is executed in the Reactive region of the same time step.

16.14.4 Restrict statement

In formal verification, for the tool to converge on a proof of a property or to initialize the design to a specific state, it is often necessary to constrain the state space. For this purpose, the assertion statement **restrict property** is introduced. It has the same semantics as **assume property**, however, in contrast to that statement, the **restrict property** statement is not verified in simulation and has no action block.

The statement has the following form:

```
restrict property ( property_spec ) ;
```

There is no action block associated with the statement.

Example:

Suppose that when a control bit `ctr` has a value 0, an ALU performs an addition, and when it is 1, it performs a subtraction. It is required to formally verify that some behavior is correct when ALU does an addition (in another verification session it is possible to do the same for subtraction by changing the restriction). The behavior can thus be constrained using the statement:

```
restrict property (@(posedge clk) ctr == '0);
```

It does not mean that `ctr` cannot be 1 in any test case in the simulation; that is not an error.

16.14.5 Using concurrent assertion statements outside procedural code

A concurrent assertion statement can be used outside a procedural context. It can be used within a module, an interface, or a program. A concurrent assertion statement is an **assert**, an **assume**, a **cover**, or a **restrict** statement. Such a concurrent assertion statement uses the **always** semantics, meaning that it specifies that a new evaluation attempt of the underlying *property_spec* begins at every occurrence of its leading clock event.

The following two forms are equivalent:

```
assert property ( property_spec ) action_block  
  
always assert property ( property_spec ) action_block ;
```

Similarly, the following two forms are equivalent:

```
cover property ( property_spec ) statement_or_null  
  
always cover property ( property_spec ) statement_or_null
```

For example:

```
module top(input logic clk);  
  logic a,b,c;  
  property rule3;  
    @(posedge clk) a |-> b ##1 c;  
  endproperty  
  a1: assert property (rule3);  
  ...  
endmodule
```

rule3 is a property declared in module `top`. The `assert` statement `a1` starts checking the property from the beginning to the end of simulation. The property is always checked. Similarly,

```
module top(input logic clk);  
  logic a,b,c;  
  sequence seq3;  
    @(posedge clk) b ##1 c;  
  endsequence  
  c1: cover property (seq3);  
  ...  
endmodule
```

The cover statement `c1` starts coverage of the sequence `seq3` from beginning to the end of simulation. The sequence is always monitored for coverage.

16.14.6 Embedding concurrent assertions in procedural code

A concurrent assertion statement can also be embedded in a procedural block. For example:

```
property rule;
  a ##1 b ##1 c;
endproperty

always @(posedge clk) begin
  <statements>
  assert property (rule);
end
```

The term *procedural concurrent assertion* is used to refer to any concurrent assertion statement (see [16.2](#)) that appears in procedural code. Unlike an immediate assertion, a procedural concurrent assertion is not immediately evaluated when reached in procedural code. Instead, the assertion and the current values of all constant and automatic expressions appearing in its assertion arguments (see [16.14.6.1](#)) are placed in a *procedural assertion queue* associated with the currently executing process. Each of the entries in this queue is said to be a *pending procedural assertion instance*. Since any given statement in a procedure may be executed multiple times (as in a loop for example), a particular procedural concurrent assertion may result in many pending procedural assertion instances within a single time step. A concurrent assertion statement that appears outside procedural code is referred to as a *static concurrent assertion statement*.

In the Observed region of each simulation time step, each pending procedural assertion instance that is currently present in a procedural assertion queue shall *mature*, which means it is confirmed for execution. When a pending procedural assertion instance matures, if the current time step is one that corresponds to that assertion instance's leading clocking event, an evaluation attempt of the assertion begins immediately within the Observed region. If the assertion's leading clocking event has not occurred in this time step, the matured instance shall be placed on the *matured assertion queue*, which will cause the assertion to begin an evaluation attempt upon the next clocking event that corresponds to the leading clocking event of the assertion.

If a *procedural assertion flush point* (see [16.14.6.2](#)) is reached in a process, its procedural assertion queue is cleared. Any currently pending procedural assertion instances will not mature, unless again placed on the queue in the course of procedural execution.

If no clocking event is specified in a procedural concurrent assertion, the leading clocking event of the assertion shall be inferred from the procedural context, if possible. If no clock can be inferred from the procedural context, then the clocks shall be inferred from the default clocking ([14.12](#)), as if the assertion were instantiated immediately before the procedure.

A clock shall be inferred for the context of an always or initial procedure that satisfies the following requirements:

- a) There is no blocking timing control in the procedure.
- b) There is exactly one event control in the procedure.
- c) One and only one event expression within the event control of the procedure satisfies both of the following conditions:
 - 1) The event expression consists solely of an event variable, solely of a clocking block event, or is of the form *edge_identifier expression1* [**iff** *expression2*] and is not a proper subexpression of an event expression of this form.
 - 2) If the event expression consists solely of an event variable or clocking block event, it does not appear anywhere else in the body of the procedure other than as a clocking event or within

assertion statements. If the event expression is of the form *edge_identifier expression1 [iff expression2]*, no term in expression1 appears anywhere else in the body of the procedure other than as a clocking event or within assertion statements.

If these requirements are satisfied, then the unique event expression from the third requirement shall be the clock inferred for the context of the procedure.

For example, in the following code fragment, the clocking event `@(posedge mclk)` is inferred as the clocking event of `r1_p1`, while `r1_p2` is clocked by `@(posedge scanclk)` as written:

```
property r1;
  q != d;
endproperty
always @(posedge mclk) begin
  q <= d1;
  r1_p1: assert property (r1);
  r1_p2: assert property (@(posedge scanclk) r1);
end
```

The resulting behavior of the preceding assertion `r1_p2` depends on the relative frequencies of `mclk` and `scanclk`. For example:

- If `scanclk` runs at twice the frequency of `mclk`, only every other `posedge` of `scanclk` will result in an evaluation of `r1_p2`. It is only queued when reached during procedural execution, which happens on a rising edge of `mclk`.
- If `mclk` runs at twice the frequency of `scanclk`, then by every `posedge` of `scanclk`, two pending procedural instances of `r1_p2` will mature. Thus every `posedge` of `scanclk` will see `r1_p2` evaluated and results reported twice.

Also see [17.4](#) for the context clock inference in checkers, and [17.5](#) for examples of clock inference in checker procedures.

Another, more complex example that is legal is as follows:

```
property r2;
  q != d;
endproperty

always_ff @(posedge clock iff reset == 0 or posedge reset) begin
  cnt <= reset ? 0 : cnt + 1;
  q <= $past(d1);
  r2_p: assert property (r2);
end
```

In the preceding example, the inferred clock is `posedge clock iff reset == 0`. The inferred clock is not `posedge clock` because `posedge clock` is a proper subexpression of `posedge clock iff reset == 0`.

In contrast, no clock is inferred for the context of the `always_ff` in the following:

```
property r3;
  q != d;
endproperty

always_ff @(clock iff reset == 0 or posedge reset) begin
  cnt <= reset ? 0 : cnt + 1;
```

```

    q <= $past(d1);           // no inferred clock
    r3_p: assert property (r3); // no inferred clock
end

```

The edge expression **posedge** reset cannot be inferred because reset is referenced within the procedure, and the expression clock **iff** reset == 0 cannot be inferred because it does not have an edge identifier. In the absence of default clocking, the code above results in an error.

In the following example, no clock is inferred due to multiple event controls and delays in the always procedure.

```

property r4;
    q != d;
endproperty

always @(posedge mclk) begin
    #10 q <= d1;           // delay prevents clock inference
    @(negedge mclk)        // event control prevents clock inference
    #10 q1 <= !d1;
    r4_p: assert property (r4); // no inferred clock
end

```

16.14.6.1 Arguments to procedural concurrent assertions

A procedural concurrent assertion saves the value of its **const** expressions and automatic variables at the time the assertion evaluation attempt is added to the procedural assertion queue. This assertion evaluation attempt uses these saved values for the evaluation. For example:

```

// Assume for this example that (posedge clk) will not occur at time 0
always @(posedge clk) begin
    int i = 10;
    for (i=0; i<10; i++) begin
        a1: assert property (foo[i] && bar[i]);
        a2: assert property (foo[const'(i)] && bar[i]);
        a3: assert property (foo[const'(i)] && bar[const'(i)]);
    end
end

```

In any given clock cycle, each of these assertions will result in 10 queued executions. Every execution of assertion a1, however, will be checking the value of (foo[10] && bar[10]), since the sampled value of i will always be 10, its final value from the previous execution of the procedure. In the case of a2, its executions will be checking (foo[0] && bar[10]), (foo[1] && bar[10]), ... (foo[9] && bar[10]). Assertion a3, since it has **const** casts on both uses of i, will be checking (foo[0] && bar[0]), (foo[1] && bar[1]), ... (foo[9] && bar[9]). So the preceding code fragment is logically equivalent (aside from instance names) to the following:

```

default clocking @(posedge clk); endclocking
generate for (genvar i=0; i<10; i++) begin
    a1: assert property (foo[10] && bar[10]);
    a2: assert property (foo[i] && bar[10]);
    a3: assert property (foo[i] && bar[i]);
end
endgenerate

```

Since automatic variables also have their immediate values preserved, in the following example, all three properties a4, a5, and a6 are logically equivalent:

```

always @(posedge clk) begin
    // variable declared in for statement is automatic (see 12.7.1)
    for (int i=0; i<10; i++) begin
        a4: assert property (foo[i] && bar[i]);
        a5: assert property (foo[const'(i)] && bar[i]);
        a6: assert property (foo[const'(i)] && bar[const'(i)]);
    end
end

```

When a procedural concurrent assertion contains temporal expressions and has matured, the execution flow of the procedure no longer directly affects the matured instance in future time steps. In other words, the procedural execution only affects the activation of the assertion instance, not the completion of temporal expressions in the future. However, any constant values that were passed into the assertion instance due to constant or automatic variables will remain constant for the duration of that instance’s evaluation. The following example illustrates this behavior:

```

wire w;
always @(posedge clk) begin : procedural_block_1
    if (my_activation_condition == 1) begin
        for (int i=0; i<2; i++) begin
            a7: assume property (foo[i] ==> bar[i] ##1 (w==1'b1));
        end
    end
end

```

During the time step when `my_activation_condition` is 1, two pending instances of `a7` will be placed on the procedural assertion queue, one for each value of `i`. Assume that they successfully mature, and `foo[0]` is true in the current time step. This means that on the next posedge of `clk`, regardless of the execution of `procedural_block_1` or the value of `my_activation_condition`, that matured instance of `a7` will be checking that `bar[0]` is true. The constant value of the automatic `i` from when the assertion was queued is still in effect, for this and any future clock cycles of this assertion evaluation. Then, one cycle later, the assertion will also be checking that the sampled value of `w` is 1'b1.

The same rules that apply to procedural concurrent assertion arguments also apply to variables appearing in their action blocks. Thus, constant or automatic values may be used in action blocks as well as the assertion statements themselves, where they behave as inputs to the action block that shall not be modified. The following example illustrates this behavior:

```

// Assume for this example that (posedge clk) will not occur at time 0
always @(posedge clk) begin
    int i = 10;
    for (i=0; i<10; i++) begin
        a8: assert property (foo[const'(i)] && bar[i]) else
            $error("a8 failed for const i=%d and i=%d",
                const'(i), $sampled(i));
    end
end

```

Upon a failure, any instance of the preceding assertion will show the constant value of `i` (may be from 0 to 9) that was used in that instance for “const i=”, while the string printed will always end in “i=10”, since 10 will be the sampled value captured from the Preponed region.

When embedding procedural concurrent assertions in code using conditionals, it is important to remember that the current values of the conditionals in the procedure are used, rather than the sampled values. This

contrasts with the assertion's expressions, where sampled values are used (see [16.5.1](#)). The following example illustrates this situation:

```
// Assume a, b, c, and en are not automatic
always @(posedge clk) begin
    en = ...;
    if (en) begin
        a9: assert property p1(a,b,c);
    end
    if ($sampled(en)) begin
        a10: assert property p1(a,b,c);
    end
end
```

Assertion a9 is queued on any time step when *en* becomes true, while a10 is queued on any time step when the sampled value of *en* was true. Thus, assuming nothing else in the code modifies *en*, checks of a10 will happen a time step later than checks on a9, even though both use the sampled values of *a*, *b*, and *c* on their respective time steps.

NOTE—This is an area of backwards-incompatibility between this standard and 17.13 of IEEE Std 1800-2005. In the 2005 definition, *en* would have been detected as the *inferred enabling condition* (a definition that no longer exists in this standard) of a9 and always sampled, so a9 and a10 would have identical behavior.

16.14.6.2 Procedural assertion flush points

A process is defined to have reached a procedural assertion flush point if any of the following occur:

- The process, having been suspended earlier due to reaching an event control or **wait** statement, resumes execution.
- The process was declared by an **always_comb** or **always_latch**, and its execution is resumed due to a transition on one of its dependent signals.
- The outermost scope of the process is disabled by a **disable** statement (see [16.14.6.4](#)).

The following example shows how procedural concurrent assertions inherently avoid multiple evaluations due to transitional combinational values in a single simulation time step:

```
assign not_a = !a;
default clocking @(posedge clk); endclocking
always_comb begin : b1
    // Probably better to not use consts in this example
    // ...but using them to illustrate effects of flushing method
    a1: assert property (const'(not_a) != const'(a));
end
```

When *a* changes in a time step during which a positive clock edge occurs, a simulator could evaluate assertion a1 twice—once for the change in *a* and once for the change in *not_a* after the evaluation of the continuous assignment. The first execution of a1, which would have ended up reporting a failure, will be scheduled on the process's procedural assertion queue. When *not_a* changes, the procedural assertion queue is flushed due to the activation of b1, and a new pending instance of the procedural concurrent assertion will now be queued with the correct values, so no failure of a1 will be reported.

The following example illustrates the behavior of procedural concurrent assertions in the presence of time delays:

```
default clocking @(posedge clk); endclocking
always @(a or b) begin : b1
    a2: assert property (a == b) r.success(0) else r.error(0, a, b);
end
```

```
#1;
a3: assert property (a == b) r.success(1) else r.error(1, a, b);
end
```

In this case, due to the time delay in the middle of the procedure, an Observed region will always be reached after the queueing of a2 and before a flush point. Thus a2 will always mature. For a3, during time steps where either a or b changes after it has been queued, the assertion will always be flushed from the queue and never mature. In general, procedural concurrent assertions must be used carefully when mixed with time delays.

The following example illustrates a typical use of a procedural concurrent assertion statement with a **cover** rather than an **assert**:

```
assign a = ...;
assign b = ...;
default clocking @(posedge clk); endclocking
always_comb begin : b1
    ...
    c1: cover property (const'(b) != const'(a));
end
```

In this example, the goal is to make sure some test is covering the case where a and b have different values at that point in the procedural code. Due to the arbitrary order of the assignments in the simulator, it might be the case that in a cycle where there is a positive clock edge and both variables are being assigned the same value, b1 executes while a has been assigned but b still holds its previous value. Thus c1 will be queued, but this is actually a glitch, and probably not a useful piece of coverage information. But, when b1 is executed the next time (after b has also been assigned its new value), that coverage point will be flushed, and when the coverage point matures, c2 will correctly not get reported as having been covered during that time step.

16.14.6.3 Procedural concurrent assertions and glitches

One common concern with assertion execution is glitches, where the same assertion executes multiple times in a time step and reports undesired failures on temporary values that have not yet received their final values for the step. In general, procedural concurrent assertions are immune to glitches due to order of procedural execution due to the flushing mechanism, but are still subject to glitches caused by execution loops between regions.

For example, if code in the Reactive region modifies signals and causes another pass to the Active region to occur, this may create some glitching behavior, as the new passage in the Active region may requeue procedural concurrent assertions, and a second evaluation attempt may be added to the matured assertion queue. The following code illustrates this situation.

```
always_comb begin : procedural_block_1
    if (en)
        foo = bar;
end

always_comb begin : procedural_block_2
    p1: assert property ( @(posedge clk) (const'(foo) == const'(bar)) );
end
```

Suppose bar is assigned a new value elsewhere in the code at the posedge of the clock, and en is 1 so the assignment in procedural_block_1 takes place. Block procedural_block_2 may be executed twice in the Active region: once upon the initial change to bar, and once after the assignment that updates foo. Upon the first execution of procedural_block_2, a pending instance of p1 will be queued and would

result in failure of the assertion if it matured. But this instance will be flushed upon the second execution of the procedural block before maturing, and thus there will be no glitch.

However, now suppose that in the same example, `en` is 0, and the assignment of the `bar` value to `foo` happens through VPI code in the Reactive region. In this case, the Observed region has already occurred, so `p1` has matured and executed, and reported the assertion failure due to `foo` and `bar` having different values. After the Reactive region, there will be another Active region in which `procedural_block_2` will be executed, and this time a newly queued instance of `p1` will pass. But this is too late to prevent the report of the failure earlier in the time step.

16.14.6.4 Disabling procedural concurrent assertions

The **disable** statement shall interact with procedural concurrent assertions as follows:

- A specific procedural concurrent assertion may be disabled. Any pending procedural instances of that assertion are cleared from the queue. Any pending procedural instances of other assertions remain in the queue.
- When a **disable** is applied to the outermost scope of a procedure that has a pending procedural assertion queue, in addition to normal disable activities (see 9.6.2), the pending procedural assertion queue is flushed and all pending assertion instances on the queue are cleared.

Once a procedural concurrent assertion evaluation attempt has matured, it shall not be impacted by any disable.

Disabling a task or a non-outermost scope of a procedure does not cause flushing of any pending procedural assertion instances.

The following example illustrates how user code can explicitly flush a pending procedural assertion instance. In this case, instances of `a1` only mature in time steps where `bad_val_ok` does not settle at a value of 1.

```
default clocking @(posedge clk); endclocking
always @(bad_val or bad_val_ok) begin : b1
    a1: assert property (bad_val) else $fatal(1, "Sorry");
    if (bad_val_ok) begin
        disable a1;
    end
end
```

The following example illustrates how user code can explicitly flush all pending procedural assertion instances on the procedural assertion queue of process `b2`:

```
default clocking @(posedge clk); endclocking
always @(a or b or c) begin : b2
    if (c == 8'hff) begin
        a2: assert property (a && b);
    end else begin
        a3: assert property (a || b);
    end
end

always @(clear_b2) begin : b3
    disable b2;
end
```

16.14.7 Inferred value functions

The following elaboration time system functions are available to query the inferred clocking event expression and disable expression:

- `$inferred_clock` returns the expression of the inferred clocking event.
- `$inferred_disable` returns the inferred disable expression.

The inferred clocking event expression is the current resolved event expression that can be used in a clocking event definition. It is obtained by applying clock flow rules to the point where `$inferred_clock` is called. If there is no current resolved event expression when `$inferred_clock` is encountered then an error shall be issued.

The inferred disable expression is the disable condition from the default disable declaration whose scope includes the call to `$inferred_disable` (see [16.15](#)). If the call to `$inferred_disable` is not within the scope of any default disable declaration, then the call to `$inferred_disable` returns 1'b0 (false).

A call to an inferred expression function may only be used as the entire default value expression for a formal argument to a property or sequence declaration. A call to an inferred expression function shall not appear within the body expression of a property or sequence declaration. If a call to an inferred expression function is used as the entire default value expression for a formal argument to a property or sequence declaration, then it is replaced by the inferred expression as determined at the point where the property or sequence is instantiated. Therefore, if the property or sequence instance is the top-level property expression in an assertion statement, the event expression that is used to replace the default argument `$inferred_clock` is that as determined at the location of the assertion statement. If the property or sequence instance is not the top-level property expression in the assertion statement, then the event expression determined by clock flow rules up to the instance location in the property expression is used as the default value of the argument.

Consider the following example:

```

module m(logic a, b, c, d, rst1, clk1, clk2);

    logic rst;

    default clocking @(negedge clk1); endclocking
    default disable iff rst1;

    property p_triggers(start_event, end_event, form, clk = $inferred_clock,
                      rst = $inferred_disable);
        @clk disable iff (rst)
            (start_event ##0 end_event[->1]) | => form;
    endproperty

    property p_multiclock(clkw, clkx = $inferred_clock, clkx, w, x, y, z);
        @clkw w ##1 @clkx x | => @clkx y ##1 z;
    endproperty

    a1: assert property (p_triggers(a, b, c));
    a2: assert property (p_triggers(a, b, c, posedge clk1, 1'b0) );

    always @(posedge clk2 or posedge rst) begin
        if (rst) ... ;
        else begin
            a3: assert property (p_triggers(a, b, c));
            ...
        end
    end

```

```
a4: assert property(p_multiclock(negedge clk2, , posedge clk1,
    a, b, c, d) );

endmodule
```

The preceding code is logically equivalent to the following:

```
module m(logic a, b, c, d, rst1, clk1, clk2);

    logic rst;

    a1: assert property (@(negedge clk1) disable iff (rst1)
        a ##0 b[->1] |=> c);

    a2: assert property (@(posedge clk1) disable iff (1'b0)
        a ##0 b[->1] |=> c);

    always @(posedge clk2 or posedge rst) begin
        if (rst) ... ;
        else begin
            ...
        end
    end

    a3: assert property
    (
        @(posedge clk2) disable iff (rst1)
        (a ##0 b[->1]) |=> c
    );

    a4: assert property (@(negedge clk2) a ##1 @(negedge clk1) b |=>
        @(posedge clk1) c ##1 d);

endmodule
```

In assertion a1 the clock event is inferred from the default clocking, therefore `$inferred_clock` is **negedge** clk1 for a1. In assertion a2 the event expression **posedge** clk1 is passed to the formal argument `clk` in the instance of property `p_triggers`. Therefore, the `$inferred_clock` is not used for `clk` in that instance. In assertion a3 the clocking event is inferred from the event control of the `always` procedure, therefore `$inferred_clock` is **posedge** clk2 for a3.

In assertion a4, as the property `p_multiclock` is instantiated in the **assert property** statement, `clkw` is replaced by the actual argument (**negedge** clk2), `clkx` by the default argument value `$inferred_clock`, which is the default clocking clock (**negedge** clk1) at the location of the property instance in the assertion. The third clock, `clky`, is replaced by the actual argument (**posedge** clk1) because it is explicitly specified.

The disable condition `rst1` is inferred for assertions a1 and a3 from the default disable statement. Assertion a2 uses explicit reset value `1'b0` in which case the **disable iff** statement could be omitted altogether in the equivalent assertion.

16.14.8 Nonvacuous evaluations

An evaluation attempt of a property is either vacuous or nonvacuous. In particular, a vacuous success on all evaluation attempts may indicate a potential problem either in the design or in the formulation of the property. For example,

a |-> b

is evaluated as a vacuous success when a is false. In that case the evaluation is independent of the value of b; even though it is a successful evaluation, the property behavior is interpreted as not matching the user intent, meaning that an assertion of this property is not considered a pass or a failure.

For a general property, nonvacuous evaluation is defined recursively on the structure of the property as follows:

- a) An evaluation attempt of a property that is a sequence is always nonvacuous.
- b) An evaluation attempt of a property of the form **strong** (*sequence_expr*) is always nonvacuous.
- c) An evaluation attempt of a property of the form **weak** (*sequence_expr*) is always nonvacuous.
- d) An evaluation attempt of a property of the form **not** *property_expr* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr* is nonvacuous.
- e) An evaluation attempt of a property of the form *property_expr1* **or** *property_expr2* is nonvacuous if, and only if, either the underlying evaluation attempt of *property_expr1* is nonvacuous or the underlying evaluation attempt of *property_expr2* is nonvacuous.
- f) An evaluation attempt of a property of the form *property_expr1* **and** *property_expr2* is nonvacuous if, and only if, either the underlying evaluation attempt of *property_expr1* is nonvacuous or the underlying evaluation attempt of *property_expr2* is nonvacuous.
- g) An evaluation attempt of a property of the form **if** (*expression_or_dist*) *property_expr1* is nonvacuous if, and only if, *expression_or_dist* evaluates to true and the underlying evaluation attempt of *property_expr1* is nonvacuous.
An evaluation attempt of a property of the form **if** (*expression_or_dist*) *property_expr1* **else** *property_expr2* is nonvacuous if, and only if, either *expression_or_dist* evaluates to true and the underlying evaluation attempt of *property_expr1* is nonvacuous, or *expression_or_dist* evaluates to false and the underlying evaluation attempt of *property_expr2* is nonvacuous.
- h) An evaluation attempt of a property of the form *sequence_expression* |-> *property_expr* is nonvacuous if, and only if, there is an end point of the antecedent *sequence_expression* and the evaluation attempt of *property_expr* that starts at the end point is nonvacuous.
An evaluation attempt of a property of the form *sequence_expression* |=> *property_expr* is nonvacuous if, and only if, there is a match point of the antecedent *sequence_expression* and the evaluation attempt of *property_expr* that starts at the clock event following the match point is nonvacuous.
- i) An evaluation attempt of an instance of a property is nonvacuous if, and only if, the underlying evaluation attempt of the *property_expr* that results from substituting actual arguments for formal arguments is nonvacuous.
- j) An evaluation attempt of a property of the form *sequence_expression* #-# *property_expr* is nonvacuous if, and only if, there is an end point of the antecedent *sequence_expression* and the evaluation attempt of *property_expr* that starts at the end point is nonvacuous.
- k) An evaluation attempt of a property of the form *sequence_expression* #=# *property_expr* is nonvacuous if, and only if, there is a match point of the antecedent *sequence_expression* and the evaluation attempt of *property_expr* that starts at the clock event following the match point is nonvacuous.
- l) An evaluation attempt of a property of the form **nexttime** *property_expr* is nonvacuous if, and only if, there is at least one more clock event, and in the evaluation attempt that starts in the next clock event, *property_expr* is nonvacuous.
- m) An evaluation attempt of a property of the form **nexttime** [*constant_expression*] *property_expr* is nonvacuous if, and only if, there is at least *constant_expression* more clock events, and

property_expr is nonvacuous in the evaluation attempt beginning at the last of the next *constant_expression* clock events.

- n) An evaluation attempt of a property of the form **s_nexttime** *property_expr* is nonvacuous if, and only if, there is at least one more clock event, and in the evaluation attempt starting at the next clock event, *property_expr* is nonvacuous.
- o) An evaluation attempt of a property of the form **s_nexttime**[*constant_expression*] *property_expr* is nonvacuous if, and only if, there is at least *constant_expression* more clock events, and *property_expr* is nonvacuous in the evaluation attempt beginning at the last of the next *constant_expression* clock events.
- p) An evaluation attempt of a property of the form **always** *property_expr* is nonvacuous if, and only if, there is a clock event where the evaluation attempt of *property_expr* is nonvacuous, and *property_expr* does not fail in prior clock events.
- q) An evaluation attempt of a property of the form **always**[*cycle_delay_const_range_expression*] *property_expr* is nonvacuous if, and only if, there is a clock event within the range specified by *cycle_delay_const_range_expression*, in which the evaluation attempt of *property_expr* is nonvacuous, and the *property_expr* does not fail in prior clock events within the range specified by *cycle_delay_const_range_expression*.
- r) An evaluation attempt of a property of the form **s_always**[*constant_range*] *property_expr* is nonvacuous if, and only if, there is a clock event within the range specified by *constant_range*, in which the evaluation attempt of *property_expr* is nonvacuous, and *property_expr* does not fail in prior clock events within the range specified by *constant_range*.
- s) An evaluation attempt of a property of the form **s_eventually** *property_expr* is nonvacuous if, and only if, there is a clock event in which the evaluation attempt of *property_expr* is nonvacuous, and the *property_expr* does not hold in prior clock events.
- t) An evaluation attempt of a property of the form **s_eventually**[*cycle_delay_const_range_expression*] *property_expr* is nonvacuous if, and only if, there is a clock event within the range specified by *cycle_delay_const_range_expression*, in which the evaluation attempt of *property_expr* is nonvacuous, and *property_expr* does not hold in prior clock events within the range specified by *cycle_delay_const_range_expression*.
- u) An evaluation attempt of a property of the form **eventually**[*constant_range*] *property_expr* is nonvacuous if, and only if, there is a clock event within the range specified by *constant_range*, in which the evaluation attempt of *property_expr* is nonvacuous, and *property_expr* does not hold in prior clock events within the range specified by *constant_range*.
- v) An evaluation attempt of a property of the form *property_expr1* **until** *property_expr2* is nonvacuous if, and only if, there is a clock event in which either the evaluation attempt of *property_expr1* or the evaluation attempt of *property_expr2* is nonvacuous, *property_expr2* does not hold in prior clock events, and *property_expr1* holds in all prior clock events.
- w) An evaluation attempt of a property of the form *property_expr1* **s_until** *property_expr2* is nonvacuous if, and only if, there is a clock event in which either the evaluation attempt of *property_expr1* or the evaluation attempt of *property_expr2* is nonvacuous, *property_expr2* does not hold in prior clock events, and *property_expr1* holds in all prior clock events.
- x) An evaluation attempt of a property of the form *property_expr1* **until_with** *property_expr2* is nonvacuous if, and only if, there is a clock event in which the evaluation attempt of *property_expr1* is nonvacuous, *property_expr2* does not hold in prior clock events, and *property_expr1* holds in all prior clock events.
- y) An evaluation attempt of a property of the form *property_expr1* **s_until_with** *property_expr2* is nonvacuous if, and only if, there is a clock event in which the evaluation attempt of *property_expr1* is nonvacuous, *property_expr2* does not hold in prior clock events, and *property_expr1* holds in all prior clock events.

- z) An evaluation attempt of a property of the form *property_expr1* **implies** *property_expr2* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr1* is true and nonvacuous, and the underlying evaluation attempt of *property_expr2* is nonvacuous.
- aa) An evaluation attempt of a property of the form *property_expr1* **iff** *property_expr2* is nonvacuous if, and only if, either the evaluation attempt of *property_expr1* is nonvacuous or the evaluation attempt of *property_expr2* is nonvacuous.
- ab) An evaluation attempt of a property of the form **accept_on** (*expression_or_dist*) *property_expr* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr* is nonvacuous and *expression_or_dist* does not hold in any time step of that evaluation attempt.
- ac) An evaluation attempt of a property of the form **reject_on** (*expression_or_dist*) *property_expr* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr* is nonvacuous and *expression_or_dist* does not hold in any time step of that evaluation attempt.
- ad) An evaluation attempt of a property of the form **sync_accept_on**(*expression_or_dist*) *property_expr* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr* is nonvacuous and *expression_or_dist* does not hold in any clock event of that evaluation attempt.
- ae) An evaluation attempt of a property of the form **sync_reject_on**(*expression_or_dist*) *property_expr* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr* is nonvacuous and *expression_or_dist* does not hold in any clock event of that evaluation attempt.
- af) An evaluation attempt of a property of the form

```

case (expression_or_dist)
    expression_or_dist1 : property_stmt1
    ...
    expression_or_distn : property_stmtn
    [ default : property_stmtd ]
endcase

```

is nonvacuous if, and only if:

- For some index *i* such that $1 \leq i \leq n$, (*expression_or_dist* === *expression_or_dist*_{*i*}), and
- For each index *j* such that $1 \leq j < i$, (*expression_or_dist* !== *expression_or_dist*_{*j*}), and
- The underlying evaluation attempt of *property_stmt*_{*i*} is nonvacuous

or

- The default is present, and
- For each index *i* such that $1 \leq i \leq n$, (*expression_or_dist* !== *expression_or_dist*_{*i*}), and
- The underlying evaluation attempt of *property_stmt*_{*d*} is nonvacuous.

- ag) An evaluation attempt of a property of the form **disable iff** (*expression_or_dist*) *property_expr* is nonvacuous if, and only if, the underlying evaluation attempt of *property_expr* is nonvacuous and *expression_or_dist* does not hold in any time step of that evaluation attempt.

An evaluation attempt of a property succeeds nonvacuously if, and only if, the property evaluates to true and the evaluation attempt is nonvacuous.

16.15 Disable iff resolution

```

module_or_generate_item_declaration ::=
    ...
    | default clocking clocking_identifier ;
    | default disable iff expression_or_dist ;

```

Syntax 16-21—Default clocking and default disable syntax (excerpt from [Annex A](#))

A **default disable iff** may be declared within a generate block or within a module, interface, or program declaration. It provides a default disable condition to all concurrent assertions in the scope and subscope of the **default disable iff** declaration. Furthermore, the default extends to any nested module, interface, or program declarations, and to nested generate blocks. However, if a nested module, interface, or program declaration, or a generate block itself has a **default disable iff** declaration, then that **default disable iff** applies within the nested declaration or generate block and overrides any **default disable iff** from outside. Any signals referenced in the **disable iff** declaration that are resolved using scopes will be resolved from the scope of the declaration.

The effect of a **default disable iff** declaration is independent of the position of the declaration within that scope. More than one **default disable iff** declaration within the same module, interface, program declaration, or generate block shall be an error. The scope does not extend into any instances of modules, interfaces, or programs.

In the following example, module `m1` declares `rst1` to be the default disable condition, and there is no **default disable iff** declaration in the nested module `m2`. The default disable condition `rst1` applies throughout the declaration of `m1` and the nested declaration of `m2`. Therefore, the inferred disable condition of both assertions `a1` and `a2` is `rst1`.

```

module m1;
  bit clk, rst1;
  default disable iff rst1;
  a1: assert property (@(posedge clk) p1);    // property p1 is
                                              // defined elsewhere
  ...
  module m2;
    bit rst2;
    ...
    a2: assert property (@(posedge clk) p2); // property p2 is
                                              // defined elsewhere
  endmodule
  ...
endmodule

```

If there is a **default disable iff** declaration in the nested module `m2`, then within `m2` this default disable condition overrides the default disable condition declared in `m1`. Therefore, in the following example the inferred disable condition of `a1` is `rst1`, but the inferred disable condition of `a2` is `rst2`.

```

module m1;
  bit clk, rst1;
  default disable iff rst1;
  a1: assert property (@(posedge clk) p1);    // property p1 is
                                              // defined elsewhere
  ...
  module m2;
    bit rst2;
    default disable iff rst2;
    ...
    a2: assert property (@(posedge clk) p2); // property p2 is
                                              // defined elsewhere
  endmodule
  ...
endmodule

```

The following rules apply for resolution of the disable condition:

- a) If an assertion has a **disable iff** clause, then the disable condition specified in this clause shall be used and any **default disable iff** declaration ignored for this assertion.
- b) If an assertion does not contain a **disable iff** clause, but the assertion is within the scope of a **default disable iff** declaration, then the disable condition for the assertion is inferred from the **default disable iff** declaration.
- c) Otherwise, no inference is performed (this is equivalent to the inference of a 1'b0 disable condition).

Following are two example modules illustrating the application of these rules:

```

module examples_with_default (input logic a, b, clk, rst, rst1);
  default disable iff rst;
  property p1;
    disable iff (rst1) a | => b;
  endproperty

  // Disable condition is rst1 - explicitly specified within a1
  a1 : assert property (@(posedge clk) disable iff (rst1) a | => b);

  // Disable condition is rst1 - explicitly specified within p1
  a2 : assert property (@(posedge clk) p1);

  // Disable condition is rst - no explicit specification, inferred from
  // default disable iff declaration
  a3 : assert property (@(posedge clk) a | => b);

  // Disable condition is 1'b0. This is the only way to
  // cancel the effect of default disable.
  a4 : assert property (@(posedge clk) disable iff (1'b0) a | => b);
endmodule

module examples_without_default (input logic a, b, clk, rst);
  property p2;
    disable iff (rst) a | => b;
  endproperty

  // Disable condition is rst - explicitly specified within a5
  a5 : assert property (@(posedge clk) disable iff (rst) a | => b);

  // Disable condition is rst - explicitly specified within p2
  a6 : assert property (@ (posedge clk) p2);

  // No disable condition
  a7 : assert property (@ (posedge clk) a | => b);

endmodule

```

16.16 Clock resolution

There are a number of ways to specify a clock for a property. They are as follows:

- Sequence instance with a clock, for example:

```

sequence s2; @(posedge clk) a ##2 b; endsequence
property p2; not s2; endproperty
assert property (p2);

```

- Property, for example:

```
property p3; @(posedge clk) not (a ##2 b); endproperty  
assert property (p3);
```

- Contextually inferred clock from a procedural block, for example:

```
always @(posedge clk) assert property (not (a ##2 b));
```

- A **clocking** block, for example:

```
clocking master_clk @(posedge clk);  
    property p3; not (a ##2 b); endproperty  
endclocking  
assert property (master_clk.p3);
```

- Default clock, for example:

```
default clocking master_clk ; // master clock as defined above  
property p4; (a ##2 b); endproperty  
assert property (p4);
```

In general, a clocking event applies throughout its scope except where superseded by an inner clocking event, as with clock flow in multiclocked sequences and properties. The following rules apply (the term *maximal property*, used in the rules below, is defined as the unique flattened property contained in the assertion statement and obtained by applying the rewriting algorithm in [F.4.1](#)):

- In a module, interface, program, or checker with a default clocking event, a concurrent assertion statement that has no otherwise specified leading clocking event is treated as though the default clocking event had been written explicitly as the leading clocking event. The default clocking event does not apply to a sequence or property declaration except in the case that the declaration appears in a **clocking** block whose clocking event is the default.
- The following rules apply within a **clocking** block:
 - No explicit clocking event is allowed in any property or sequence declaration within the **clocking** block. All sequence and property declarations within the **clocking** block are treated as though the clocking event of the **clocking** block had been written explicitly as the leading clocking event.
 - Multiclocked sequences and properties are not allowed within the **clocking** block.
 - If a named sequence or property that is declared outside the **clocking** block is instantiated within the **clocking** block, the instance shall be singly clocked and its clocking event shall be identical to that of the **clocking** block.
- A contextually inferred clocking event from a procedural block supersedes a default clocking event. The contextually inferred clocking event is treated as though it had been written as the leading clocking event of any concurrent assertion statement to which the inferred clock applies.
- An explicitly specified leading clocking event in a concurrent assertion statement supersedes a default clocking event.
- A multiclocked sequence or property can inherit the default clocking event as its leading clocking event. If a multiclocked property is the maximal property of a concurrent assertion statement, then the property shall have a unique semantic leading clock (see [16.16.1](#)).

- f) If a concurrent assertion statement has no explicit leading clocking event, there is no default clocking event, and no contextually inferred clocking event applies to the assertion statement, then the maximal property of the assertion statement shall be an instance of a sequence or property for which a unique leading clocking event is determined.

The following are two example modules illustrating the application of these rules with some legal and some illegal declarations, as indicated by the comments:

```

module examples_with_default (input logic a, b, c, clk);

  property q1;
    $rose(a) |-> ##[1:5] b;
  endproperty

  property q2;
    @(posedge clk) q1;
  endproperty

  default clocking posedge_clk @(posedge clk);
    property q3;
      $fell(c) |=> q1;
      // legal: q1 has no clocking event
    endproperty

    property q4;
      $fell(c) |=> q2;
      // legal: q2 has clocking event identical to that of
      // the clocking block
    endproperty

    sequence s1;
      @(posedge clk) b[*3];
      // illegal: explicit clocking event in clocking block
    endsequence
  endclocking

  property q5;
    @(negedge clk) b[*3] |=> !b;
  endproperty

  always @(negedge clk)
  begin
    a1: assert property ($fell(c) |=> q1);
      // legal: contextually inferred leading clocking event,
      // @(negedge clk)
    a2: assert property (posedge_clk.q4);
      // legal: will be queued (pending) on negedge clk, then
      // (if matured) checked at next posedge clk (see 16.14.6)
    a3: assert property ($fell(c) |=> q2);
      // illegal: multiclocked property with contextually
      // inferred leading clocking event
    a4: assert property (q5);
      // legal: contextually inferred leading clocking event,
      // @(negedge clk)
  end

  property q6;
    q1 and q5;

```

```

endproperty

a5: assert property (q6);
    // illegal: default leading clocking event, @(posedge clk),
    // but semantic leading clock is not unique
a6: assert property ($fell(c) |=> q6);
    // legal: default leading clocking event, @(posedge clk),
    // is the unique semantic leading clock

sequence s2;
    $rose(a) ##[1:5] b;
endsequence

c1: cover property (s2);
    // legal: default leading clocking event, @(posedge clk)
c2: cover property @(negedge clk) s2;
    // legal: explicit leading clocking event, @(negedge clk)

endmodule

module examples_without_default (input logic a, b, c, clk);

    property q1;
        $rose(a) |-> ##[1:5] b;
    endproperty

    property q5;
        @(negedge clk) b[*3] |=> !b;
    endproperty

    property q6;
        q1 and q5;
    endproperty

a5: assert property (q6);
    // illegal: no leading clocking event
a6: assert property ($fell(c) |=> q6);
    // illegal: no leading clocking event

sequence s2;
    $rose(a) ##[1:5] b;
endsequence

c1: cover property (s2);
    // illegal: no leading clocking event
c2: cover property @(negedge clk) s2;
    // legal: explicit leading clocking event, @(negedge clk)

sequence s3;
    @(negedge clk) s2;
endsequence

c3: cover property (s3);
    // legal: leading clocking event, @(negedge clk),
    // determined from declaration of s3
c4: cover property (s3 ##1 b);
    // illegal: no default, inferred, or explicit leading
    // clocking event and maximal property is not an instance

```

endmodule

16.16.1 Semantic leading clocks for multiclocked sequences and properties

Throughout this subclause, s , s_1 , and s_2 denote sequences without clocking events; p , p_1 , and p_2 denote properties without clocking events; m , m_1 , and m_2 denote multiclocked sequences, q , q_1 , and q_2 denote multiclocked properties; and c , c_1 , and c_2 denote nonidentical clocking event expressions.

This subclause defines a notion of the set of semantic leading clocks for a multiclocked sequence or property.

Some sequences and properties have no explicit leading clock event. Their initial clocking event is inherited from an outer clocking event according to the flow of clocking event scope. In this case, the semantic leading clock is said to be *inherited*. For example, in the property

$$@ (c) \ s \mid=> \ p \text{ and } @ (c_1) \ p_1$$

the semantic leading clock of the subproperty p is inherited because the initial clock of p is the clock that flows across $\mid=>$.

A multiclocked sequence has a unique semantic leading clock, defined inductively as follows:

- The semantic leading clock of s is *inherited*.
- The semantic leading clock of $@ (c) \ s$ is c .
- If *inherited* is the semantic leading clock of m , then the semantic leading clock of $@ (c) \ m$ is c . Otherwise, the semantic leading clock of $@ (c) \ m$ is equal to the semantic leading clock of m .
- The semantic leading clock of (m) is equal to the semantic leading clock of m .
- The semantic leading clock of $m_1 \ \#\#1 \ m_2$ is equal to the semantic leading clock of m_1 .
- The semantic leading clock of $m_1 \ \#\#0 \ m_2$ is equal to the semantic leading clock of m_1 .

The set of semantic leading clocks of a multiclocked property is defined inductively as follows:

- The set of semantic leading clocks of **strong** (m) is $\{c\}$, where c is the unique semantic leading clock of m .
- The set of semantic leading clocks of **weak** (m) is $\{c\}$, where c is the unique semantic leading clock of m .
- The set of semantic leading clocks of p is $\{inherited\}$.
- If *inherited* is an element of the set of semantic leading clocks of q , then the set of semantic leading clocks of $@ (c) \ q$ is obtained from the set of semantic leading clocks of q by replacing *inherited* by c . Otherwise, the set of semantic leading clocks of $@ (c) \ q$ is equal to the set of semantic leading clocks of q .
- The set of semantic leading clocks of (q) is equal to the set of semantic leading clocks of q .
- The set of semantic leading clocks of **not** q is equal to the set of semantic leading clocks of q .
- The set of semantic leading clocks of q_1 **and** q_2 is the union of the set of semantic leading clocks of q_1 with the set of semantic leading clocks of q_2 .
- The set of semantic leading clocks of q_1 **or** q_2 is the union of the set of semantic leading clocks of q_1 with the set of semantic leading clocks of q_2 .
- The set of semantic leading clocks of $m \mid-> p$ is equal to the set of semantic leading clocks of m .
- The set of semantic leading clocks of $m \mid=> p$ is equal to the set of semantic leading clocks of m .
- The set of semantic leading clocks of **if** (b) q is $\{inherited\}$.
- The set of semantic leading clocks of **if** (b) q_1 **else** q_2 is $\{inherited\}$.

- The set of semantic leading clocks of **case** (b) $b_1: q_1 \dots b_n: q_n$ [**default:** q_d] **endcase** is *{inherited}*.
- The set of semantic leading clocks of **nexttime** q is *{inherited}*.
- The set of semantic leading clocks of **always** q is *{inherited}*.
- The set of semantic leading clocks of **s_eventually** q is *{inherited}*.
- The set of semantic leading clocks of q_1 **until** q_2 is *{inherited}*.
- The set of semantic leading clocks of q_1 **until_with** q_2 is *{inherited}*.
- The set of semantic leading clocks of **accept_on** (b) q is the set of semantic leading clocks of q .
- The set of semantic leading clocks of **reject_on** (b) q is the set of semantic leading clocks of q .
- The set of semantic leading clocks of **sync_accept_on** (b) q is *{inherited}*.
- The set of semantic leading clocks of **sync_reject_on** (b) q is *{inherited}*.
- The set of semantic leading clocks of a property instance is equal to the set of semantic leading clocks of the multiclocked property obtained from the body of its declaration by substituting in actual arguments.

For example, the multiclocked sequence

```
@(c1) s1 ##1 @(c2) s2
```

has c_1 as its unique semantic leading clock, while the multiclocked property

```
not (p1 and @(c2) p2)
```

has *{inherited, c₂}* as its set of semantic leading clocks.

In the presence of an incoming outer clock, the inherited semantic leading clock is always understood to refer to the incoming outer clock. Therefore, the clocking of a property q in the presence of incoming outer clock c is equivalent to the clocking of the property $@(c) q$.

A multiclocked property has a unique semantic leading clock in cases where all its leading clocks are identical. Consider the following example:

```
wire clk1, clk2;
logic a, b;
...
assign clk2 = clk1;
a1: assert property @(clk1) a and @(clk2) b; // Illegal
a2: assert property @(clk1) a and @(clk1) b; // OK
always @(posedge clk1) begin
    a3: assert property(a and @(posedge clk2)); //Illegal
    a4: assert property(a and @(posedge clk1)); // OK
end
```

The assertions a2 and a4 are legal, while the assertions a1 and a3 are not. Though both clocks of a1 have the same value, they are not identical. Therefore, a1 does not have a unique semantic leading clock. The assertions a3 and a4 have $@(\text{posedge } \text{clk1})$ as their inferred clock. This clock is not identical to $@(\text{posedge } \text{clk2})$ therefore a3 does not have a unique semantic leading clock.

16.17 Expect statement

The **expect** statement is a procedural blocking statement that allows waiting on a property evaluation. The syntax of the **expect** statement accepts a named property or a property declaration and is given in [Syntax 16-22](#).

```
expect_property_statement ::=                                     // from A.2.10
    expect ( property_spec ) action_block
```

Syntax 16-22—Expect statement syntax (excerpt from [Annex A](#))

The **expect** statement accepts the same syntax used to assert a property. An **expect** statement causes the executing process to block until the given property succeeds or fails. The statement following the **expect** is scheduled to execute after processing the Observed region in which the property completes its evaluation. When the property succeeds or fails, the process unblocks, and the property stops being evaluated (i.e., no property evaluation is started until that **expect** statement is executed again).

When executed, the **expect** statement starts a single thread of evaluation for the given property on the subsequent clocking event, that is, the first evaluation shall take place on the next clocking event. If the property fails at its clocking event, the optional **else** clause of the action block is executed. If the property succeeds, the optional pass statement of the action block is executed. The execution of pass and fail statements can be controlled by using assertion action control tasks. The assertion action control tasks are described in [20.12](#).

```
program tst;
    initial begin
        # 200ms;
        expect( @(posedge clk) a ##1 b ##1 c ) else $error( "expect failed" );
        ABC: ...
    end
endprogram
```

In the preceding example, the **expect** statement specifies a property that consists of the sequence a ##1 b ##1 c. The **expect** statement (second statement in the **initial** procedure of program `tst`) blocks until the sequence a ##1 b ##1 c is matched or is determined not to match. The property evaluation starts on the occurrence of the **posedge** `clk` event following the 200 ms delay. If the sequence is matched, the process is unblocked and continues to execute on the statement labeled ABC. If the sequence fails to match, then the **else** clause is executed, which in this case generates a run-time error. For the preceding **expect** statement to succeed, the sequence a ##1 b ##1 c must match starting on the occurrence of the **posedge** `clk` event immediately after time 200ms. The sequence will not match if a, b, or c is evaluated to be false at the first, second, or third clocking event occurrence, respectively.

The **expect** statement can appear anywhere a **wait** statement (see [9.4.3](#)) can appear. Because it is a blocking statement, the property can refer to automatic variables as well as static variables. For example, the task below waits between 1 and 10 clock ticks for the variable `data` to equal a particular value, which is specified by the automatic argument value. The second argument, `success`, is used to return the result of the **expect** statement: 1 for success and 0 for failure.

```
integer data;
...
task automatic wait_for( integer value, output bit success );
expect( @(posedge clk) ##[1:10] data == value ) success = 1;
    else success = 0;
endtask
```

```
initial begin
    bit ok;
    wait_for( 23, ok ); // wait for the value 23
    ...
end
```

16.18 Clocking blocks and concurrent assertions

If a variable used in a concurrent assertion is a **clocking** block variable, it will be sampled only in the **clocking** block.

Examples:

```
module A;
    logic a, clk;

    clocking cb_with_input @(posedge clk);
        input a;
        property p1;
            a;
        endproperty
    endclocking

    clocking cb_without_input @(posedge clk);
        property p1;
            a;
        endproperty
    endclocking

    property p1;
        @(posedge clk) a;
    endproperty

    property p2;
        @(posedge clk) cb_with_input.a;
    endproperty

    a1: assert property (p1);
    a2: assert property (cb_with_input.p1);
    a3: assert property (p2);
    a4: assert property (cb_without_input.p1);

endmodule
```

[Figure 16-17](#) explains the behavior of all the assertions. In the preceding example, a1, a2, a3, and a4 are equivalent.

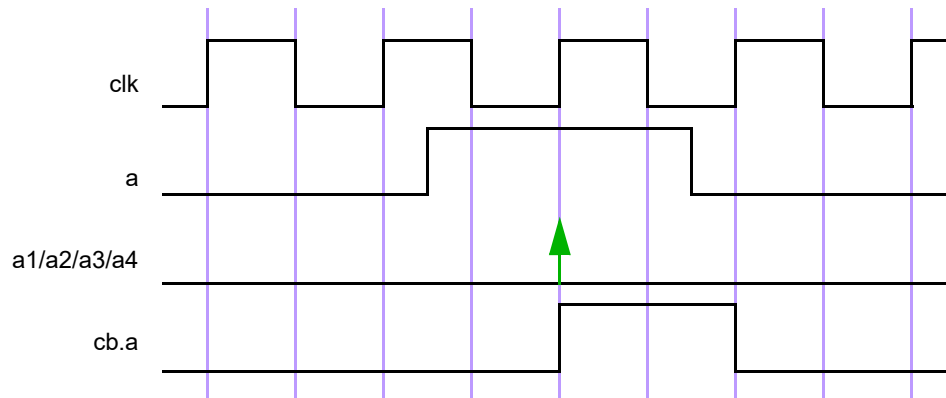


Figure 16-17—Clocking blocks and concurrent assertion